



PRO SWIFT

BOOK AND VIDEOS

Break out of beginner's Swift
with this hands-on guide

Paul Hudson

Pro Swift

Paul Hudson

Contents

Preface	6
Welcome	
Pattern Matching	11
Matching tuples	
Matching loops	
Matching optionals	
Matching ranges	
Matching enums and associated values	
Matching types	
Using the where keyword	
Syntax	33
If and switch expressions	
Guard	
Nil coalescing	
Lazy loading	
Lazy singletons	
Lazy sequences	
Destructuring	
Labeled statements	
Nested types	
Nested functions	
Documentation markup	
Types	79
Useful initializers	
Strings	
Enums	
Arrays	
Dictionaries	
Sets	
Tuples	
Ranges and RangeSet	
Generics	
Key paths	

References and Values **136**

- What's the difference between structs and classes?
- Why use structs?
- Why use classes?
- Immutability in structs and classes
- Closures are reference types
- Mixing classes and structs

Functions and Closures **155**

- Variadic functions
- Operator overloading
- Adding a new operator
- Closures: the basics
- Closure capturing
- Closure shorthand syntax
- Closures, operators, and functions
- Escaping closures
- @autoclosure

Errors **191**

- Error fundamentals
- Error propagation
- Throwing functions as parameters
- Typed throws
- try vs try? vs try!
- Assertions and preconditions
- Never and fatalError()

Functional Programming **217**

- What is functional programming?
- map()
- compactMap()
- flatMap()
- filter()
- reduce()
- sort()
- Key path expressions as functions
- Function composition
- Lazy functions
- Functors and monads

Object-oriented Programming	266
Object-oriented programming	
Encapsulation	
Inheritance	
Super methods	
Polymorphism	
Final classes	
Static properties and methods	
is-a vs has-a	
 Protocol-oriented Programming	 287
Protocol-oriented programming	
Protocol extensions in detail	
Thinking horizontally	
POP in practice	
Constrained extensions	
Example extensions	
 Afterword	 316

Preface

Welcome

This is not a book for people who are new to Swift. I realize that's obvious given that the book is called Pro Swift, but I want to make sure we're getting off on the right foot.

Instead, this is a book for people who have some real-world knowledge of the language – perhaps you've made some apps yourself, perhaps you've read through Apple's official Swift reference guide, or perhaps you even read my first book, *Hacking with iOS*.

It is *not* required that you have read *Hacking with iOS* in order to make the most of this book, but it certainly helps. At the very least you should have read the introduction chapter, which goes through the basics of Swift in about an hour of coding. If you haven't already read that chapter, I suggest you pause here then go and read it before continuing.

If you're still here, I'm assuming it means you're ready to learn about what's in this book. Well, I have some good news: I have distilled 57,000 words of knowledge into this volume, all of which is dedicated to helping you level up your Swift code.

I've tried to cover a wide range of topics so that everyone will find something of interest, which means you'll find chapters on closures, chapters on functional programming, chapters on operator overloading, and more – it's a real feast for developers who want to see what Swift is truly capable of.

I have to admit that it's easy for the Swift compiler to seem like a bossy school teacher that enjoys pointing out your mistakes. But once you start writing idiomatic Swift – Swift as it was meant to be written – suddenly you realize you have an elegant, efficient, and expressive language that will help you ship better software.

There is no right way to read Pro Swift. You can jump in at whatever chapter interests you if that's what works, although I suspect you'll find it easier just to start at the beginning and work your way through. I make no apologies for jumping around so many topics – I've tried to be as precise as possible while still giving you hands on examples you can try as you read. And please do try the examples!

Preface

One more thing: obviously I hope you enjoy all the chapters in the book, but there were a few that I particularly enjoyed writing and I hope you enjoy them as much while reading. They are “Labeled statements”, “Throwing functions as parameters”, “compactMap()”, and “Function composition” – have fun!

Xcode and Swift

I’ve written these tutorials using Xcode 16, which is available for free from the Mac App Store. If you’re using an earlier version of Xcode you should upgrade before continuing otherwise you may encounter bugs.

As for the Swift language itself, against all odds things continue to evolve quickly, so to get the most from this series you should be using Swift 6 or later.

Because this book does *not* focus on any of Apple's user interface frameworks, I would suggest you run your code in a macOS Command Line App template project just to keep things simple.

Frequent Flyer Club

You can buy Swift tutorials from anywhere, but I’m pleased, proud, and very grateful that you chose mine. I want to say thank you, and the best way I have of doing that is by giving you bonus content above and beyond what you paid for – you deserve it!

Every book contains a word that unlocks bonus content for Frequent Flyer Club members. The word for this book is **CYCLONE**. Enter that word, along with words from any other Hacking with Swift books, here: <https://www.hackingwithswift.com/frequent-flyer>.

Updates

When you buy your books from Hacking with Swift, you get Swift updates for free. You can read the full version of my update policy at <https://www.hackingwithswift.com/update-policy>, but the abridged version is this: whenever I release to the public an updated book or video to reflect these changes, all existing buyers will get that update free.

This edition has the version 2024-09-16.

Dedication

This book is dedicated to my nephew John, who is right now taking his first steps in programming. He's a smart kid with a lot of ideas, but most importantly he's massively curious about the world around him. I hope one day not too far in the future this book will be useful to him too.

Acknowledgements

In places I have used small amounts of source code from the official Swift open source project, mostly to illustrate specific techniques. This code is released under the Apache License v2.0 with the following header:

```
Copyright (c) 2014 – 2024 Apple Inc. and the Swift project
authors
Licensed under Apache License v2.0 with Runtime Library
Exception
See http://swift.org/LICENSE.txt for license information
See http://swift.org/CONTRIBUTORS.txt for the list of Swift
project authors
```

The Swift development team continue to do incredible work. My job – and yours! – would be very different without them, and it's important we thank them for their continuing efforts.

Get in touch

If you spot any typos or other mistakes, please email me at paul@hackingwithswift.com or tweet me [@twostraws](https://twitter.com/twostraws) and I'll do my best to correct them quickly.

Thank you for your support – I love programming in Swift, writing about Swift, and teaching others about Swift. Your support is what makes that possible, and it means a lot to me.

Copyright

Swift, the Swift logo, Xcode, Instruments, Cocoa Touch, Touch ID, AirDrop, iBeacon, iPhone, iPad, Safari, App Store, watchOS, tvOS, Mac and macOS are trademarks of Apple Inc., registered in the U.S. and other countries.

Pro Swift and Hacking with Swift are copyright Paul Hudson. All rights reserved. No part of this book or corresponding materials (such as text, images, or source code) may be reproduced or distributed by any means without prior written permission of the copyright owner.

Chapter 1

Pattern Matching

Matching tuples

Swift's **switch/case** is not a new concept: insert a value, then take one of several courses of action. Swift's focus on safety adds to the mix a requirement that all possible cases be catered for – something you don't get in most other languages unless you explicitly ask for it.

What makes Swift's **switch** syntax interesting is its flexible, expressive pattern matching. What makes it *doubly* interesting is that this same pattern matching is extended elsewhere, so that same flexible, expressive syntax is available in **if** conditions and **for** loops.

Admittedly, if you jump in at the deep end you're more likely to sink rather than swim, so I want to work up from basic examples. To refresh your memory, here's a basic **switch** statement:

```
let name = "twostraws"

switch name {
case "bilbo":
    print("Hello, Bilbo Baggins!")
case "twostraws":
    print("Hello, Paul Hudson!")
default:
    print("Who are you?")
}
```

It's easy enough when you're working with a simple string, but things get more complicated when working with two or more values. For example, if we wanted to validate a name as well as a password, we would evaluate them as a tuple:

```
let name = "twostraws"
let password = "fr0st1es"

switch (name, password) {
```

```

case ("bilbo", "bagglin5"):
    print("Hello, Bilbo Baggins!")
case ("twostraws", "fr0stles"):
    print("Hello, Paul Hudson!")
default:
    print("Who are you?")
}

```

You can combine the two values into a single tuple if you prefer, like this:

```

let authentication = (name: "twostraws", password: "fr0stles")

switch authentication {
case ("bilbo", "bagglin5"):
    print("Hello, Bilbo Baggins!")
case ("twostraws", "fr0stles"):
    print("Hello, Paul Hudson!")
default:
    print("Who are you?")
}

```

In this instance, both parts of the tuple must match the **case** in order for it to be executed.

When working with tuples, there are some occasions when you want a partial match: you care what some values are but don't care about others. In this situation, use underscores to represent "any value is fine", like this:

```

let authentication = (name: "twostraws", password: "fr0stles",
ipAddress: "127.0.0.1")

switch authentication {
case ("bilbo", "bagglin5", _):
    print("Hello, Bilbo Baggins!")
}

```

Pattern Matching

```
case ("twostraws", "fr0stles", _):  
    print("Hello, Paul Hudson!")  
default:  
    print("Who are you?")  
}
```

Remember: Swift will take the first matching case it finds, so you need to ensure you look for the most specific things first. For example, the code below would print “You could be anybody!” because the first **case** matches immediately, even though later cases are “better” matches because they match more things:

```
switch authentication {  
case (_, _, _):  
    print("You could be anybody!")  
case ("bilbo", "bagglin5", _):  
    print("Hello, Bilbo Baggins!")  
case ("twostraws", "fr0stles", _):  
    print("Hello, Paul Hudson!")  
default:  
    print("Who are you?")  
}
```

Finally, if you want to match only part of a tuple, but still want to know what the other part was, you should use **let** syntax.

```
switch authentication {  
case ("bilbo", "bagglin5", _):  
    print("Hello, Bilbo Baggins!")  
case ("twostraws", let password, _):  
    print("Hello, Paul Hudson: your password was \(password)!")  
default:  
    print("Who are you?")  
}
```

But we can do better: tuples are like any other data structure in that they can be created using dynamic code. This is particularly useful when you want to narrow the range of values in a tuple down to a smaller subset so that you need only a handful of **case** statements.

To give you a practical example, consider the “fizzbuzz” test: write a function that accepts any number, and returns “Fizz” if the number is evenly divisible by 3, “Buzz” if it’s evenly divisible by 5, “FizzBuzz” if its evenly divisible by 3 *and* 5, or the original input number in other cases.

We can calculate a tuple to solve this problem, then pass that tuple into a **switch** block to create the correct output. Here’s the code:

```
func fizzbuzz(number: Int) -> String {
    switch (number.isMultiple(of: 3), number.isMultiple(of: 5)) {
    case (true, false):
        "Fizz"
    case (false, true):
        "Buzz"
    case (true, true):
        "FizzBuzz"
    case (false, false):
        String(number)
    }
}

print(fizzbuzz(number: 15))
```

This approach breaks down a large input space – any number – into simple combinations of true and false, and we then use tuple pattern matching in the case statements to select the correct output.

Matching loops

As you've seen, pattern matching using part of a tuple is easy enough: you either tell Swift what should be matched, use **let** to bind a value to a local constant, or use **_** to signal that you don't care what a value is.

We can use this same approach when working with loops, which allows us to loop over items only if they match the criteria we specify. Let's start with a basic example again:

```
let twostraws = (name: "twostraws", password: "fr0stles")
let bilbo = (name: "bilbo", password: "baggln5")
let taylor = (name: "taylor", password: "fr0stles")

let users = [twostraws, bilbo, taylor]

for user in users {
    print(user.name)
}
```

That creates an array of tuples, then loops over each one and prints its **name** value.

Just like the **switch** blocks we looked at earlier, we can use **case** with a tuple to match specific values inside the tuples. Add this code below the previous loop:

```
for case ("twostraws", "fr0stles") in users {
    print("User twostraws has the password fr0stles")
}
```

We also have identical syntax for binding local constants to the values of each tuple, like this:

```
for case (let name, let password) in users {
    print("User \(name) has the password \(password)")
}
```


Usually, though, it's preferable to re-arrange the **let** to this:

```
for case let (name, password) in users {  
    print("User \(name) has the password \(password)")  
}
```

The magic comes when you combine these two approaches, and again is syntactically identical to a **switch** example we already saw:

```
for case let (name, "fr0st1es") in users {  
    print("User \(name) has the password \"fr0st1es\")  
}
```

That filters the **users** array so that only items with the password “fr0st1es” will be used in the loop, then creates a **name** constant inside the loop for you to work with.

Don't worry if you're staring at **for case let** and seeing three completely different keywords mashed together: it's not obvious what it does until someone explains it to you, and it will take a little time to sink in. But we're only getting started...

Matching optionals

Swift has two ways of matching optionals, and you're likely to meet both. Helpfully, both work just as well with **switch** and loops, so they are useful in lots of places.

First up is using **.some** and **.none** to match “has a value” and “has no value”, and in the code below this is used to check for values and bind them to local constants:

```
let name: String? = "twostraws"
let password: String? = "fr0stles"

switch (name, password) {
case let (.some(name), .some(password)):
    print("Hello, \(name)")
case let (.some(name), .none):
    print("Please enter a password.")
default:
    print("Who are you?")
}
```

That code is made more confusing because **name** and **password** are used for the input constants as well as the locally bound constants. They are different things, though, which is why **print("Hello, \(name)")** won't print **Hello, Optional("twostraws")** – the **name** being used is the locally bound unwrapped optional.

If it's easier to read, here's the same code with different names used for the matched constants:

```
switch (name, password) {
case let (.some(matchedName), .some(matchedPassword)):
    print("Hello, \(matchedName)")
case let (.some(matchedName), .none):
    print("Please enter a password.")
default:
    print("Who are you?")
}
```

```
print("Who are you?")
}
```

The second way Swift matches optionals is using much simpler syntax, although if you have a fear of optionals this might only make it worse:

```
switch (name, password) {
case let (name?, password?):
    print("Hello, \(name)")
case let (name?, nil):
    print("Please enter a password.")
default:
    print("Who are you?")
}
```

This time the question marks work in a similar way as optional chaining: continue only if a value was found.

Both of these methods work equally well in **for case let** code. The code below uses them both to filter out **nil** values in a loop:

```
import Foundation
let data: [Any?] = ["Bill", nil, 69, "Ted"]

for case let .some(datum) in data {
    print(datum)
}

for case let datum? in data {
    print(datum)
}
```

Matching ranges

Swift supports both closed ranges (...) and half-open ranges (.. $<$) with its pattern matching, although the syntax might make you do a double take at first.

You're probably already using pattern matching with ranges, usually with code something like this:

```
let age = 36

switch age {
case 0.. $<$ 18:
    print("You have the energy and time, but not the money")
case 18.. $<$ 70:
    print("You have the energy and money, but not the time")
default:
    print("You have the time and money, but not the energy")
}
```

A very similar syntax is also available for regular conditional statements – we could rewrite that code like this:

```
if case 0.. $<$ 18 = age {
    print("You have the energy and time, but not the money")
} else if case 18.. $<$ 70 = age {
    print("You have the energy and money, but not the time")
} else {
    print("You have the time and money, but not the energy")
}
```

That produces identical results to the **switch** block while using similar syntax, but I'm not a big fan of this approach.

The reason for my dislike is simple readability: I don't think "if case 0 up to 18 equals age" makes sense if you don't already know what it means. A much nicer approach is to use the pattern match operator, `~=`, which would look like this:

```
if 0..<18 ~= age {
    print("You have the energy and time, but not the money")
} else if 18..<70 ~= age {
    print("You have the energy and money, but not the time")
} else {
    print("You have the time and money, but not the energy")
}
```

Now the condition reads "if the range 0 up to 18 matches age", which I think makes a lot more sense.

An even cleaner solution becomes clear when you remember that `0..<18` creates an instance of a **Range** struct, which has its own set of methods. Right now, its `contains()` method is particularly useful: it's longer to type than `~=` but it's significantly easier to understand:

```
if (0..<18).contains(age) {
    print("You have the energy and time, but not the money")
} else if (18..<70).contains(age) {
    print("You have the energy and money, but not the time")
} else {
    print("You have the time and money, but not the energy")
}
```

You can combine this range matching into our existing tuple matching code, like this:

```
let user = (name: "twostraws", password: "fr0stles", age: 36)

switch user {
case let (name, _, 0..<18):
```

Pattern Matching

```
    print("\(name) has the energy and time, but not the money")
case let (name, _, 18..70):
    print("\(name) has the money and energy, but not the time")
case let (name, _, _):
    print("\(name) has the time and money, but not the energy")
}
```

That last case binds the user's name to a local constant called **name** irrespective of the two other values in the tuple. This is a catch all, but because Swift looks for the *first* matching case this won't conflict with the other two in the **switch** block.

Matching enums and associated values

Matching simple enums is straightforward using `==` or similar, but things get a little trickier when dealing with associated values – particularly when you're trying to filter based on those values.

Basic enum matching looks like this:

```
enum WeatherType {
    case cloudy
    case sunny
    case windy
}

let today = WeatherType.cloudy

switch today {
case .cloudy:
    print("It's cloudy")
case .windy:
    print("It's windy")
default:
    print("It's sunny")
}
```

You'll also have used enums in basic conditional statements, like this:

```
if today == .cloudy {
    print("It's cloudy")
}
```

Pattern Matching

As soon as you add associated values, things get more complicated because you can use them, filter on them, or ignore them depending on your goal.

First up, the easiest option: creating an associated value but ignoring it:

```
enum WeatherType {
    case cloudy(coverage: Int)
    case sunny
    case windy
}

let today = WeatherType.cloudy(coverage: 100)

switch today {
case .cloudy:
    print("It's cloudy")
case .windy:
    print("It's windy")
default:
    print("It's sunny")
}
```

Using this approach, the actual **switch** code is unchanged.

Second: creating an associated value and using it. This uses the same local constant bind we've seen several times now:

```
enum WeatherType {
    case cloudy(coverage: Int)
    case sunny
    case windy
}
```



```

let today = WeatherType.cloudy(coverage: 100)

switch today {
case .cloudy(let coverage):
    print("It's cloudy with \(coverage)% coverage")
case .windy:
    print("It's windy")
default:
    print("It's sunny")
}

```

Lastly: creating an associated type, binding a local constant to it, but also using that binding to filter for specific values. This uses the **where** keyword to create a requirements clause that clarifies what you're looking for. In our case, the code below prints two different messages depending on the associated value that is used with **cloudy**:

```

enum WeatherType {
    case cloudy(coverage: Int)
    case sunny
    case windy
}

let today = WeatherType.cloudy(coverage: 100)

switch today {
case .cloudy(let coverage) where coverage < 100:
    print("It's cloudy with \(coverage)% coverage")
case .cloudy(let coverage) where coverage == 100:
    print("You must live in the UK")
case .windy:
    print("It's windy")
default:

```

Pattern Matching

```
    print("It's sunny")
}
```

As promised I'm building up from basic examples, but if you're ready I want to show you how to combine two of these techniques together: associated values and range matching. The code below now prints four different messages: one when coverage is 0, one when it's 100, and two more using ranges from 1 to 50 and 51 to 99:

```
enum WeatherType {
    case cloudy(coverage: Int)
    case sunny
    case windy
}

let today = WeatherType.cloudy(coverage: 100)

switch today {
case .cloudy(let coverage) where coverage == 0:
    print("You must live in Death Valley")
case .cloudy(let coverage) where (1...50).contains(coverage):
    print("It's a bit cloudy, with \(coverage)% coverage")
case .cloudy(let coverage) where (51...99).contains(coverage):
    print("It's very cloudy, with \(coverage)% coverage")
case .cloudy(let coverage) where coverage == 100:
    print("You must live in the UK")
case .windy:
    print("It's windy")
default:
    print("It's sunny")
}
```

If you want to match associated values in a loop, adding a **where** clause is the wrong approach.

In fact, this kind of code won't even compile:

```
let forecast: [WeatherType] = [.cloudy(coverage:
40), .sunny, .windy, .cloudy(coverage: 100), .sunny]

for day in forecast where day == .cloudy {
    print(day)
}
```

That code would be fine without associated values, but because the associated value has meaning the **where** clause isn't up to the job – it has no way to say “and bind the associated value to a local constant.” Instead, you're back to **case let** syntax, like this:

```
let forecast: [WeatherType] = [.cloudy(coverage:
40), .sunny, .windy, .cloudy(coverage: 100), .sunny]

for case let .cloudy(coverage) in forecast {
    print("It's cloudy with \(coverage)% coverage")
}
```

If you know the associated value and want to use it as a filter, the syntax is similar:

```
let forecast: [WeatherType] = [.cloudy(coverage:
40), .sunny, .windy, .cloudy(coverage: 100), .sunny]

for case .cloudy(40) in forecast {
    print("It's cloudy with 40% coverage")
}
```

Matching types

You should already know the **is** keyword for matching, but you might not know that it can be used as pattern matching in loops and **switch** blocks. I think the syntax is quite pleasing, so I want to demonstrate it just briefly.

For this example, make a new Xcode playground for iOS using the Blank template, so we can access UIKit.

```
let view: AnyObject = UIButton()

switch view {
case is UIButton:
    print("Found a button")
case is UILabel:
    print("Found a label")
case is UISwitch:
    print("Found a switch")
case is UIView:
    print("Found a view")
default:
    print("Found something else")
}
```

I've used UIKit as an example because you should already know that **UIButton** inherits from **UIView**, and I need to give you a big warning...

Remember: Swift will take the first matching case it finds, and **is** returns true if an object is a specific type or one of its parent classes. So, the above code will print “Found a button”, whereas the below code will print “Found a view”:

```
let view: AnyObject = UIButton()
```

```

switch view {
case is UIView:
    print("Found a view")
case is UIButton:
    print("Found a button")
case is UILabel:
    print("Found a label")
case is UISwitch:
    print("Found a switch")
default:
    print("Found something else")
}

```

To give you a more useful example, you can use this approach to loop over all subviews in an array and filter for labels:

```

for label in view.subviews where label is UILabel {
    print("Found a label with frame \(label.frame)")
}

```

Even though **where** ensures only **UILabel** objects are processed in the loop, it doesn't actually do any typecasting. This means if you wanted to access a label-specific property of **label**, such as its **text** property, you need to typecast it yourself. In this situation, using **for case let** instead is easier, as this filters and typecasts in one:

```

for case let label as UILabel in view.subviews {
    print("Found a label with text \(label.text)")
}

```

Using the where keyword

To wrap up pattern matching, I want to demonstrate a couple of interesting ways to use **where** clauses so that you can get an idea of what it's capable of.

First, an easy one: loop over an array of numbers and print only the even ones. This is trivial using **where** and **isMultiple(of:)**, but it demonstrates that your **where** clause can contain calculations:

```
let numbers = [1, 2, 3, 7, 8, 9]

for number in numbers where number.isMultiple(of: 2) {
    print(number)
}
```

You can also call methods, like this:

```
let celebrities = ["Michael Jackson", "Taylor Swift", "Michael Caine", "Adele Adkins", "Michael Jordan"]

for name in celebrities where name.starts(with: "Michael") {
    print(name)
}
```

That will print "Michael Jackson", "Michael Caine", and "Michael Jordan". If you want to make your **where** clause more complicated, just add operators such as **&&**:

```
let celebrities = ["Michael Jackson", "Taylor Swift", "Michael Caine", "Adele Adkins", "Michael Jordan"]

for name in celebrities where name.starts(with: "Michael") &&
name.count == 13 {
    print(name)
}
```

```
}
```

That will print “Michael Caine”.

While it’s possible to use **where** to strip out optionals, I wouldn’t recommend it. Consider the example below:

```
let celebrities: [String?] = ["Michael Jackson", nil, "Michael
Caine", nil, "Michael Jordan"]

for name in celebrities where name != nil {
    print(name)
}
```

That certainly works, but it does nothing about the optionality of the strings in the loop so it prints out this:

```
Optional("Michael Jackson")
Optional("Michael Caine")
Optional("Michael Jordan")
```

Instead, use **for case let** to handle optionality, and use **where** to focus on filtering values. Here’s the preferred way of writing that loop:

```
for case let name? in celebrities {
    print(name)
}
```

When that runs, **name** will only contain the strings that had values, so its output will be:

```
Michael Jackson
Michael Caine
Michael Jordan
```

Pattern Matching

Chapter 2

Syntax

If and switch expressions

Writing Swift is very much like writing a spoken language such as English: when you're learning you tend to be quite verbose in what you say, but when you're more experienced your code is usually shorter and snappier – you're not *lazy*, but you certainly find yourself avoiding writing any code that doesn't need to be there.

One common example of this is how Swift lets us avoid writing the **return** keyword in single-expression functions. Expressions are pieces of code that result in a value, such as **5 + 5** or **someValue * 2**. On the other hand, statements do work such as creating variables or running loops.

Swift always allowed us to omit the **return** keyword in closures, so this kind of code has been legal since day 1:

```
let numbers = [1, 2, 3]
let doubled = numbers.map { $0 * 2 }
```

That **map()** method takes each number from **numbers**, doubles it, then returns it – the **return** keyword isn't needed.

From Swift 5.1 onwards, this was also allowed in functions and computed properties that contained a single expression:

```
func double(_ number: Int) -> Int {
    number * 2
}
```

This was important for SwiftUI, where it's common to write code like this:

```
struct ContentView: View {
    var body: some View {
        VStack {
            Text("Hello, world!")
        }
    }
}
```

```

        Image(systemName: "globe")
    }
    .padding()
}
}

```

That **body** property needs to return some kind of view, but writing **return VStack {...** would be kind of clumsy.

All this matters because in Swift 5.9 and later **if** and **switch** gained the ability to be used as expressions, when previously they were statements. This allows two important pieces of syntax you'll see a lot:

1. Omitting the **return** keyword in more places.
2. Placing **if** and **switch** checks *after* an assignment.

So, again: *expressions* are things that produce a value, such as **5 + 5**, which means we can write code like this:

```

func get10() -> Int {
    5 + 5
}

```

We don't need to write **return 5 + 5** there, because **5 + 5** is an expression, and Swift allows us to omit the **return** keyword when a function has just one expression.

But because **if** conditions are also expressions, this is also allowed:

```

func get10Or20() -> Int {
    if Bool.random() {
        10
    } else {
        20
    }
}

```

Syntax

```
}
```

That **if** is an expression, meaning that it produces a value. In this case, it happens to be one of two values depending on how the condition evaluates, but it still produces a single value, which means no **return** keyword is required in there.

To be clear, this is only possible if both branches of your condition contain a single expression – if you try to include any kind of work in there, you need to use **return**.

For example, if you had a constant being created in one of the two branches, you need **return**:

```
func get10Or20() -> Int {
    if Bool.random() {
        let sum = 5 + 5
        return sum
    } else {
        return 20
    }
}
```

This is also possible with **switch**. For example, we could a simple **Weather** enum such as this one:

```
enum Weather {
    case rainy, snowy, sunny, windy
}
```

Now we could write a method to return different items to pack based on the weather:

```
func getItem(for weather: Weather) -> String {
    switch weather {
    case .rainy:
        return "Umbrella"
    case .snowy:
```

```

        return "Sweater"
    case .sunny:
        return "Sunscreen"
    case .windy:
        return "Jacket"
    }
}

```

But as you can see, each of those cases is a single expression – it's just one value being sent back for each possible type of weather. So, we can drop the **return** keywords like this:

```

func getItem(for weather: Weather) -> String {
    switch weather {
    case .rainy:
        "Umbrella"
    case .snowy:
        "Sweater"
    case .sunny:
        "Sunscreen"
    case .windy:
        "Jacket"
    }
}

```

This approach is so useful that it's common to remove the line breaks after the colons, like this:

```

func getItem(for weather: Weather) -> String {
    switch weather {
    case .rainy: "Umbrella"
    case .snowy: "Sweater"
    case .sunny: "Sunscreen"
    case .windy: "Jacket"
    }
}

```

Syntax

```
}
```

Lining code up like that helps readability, so you'll see it a lot.

Now, where things get more interesting is when we place **if** and **switch** checks *after* an assignment. Expressions produce values, which means we use them when making constants and variables, like this:

```
let score = 5 + 5
let passedTest = true
```

Because **if** and **switch** can be used as expressions, you can use them in place of **5 + 5** or **true** in that code.

For example, we could write this:

```
let passedTest = if score > 8 {
    true
} else {
    false
}
```

This usually causes folks to do a bit of a double-take at first, but honestly this style of Swift isn't that common just yet so don't feel obliged to use it.

You might think this syntax works the same as Swift's ternary conditional operator, but the two are not identical, and there is one place in particular that might catch you out – you can see it in this code:

```
let customerRating = 4
let bonusMultiplier1 = customerRating > 3 ? 1.5 : 1
let bonusMultiplier2 = if customerRating > 3 { 1.5 } else
{ 1.0 }
```

Both those calculations produce a **Double** with the value of 1.5, but pay attention to the alternative value for each of them: for the ternary option I've written 1, and for the if expression I've written 1.0.

This is intentional: when using the ternary Swift checks the types of both values at the same time and so automatically considers 1 to be 1.0, whereas with the if expression the two options are type checked independently: if we use 1.5 for one case and 1 for the other then we'll be sending back a **Double** and an **Int**, which isn't allowed.

If and switch expressions are one of those features you might choose not to *write* much, but you should certainly be sure you know how to *read* them so you don't hit mental speed bumps when reading someone else's code.

Guard

The **guard** keyword has been with us since early Swift, but because it does four things in one you'd be forgiven for not using it fully.

The first use is the most obvious: **guard** is used for early returns, which means you exit a function if some preconditions are not satisfied. For example, we could write a rather biased function to give an award to a named person:

```
func giveAward(to name: String) {  
    guard name == "Taylor Swift" else {  
        print("No way!")  
        return  
    }  
  
    print("Congratulations, \(name)!")  
}  
  
giveAward(to: "Taylor Swift")
```

Using that **guard** statement in the **giveAward(to:)** method ensures that only Taylor Swift wins awards. It's biased like I said, but the precondition is clear and this code will only run when requirements I have put in place are satisfied.

This initial example looks almost identical to using **if**, but **guard** has one massive advantage: it makes your intention clear, not only to people but also to the compiler. This is an early return, meaning that you want to exit the method if your preconditions aren't satisfied. Using **guard** makes that clear: this condition isn't part of the functionality of the method, it's just there to ensure the actual code is safe to run.

It's also clear to the compiler, meaning that if you remove the **return** statement your code will no longer build – Swift knows this is an early return, so it will not let you forget to exit.

The second use of guard is a happy side effect of the first: using **guard** and early returns

allows you to reduce your indent level. Some developers believe very strongly that early returns must never be used, and instead each function should return from only one place. This forces extra indents in the main body of your function code, something like this:

```
func giveAward(to name: String) -> String {
    let message: String

    if name == "Taylor Swift" {
        message = "Congratulations, \(name)!"
    } else {
        message = "No way!"
    }

    return message
}

giveAward(to: "Taylor Swift")
```

With **guard**, you get your preconditions resolved immediately, and lose the extra indent – hurray for neat code!

The third thing that **guard** brings us is a visibility increase for the happy path. This is a common concept in software design and testing, and refers to the path that your code will take when no exceptions or errors happen. Thanks to **guard**, common errors are removed immediately, and the remainder of your code might all be the happy path.

That's all the easy stuff out of the way, but **guard** still has one more feature I want to discuss, and it's an important differentiator between **guard** and **if**: when you use **guard** to check and unwrap an optional, that optional stays in scope.

To demonstrate this, I'm going to rewrite the **giveAward(to:)** method so that it takes an optional string:

```
func giveAward(to name: String?) {
```

Syntax

```
guard let name else {  
    print("No one won the award")  
    return  
}  
  
print("Congratulations, \(name)!")  
}
```

With a regular **if let**, the **name** constant would only be usable inside the braces that belong to **guard**. However, **guard** keeps its optional unwraps in scope, so **name** stays around for the second **print()** statement. This code reads “try to unwrap name into a new constant of the same name so I can use it, but if you can’t then print a message and exit.”

There’s one last feature of **guard** I want to touch on, but it’s not new. Instead, it’s just a different way of using what you already know.

The feature is this: **guard** lets you exit *any* scope if preconditions fail, not just functions and methods. This means you can exit a **switch** block or a loop by using **guard**, and it carries the same meaning: the contents of this scope should only be executed if these preconditions are true.

To give you a simple example, this loop counts from 1 to 100, printing out all the numbers that are evenly divisible by 8:

```
for i in 1...100 {  
    guard i.isMultiple(of: 8) else { continue }  
  
    print(i)  
}
```

Can you rewrite that using **where**? Give it a try – it’s easier than you think!

Nil coalescing

Swift optionals are one of the fundamental ways it guarantees program safety: a variable can only be used if it definitely has a value. The problem is that optionals make your code a bit harder to read and write, because you need to unwrap them safely.

One alternative is to explicitly unwrap optionals using `!`. This is also known as the “crash operator” because if you use `!` with an optional that is `nil`, your program will die immediately and your users will be paying for blood.

A smarter alternative is the nil coalescing operator, `??`, which allows you to access an optional and provide a default value if the optional is `nil`.

Consider this optional:

```
let name: String? = "Taylor"
```

That’s a constant called **name** that contains either a string or `nil`. If you try to print that using `print(name)` you’ll see `Optional("Taylor")` rather than just “Taylor”, which isn’t really what you want.

Using nil coalescing allows us to use an optional’s value or provide a default value if it’s `nil`. So, you could write this:

```
let name: String? = "Taylor"
let unwrappedName = name ?? "Anonymous"
print(unwrappedName)
```

That will print “Taylor”: **name** was a `String?`, but **unwrappedName** is guaranteed to be a regular `String` – not optional – because of the nil coalescing operator. To see the default value in action, try this instead:

```
let name: String? = nil
let unwrappedName = name ?? "Anonymous"
```

Syntax

```
print(unwrappedName)
```

That will now print “Anonymous”, because the default value is used instead.

Of course, you don’t need a separate constant when using nil coalescing – you can write it inline, like this:

```
let name: String? = "Taylor"
print(name ?? "Anonymous")
```

As you can imagine, nil coalescing is great for ensuring sensible values are in place before you use them, but it’s particularly useful in removing some optionality from your code. For example:

```
func returnsOptionalName() -> String? {
    nil
}

let returnedName = returnsOptionalName() ?? "Anonymous"
print(returnedName)
```

Using this approach, **returnedName** is a **String** rather than a **String?** because it’s guaranteed to have a value.

So far, so straightforward. However, nil coalescing gets more interesting when you combine it with the **try?** keyword.

Consider a simple app that lets a user type and save text. When the app runs, it wants to load whatever the user typed previously, so it probably uses code like this:

```
do {
    let savedText = try String(contentsOfFile: "saved.txt",
encoding: .utf8)
    print(savedText)
```

```

} catch {
    print("Failed to load saved text.")
}

```

If the file exists, it will be loaded into the **savedText** constant. If not, the **contentsOfFile** initializer will throw an error, and “Failed to load saved text” will be printed. In practice, you’d want to extend this so that **savedText** always has a value, so you end up with something like this:

```

let savedText: String

do {
    savedText = try String(contentsOfFile: "saved.txt")
} catch {
    print("Failed to load saved text.")
    savedText = "Hello, world!"
}

print(savedText)

```

That’s a lot of code and it doesn’t really accomplish very much. Fortunately, we can do better: nil coalescing. Remember, **try** has three variants: **try** attempts some code and might throw an exception, **try!** attempts some code and crashes your app if it fails, and **try?** attempts some code and returns nil if the call failed.

That last one is where nil coalescing steps up to the plate, because this exactly matches our previous examples: we want to work with an optional value, and provide a sensible default if the optional is nil. In fact, using nil coalescing we can rewrite all that into just two lines of code:

```

let savedText = (try? String(contentsOfFile: "saved.txt",
encoding: .utf8)) ?? "Hello, world!"
print(savedText)

```

Syntax

That means “try loading the file, but if loading fails use this default text instead” – a neater solution and much more readable.

Combining **try?** with nil coalescing is perfect for situations when a failed **try** isn’t an error, and I think you’ll find this pattern useful in your own code.

Just to finish off, I want to add that nil coalescing has a second form that accept an optional on both sides, which allows you to chain together multiple possibilities: one optional, then a second optional, then a third non-optional. Swift will try them all one by one, left to right.

Lazy loading

Lazy loading is one of the most important, system-wide performance optimizations that Swift coders work with. It's endemic in iOS, as anyone who has tried to manipulate a view controller's view before it's shown can tell you.

Objective-C didn't have a concept of lazy properties, so you had to write your own boilerplate code each time you wanted this behavior. Happily, Swift has it baked right in, so you can claim immediate performance benefits with hardly any code.

But first: a reminder of what lazy properties are. Consider this class:

```
class Singer {
    let name: String

    init(name: String) {
        self.name = name
    }

    func reversedName() -> String {
        "\ (name.uppercased()) backwards is \
(String(name.uppercased()).reversed()) "
    }
}

let taylor = Singer(name: "Taylor Swift")
print(taylor.reversedName())
```

That will print “TAYLOR SWIFT backwards is TFIWS ROLYAT” when run.

So each **Singer** has a single property called **name**, and a single method that does a small amount of processing to that property. Obviously in your own code these functions are likely to do more significant work, but I'm trying to keep it simple here.

Syntax

Every time you want to print the message “TAYLOR SWIFT backwards is TFIWS ROLYAT” you need to call the **reversedName()** method – the work it does isn’t stored, and if that work is non-trivial then calling the method repeatedly is wasteful.

An alternative is to create an additional property to store **reversedName** so that it’s calculated only once, like this:

```
class Singer {
    let name: String
    let reversedName: String

    init(name: String) {
        self.name = name
        reversedName = "\(name.uppercased()) backwards is \
(String(name.uppercased()).reversed())"
    }
}

let taylor = Singer(name: "Taylor Swift")
print(taylor.reversedName)
```

That’s a performance improvement for situations when you use **reversedName** a lot, but now causes your code to run *slower* if you never use **reversedName** – it gets calculated regardless of whether it’s used, whereas when **reversedName()** was a method it would only be calculated when called.

Lazy properties are the middle ground: they are properties that are calculated only once and stored, but never calculated if they aren’t used. So if your code uses a lazy property repeatedly you only pay the performance cost once, and if they are never used then the code is never run.

Lazy closures

The easiest way to get started with the **lazy** keyword is using closures. Yes, I know it’s rare to

see “closures” and “easiest” in the same sentence, but there’s a reason this book isn’t called “Newbie Swift”!

The syntax here is a little unusual at first:

```
lazy var yourVariableName: SomeType = {
    SomeType(whatever: "foobar")
}()
```

Yes, you often need to explicitly declare the type. Yes, you need that = sign. Yes, you need the parentheses after the closing brace. It’s a little unusual, like I said, but it’s all there for a reason: you’re creating the closure, applying it immediately (rather than later on), and assigning its result back to **yourVariableName**.

Using this approach, we can convert our **reversedName()** method into a lazy property like this:

```
class Singer {
    let name: String

    init(name: String) {
        self.name = name
    }

    lazy var reversedName = {
        "\ (name.uppercased()) backwards is \
(String(name.uppercased()).reversed()) "
    }()
}

let taylor = Singer(name: "Taylor Swift")
print(taylor.reversedName)
```

Syntax

Note: as it's now a property rather than a method, we need to use **print(taylor.reversedName)** rather than **print(taylor.reversedName())** to access the value.

That's it: the property is now lazy, which means the code inside the closure will be executed only the first time we read the **reversedName** property.

“But Paul,” I hear you say, “you’re using **name** inside a closure that’s owned by a class instance – why are you giving me retain cycles?” Don’t worry: this code is quite safe. Swift is smart enough to realize what’s going on, and no retain cycle will be created.

Under the hood, any closure like this – one that is immediately applied – is considered to be “non-escaping”, which in our situation means it won’t be used anywhere else. That is, this closure can’t be stored as a property and called later on. Not only does this automatically ensure **self** is considered to be **unowned**, but it also enables the Swift compiler to make some extra optimizations because it has more information about the closure.

Lazy methods

A common complaint people have when using **lazy** is that it clutters up their code: rather than having a neat separation of properties and methods, lazy properties become a gray area where properties and functionality mix together. There is a simple solution to this: create methods to separate your lazy properties from the code they rely on.

If you want to use this approach, I suggest you mark as private the separate method you create so that it doesn’t get used by accident. Something like this ought to do the trick:

```
class Singer {
    let name: String

    init(name: String) {
        self.name = name
    }

    lazy var reversedName = self.getReversedName()
```

```
private func getReversedName() -> String {  
    "\(name.uppercased()) backwards is \  
(String(name.uppercased().reversed()))"  
}  
}
```

```
let taylor = Singer(name: "Taylor Swift")  
print(taylor.reversedName)
```

Lazy singletons

Singletons are one of several common coding patterns that generate strong reactions when mentioned. If you're not familiar with them, a singleton is a value or object that is designed (and coded) to be created only once, and shared across a program.

For example, if your app uses a logger, you might create a logger object once when the app runs, and have all your other code use that shared instance.

The reason some folks don't like singletons is a simple one: they are all too often used like global variables. Many people will preach that global variables are bad then happily abuse singletons in much the same way, which is just sloppy.

That being said, there are good reasons for using singletons, and indeed Apple uses them on occasion. If your object literally can only exist once – such as an instance of **UIApplication** – then singletons make sense. On iOS, things like **UIDevice** make sense as singletons, again because they can exist only once. Singletons are also useful (at least compared to global variables!) if you want to add extra code when they are used.

So: singletons have a place, as long as you consider their use carefully. If you think singletons are the perfect choice for you, I have some good news: Swift makes singletons really easy.

To give you a practical example, we're going to create a **Singer** class that will have a **MusicPlayer** class as a property. This needs to be a singleton, because no matter how many singers our app tracks, we want all their songs to go through the same music player so that music doesn't overlap.

Here's the **MusicPlayer** class:

```
class MusicPlayer {
    init() {
        print("Ready to play songs!")
    }
}
```

It doesn't do anything other than print a message when it's created.

Here's the basic **Singer** class that also does nothing other than print a message when it's created:

```
@MainActor
class Singer {
    init() {
        print("Creating a new singer")
    }
}
```

Tip: I declared that as being bound to the main actor so this sample works, but in your own code you might want a different actor

Now for the singleton: if we want to give our **Singer** class a **MusicPlayer** singleton property, we need to add just one line of code inside the **Singer** class:

```
static let musicPlayer = MusicPlayer()
```

That's it. The **static** part means this property is shared by the class rather than instances of the class, meaning that you use **Singer.musicPlayer** rather than **taylor.musicPlayer**. The **let** part of course means that it's a constant.

You might be wondering why I've put this directly after the chapter on lazy properties, and it's time to find out – try this:

```
class MusicPlayer {
    init() {
        print("Ready to play songs!")
    }
}
```

Syntax

```
class Singer {  
    static let musicPlayer = MusicPlayer()  
  
    init() {  
        print("Creating a new singer")  
    }  
}  
  
let taylor = Singer()
```

When it runs, the output is “Creating a new singer” – the “Ready to play songs!” message won’t appear. If you add one more line of code, only then will the message appear:

```
Singer.musicPlayer
```

Yes: all Swift **static let** singletons are automatically lazy – they only get created when they are needed. It’s so easy to do, and yet perfectly efficient too. Thanks, Swift team!

Lazy sequences

Now that you understand lazy properties, I want to explain briefly the usefulness of lazy sequences. These are similar to lazy properties in that they delay work until necessary, but they aren't *quite* as efficient as you'll see shortly.

Let's start with a trivial example: the Fibonacci sequence. As a reminder, this is a sequence of numbers starting with 0 and 1, where every following number is the sum of adding the previous two. So the sequence goes 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, and so on.

We can write a naïve function that calculates the Fibonacci number at a particular point in the sequence like this:

```
func fibonacci(of num: Int) -> Int {
    if num < 2 {
        num
    } else {
        fibonacci(of: num - 1) + fibonacci(of: num - 2)
    }
}
```

This is a recursive function: it calls itself. It's a naïve implementation because it doesn't cache the results as it goes, which means all the adding done by **fibonacci(of: num - 1)** won't get re-used by **fibonacci(of: num - 2)** even though it could be.

However, this implementation is perfect for demonstrating the usefulness (and drawbacks!) of lazy sequences.

To try this next piece of course, I'd like you to create a new, blank Xcode playground, so we can track exactly how often each line of code is run:

```
var count = 0

func fibonacci(of num: Int) -> Int {
```

Syntax

```
count += 1

if num < 2 {
    return num
} else {
    return fibonacci(of: num - 1) + fibonacci(of: num - 2)
}
}

let fibonacciSequence = (0...20).map(fibonacci)
print(fibonacciSequence[10])
```

That calculates the first 21 numbers of the Fibonacci sequence, and prints out the 11th: 55. I asked you to put this code in a playground because you'll see the **count += 1** line being called 57,291 times – a huge amount of work. If you try using **0...21** for the range – just one number higher! – you'll see that number goes up to 92,712 times.

Like I said, it's a naïve implementation, and it really doesn't scale well. Can you imagine using **0...199**? And what if you only need a few numbers rather than all of them?

This is where lazy sequences come into play: you give it a sequence to work with and tell it what code you want to run just like you would with a normal sequence, but now that code is executed on demand as you access items. So, we could prepare to generate the first 200 numbers in the Fibonacci, then only use the 20th value by using the **lazy** property of a sequence:

```
let lazyFibonacciSequence = Array(0...199).lazy.map(fibonacci)
print(lazyFibonacciSequence[19])
```

Note: you need to add the **Array** in there to ensure that Swift creates the lazy map on an array, rather than on the range **0...199**.

That new code takes a small amount of time to run, because all the other calculations are never

run – no time is wasted.

However, as clever as lazy sequences are, they do have a drawback that lazy properties do not: they have no memoization. This is a common optimization technique that stores the result of computationally expensive code so that it doesn't need to be created again.

This is essentially what regular lazy variables offer us: a guarantee not only that a property won't be created if it isn't used, but that it won't be created repeatedly when used again and again.

As I said, lazy sequences have *no* memoization, which means requesting the same data twice will require the work to be done twice. Try this now:

```
let lazyFibonacciSequence = Array(0...199).lazy.map(fibonacci)
print(lazyFibonacciSequence[19])
print(lazyFibonacciSequence[19])
print(lazyFibonacciSequence[19])
```

You'll see the code now takes three times as long to run. So, use lazy sequences when necessary, but remember there are situations when they might actually slow you down!

Destructuring

Destructuring (sometimes called *decomposition*) is a smart way to transfer data into and out from tuples, and when you start to understand it you'll realize how destructuring and pattern matching are closely linked.

Destructuring has three uses: pulling a tuple apart into multiple values, assigning multiple things simultaneously, and switching values.

Consider this tuple:

```
let data = ("one", "two", "three")
```

If you want to create three different constants out of those three values, without destructuring you'd need to write this:

```
let one = data.0
let two = data.1
let three = data.2
```

With destructuring you can write this:

```
let (one, two, three) = data
```

Swift pulls the **data** tuple apart into those three individual constants, all in a single line of code.

This technique is particularly helpful when you're working with functions that return tuples, which is commonly used when you want to return multiple values. It's common to want to split up those return values so you can refer to them on your terms, particularly if they have no names in the tuple. For example:

```
func getPerson() -> (String, Int) {
    ("Taylor Swift", 26)
```

```
}
```

```
let (name, age) = getPerson()
print("\(name) is \(age) years old")
```

If you want to ignore values during destructuring, use `_`, like this:

```
let (_, age) = getPerson()
print("That person is \(age) years old")
```

You can use this same technique to assign multiple things simultaneously, either using fixed values or using function calls. For example:

```
let (captain, chef) = ("Janeway", "Neelix")
let (engineer, pilot) = (getEngineer(), getPilot())
```

This is particularly useful when you're working with values that are closely related, such as coordinates for a rectangle, and can help improve readability.

Finally, tuple destructuring is good for swapping values. Now, I'll be honest: this technique is rarely useful outside of interviews, and even then it's a pretty poor choice for an interview question. However, I want to show it to you because I think it demonstrates just how graceful Swift can be.

So, here goes: given two integers, A and B, how do you swap them without using a third variable?

Have a think for a moment, and perhaps even try some code.

Here's how you would solve this in most languages:

```
var a = 10
var b = 20
```

Syntax

```
a = a + b  
b = a - b  
a = a - b
```

```
print(a)  
print(b)
```

In Swift, thanks to destructuring, you can write it in a single line:

```
(b, a) = (a, b)
```

Graceful, efficient, and quite beautiful I think. If you're ever asked this question in an interview, you should be able to ace it!

Labeled statements

Labels have been in use for a long time, but they largely fell out of favor when developers started to frown on **goto**. Swift brings them back, but without **goto**: instead they are used with loops to let you exit them more easily.

Here's some code that creates a grid of strings, and marks one of those squares with an “x” for where some treasure is – it's a hard-coded position here, but in a real game you'd obviously randomize it. The code then has two loops to try to find the treasure, with one loop nested inside the other: loop through all the rows in the board, then loop through each column in each row.

Here's the code:

```
var board = Array(repeating: Array(repeating: "", count: 10),
count: 5)
```

```
board[3][5] = "x"
```

```
for (rowIndex, cols) in board.enumerated() {
    for (colIndex, col) in cols.enumerated() {
        if col == "x" {
            print("Found the treasure at row \(rowIndex) col \(
(colIndex)!")
        }
    }
}
```

Given that the treasure can appear once on the board, this code is quite wasteful: even though the treasure is found early on in the search, it carries on looking. If you're thinking it's time to deploy **break** you'd be right, at least partially. Here's how it might look:

```
for (rowIndex, cols) in board.enumerated() {
```

Syntax

```
for (colIndex, col) in cols.enumerated() {
    if col == "x" {
        print("Found the treasure at row \(rowIndex) col \(colIndex)!")
        break
    }
}
```

However, **break** only exits one level of loop, so it would exit the **for (colIndex, col)** loop then continue running the **for (rowIndex, cols)** loop. Yes, it wastes *less* time, but it's still wasting *some*. You could add a Boolean variable that gets set to true when treasure is found, which you then use to break the outer loop, but Swift has a better solution: labeled statements.

Labeled statements let you give any loop a name, which allows you to refer to a specific loop when using **break** or **continue**. To create a label, just write a name then a colon before any loop. You can then use **break yourLabelName** or **continue yourLabelName** to refer directly to it.

So, the least wasteful way to write that code is like so:

```
var board = Array(repeating: Array(repeating: "", count: 10),
count: 5)
```

```
board[3][5] = "x"
```

```
rowLoop: for (rowIndex, cols) in board.enumerated() {
    for (colIndex, col) in cols.enumerated() {
        if col == "x" {
            print("Found the treasure at row \(rowIndex) col \(colIndex)!")
            break rowLoop
        }
    }
}
```

```

    }
}

```

That immediately jumps out of both loops, and continues on after the end of the **for (rowIndex, cols)** loop – perfect.

Labeling your loops is clever, but Swift takes it a step further: it lets you label **if** statements then **break** from them as if they were a loop.

This is extraordinarily useful when you find yourself deeply nested inside several conditions and want to bail out immediately, and without it you can end up with a pyramid of increasingly indented conditions.

Here's a worked example so you can see it in action:

```

if userRequestedPrint() {
    if documentSaved() {
        if userAuthenticated() {
            if connectToNetwork() {
                if uploadDocument("resignation.pages") {
                    if printDocument() {
                        print("Printed successfully!")
                    }
                }
            }
        }
    }
}

```

That code goes through a series of checks to allow a user to print a document: don't try running it, because those functions aren't real!

If all the conditions evaluate to **true**, then you'll see “Printed successfully!”.

Syntax

What labeled statements let you do is create early returns for your **if** statements. They run normally, but at any time you feel necessary you can exit any conditional statement. For example, we can rewrite the above pyramid into this:

```
printing: if userRequestedPrint() {
    if !documentSaved() { break printing }
    if !userAuthenticated() { break printing }
    if !connectToNetwork() { break printing }
    if !uploadDocument("work.doc") { break printing }
    if !printDocument() { break printing }

    print("Printed successfully!")
}
```

That takes up fewer lines, forces less indenting on people reading the code, and the happy path is immediately clear.

If you wanted to, you could even use **guard** to make your intention even clearer, like this:

```
printing: if userRequestedPrint() {
    guard documentSaved() else { break printing }
    guard userAuthenticated() else { break printing }
    guard connectToNetwork() else { break printing }
    guard uploadDocument("work.doc") else { break printing }
    guard printDocument() else { break printing }

    print("Printed successfully!")
}
```

For the sake of readability, I prefer to test for positive conditions rather than negative. That is, I'd prefer to test for **if documentSaved()** rather than **if !documentSaved()** because it's a little easier to understand, and **guard** does exactly that.

Nested types

Swift lets you nest one data type inside another, such as a struct within a struct or an enum within a class. This is most commonly used to help you mentally group things together by logical behavior, but can sometimes have access semantics attached to stop nested data types being used incorrectly.

Let's deal with the easy situation first: using nested types for logical grouping. Consider the code below, which defines an enum called **London**:

```
enum London {
    static let coordinates = (lat: 51.507222, long: -0.1275)

    enum SubwayLines {
        case bakerloo, central, circle, district, elizabeth,
hammersmithCity, jubilee, metropolitan, northern, piccadilly,
victoria, waterlooCity
    }

    enum Places {
        case buckinghamPalace, cityHall, oldBailey, piccadilly,
stPaulsCathedral
    }
}
```

That enum has one constant called **coordinates**, then two nested enums: **SubwayLines** and **Places**. But, notably, it has no cases of its own – it's just being used as a wrapper for other data.

This has two immediate benefits: first, any IDE with code completion makes it quick and easy to drill down to specific items by listing possible options as you type, for example **London.Places.cityHall**. Second, because you're effectively making namespaced constants, you can use sensible names like "Piccadilly" without worrying whether you mean the subway

line or the place, or whether you mean London Piccadilly or Manchester Piccadilly.

Nested types also work with other data types, for example you can have a struct that contains its own enum:

```
struct Cat {
    enum Breed {
        case britishShortHair, burmese, persian, ragdoll,
russianBlue, scottishFold, siamese
    }

    var name: String
    var breed: Breed
}
```

You can also place structs within structs when they are used together, for example:

```
struct Deck {
    struct Card {
        enum Suit {
            case hearts, diamonds, clubs, spades
        }

        var rank: Int
        var suit: Suit
    }

    var cards = [Card]()
}
```

As you can see in that last example, you can nest as many times as you want to – an enum within a struct within another struct is perfectly legal.

Syntax

Nesting for logical grouping doesn't stop you from referring to any of the nested types, although if you nest things too much it gets a bit cumbersome:

```
let burmese = Cat.Breed.burmese
let hearts = Deck.Card.Suit.hearts
```

However, Swift allows you to assign access control modifiers to nested types to control how they can be used. This is useful when a nested type has been designed to work specifically inside its parent: if the **Card** struct must only be used by the **Deck** struct, then you need access control.

Be warned: if a property uses a private type, the property itself must be private. To demonstrate, let's look at the **Deck** example again:

```
struct Deck {
    struct Card {
        enum Suit {
            case hearts, diamonds, clubs, spades
        }

        var rank: Int
        var suit: Suit
    }

    var cards = [Card]()
}
```

If we wanted that **Suit** enum to be private so that only **Card** instances can use it, we'd need to use **private enum Suit**.

However, this has the knock-on effect of requiring that the **suit** property of **Card** also be private, otherwise it would be accessible in places where the **Suit** enum was not.

So, the updated code is this:

```
struct Deck {  
    struct Card {  
        private enum Suit {  
            case hearts, diamonds, clubs, spades  
        }  
  
        var rank: Int  
        private var suit: Suit  
    }  
  
    var cards = [Card]()  
}
```

Access control for nested types is an important way of clarifying your intent – do you mean for the nested types to be accessed externally, or are they for inside their parent types only?

Nested functions

Nested functions are an interesting corner case of access control for nested types, because they are automatically restricted to their enclosing function unless you specify otherwise. Swift implements nested functions as named closures, which means they automatically capture values from their enclosing function.

To illustrate nested functions, I'm going to create a function that calculates the distance between two points using one of three distance calculation techniques: Euclidean (using Pythagoras's theorem), Euclidean squared (using Pythagoras's theorem, but avoiding the `sqrt()` call for performance reasons), and Manhattan.

If you're not familiar with these terms, "Euclidean distance" is basically drawing a straight line between two points, and "Manhattan distance" uses rectilinear geometry to calculate the absolute difference of two Cartesian coordinates. Try to imagine a taxi cab moving through a city where the blocks are square: you can move two blocks up then two to the right, or two to the right and two up, or one up, one right, one up, one right – the actual distance traveled is the same.

First, the code for the types we're going to work with:

```
import Foundation

struct Point {
    var x: Double
    var y: Double
}

enum DistanceTechnique {
    case euclidean
    case euclideanSquared
    case manhattan
}
```

I've created my own **Point** struct to avoid relying on **CGPoint** and thus Core Graphics.

We're going to create three functions, each of which will be nested inside one parent function. The point of this chapter isn't to explain distance calculation, so let's get them out of the way quickly!

First, we need a function that will be the parent of our three calculations, so start with this:

```
func calculateDistance(start: Point, end: Point, technique:
DistanceTechnique) -> Double {
    // functions go here
}
```

And now we can implement the three algorithms inside there:

```
func calculateEuclideanDistanceSquared(start: Point, end:
Point) -> Double {
    let deltaX = start.x - end.x
    let deltaY = start.y - end.y

    return deltaX * deltaX + deltaY * deltaY
}

func calculateEuclideanDistance(start: Point, end: Point) ->
Double {
    sqrt(calculateEuclideanDistanceSquared(start: start, end:
end))
}

func calculateManhattanDistance(start: Point, end: Point) ->
Double {
    abs(start.x - end.x) + abs(start.y - end.y)
```

Syntax

```
}
```

The first function, **calculateEuclideanDistanceSquared()**, uses Pythagoras's theorem to calculate the straight line distance between two points. If it's been a while since you were at school, this function considers the X and Y delta between two points to be two edges of a triangle, then calculates the hypotenuse of that triangle to be the distance between the two points – $A^2 + B^2 = C^2$.

The second function, **calculateEuclideanDistance()** builds on the **calculateEuclideanDistanceSquared()** function by calculating the square root of the result to give the true distance. If you need to calculate distances very frequently, e.g. every time the user's finger moves, having this call to **sqrt()** might become a performance liability, which is why the **calculateEuclideanDistanceSquared()** function is there.

Finally, the third function is **calculateManhattanDistance()**, which calculates the sum of the absolute distances between the two points' X and Y coordinates, as if you were in a taxi cab moving through a city built around square blocks.

With those three nested functions in place, it's now just a matter of choosing the correct option based on the technique that was requested:

```
switch technique {
case .euclidean:
    return calculateEuclideanDistance(start: start, end: end)
case .euclideanSquared:
    return calculateEuclideanDistanceSquared(start: start, end:
end)
case .manhattan:
    return calculateManhattanDistance(start: start, end: end)
}
```

That's it! We can now call the parent function like this:


```
let distance = calculateDistance(start: Point(x: 10, y: 10),
end: Point(x: 100, y: 100), technique: .euclidean)
print(distance)
```

Now, all that code is perfectly valid, but it's also more verbose than it needs to be. As a reminder, nested functions are just named closures and so they capture any values from their enclosing function.

In this context, it means we don't need to make the three nested functions accept any parameters, because they are identical to the parameters accepted by the enclosing functions – if we remove them, they'll just be automatically captured. This helps make our intention clearer: these nested functions are just different ways of operating on the same data rather than using specific values.

Here's the code to rewrite the **calculateDistance()** function so that it eliminates parameters from its nested functions and relies on capturing instead:

```
func calculateDistance(start: Point, end: Point, technique:
DistanceTechnique) -> Double {
    func calculateEuclideanDistanceSquared() -> Double {
        let deltaX = start.x - end.x
        let deltaY = start.y - end.y

        return deltaX * deltaX + deltaY * deltaY
    }

    func calculateEuclideanDistance() -> Double {
        sqrt(calculateEuclideanDistanceSquared())
    }

    func calculateManhattanDistance() -> Double {
        abs(start.x - end.x) + abs(start.y - end.y)
    }
}
```

Syntax

```
switch technique {  
case .euclidean:  
    return calculateEuclideanDistance()  
case .euclideanSquared:  
    return calculateEuclideanDistanceSquared()  
case .manhattan:  
    return calculateManhattanDistance()  
}  
}
```

Nested functions are automatically restricted to their enclosing function unless you specify otherwise, i.e. if you return them. Remember, functions are first-class data types in Swift, so you can use one function to return another based on specific criteria.

In our case, we could make the **calculateDistance()** function into **createDistanceAlgorithm()**, which accepts only a technique parameter and returns one of its three nested functions depending on which technique was requested.

I know it's obvious, but it bears repeating that when you use this approach your nested function stops being private – it gets sent back as a return value for anyone to use.

Here's how to rewrite **calculateDistance()** so that it returns one of three functions:

```
func createDistanceAlgorithm(technique: DistanceTechnique) ->  
(Point, Point) -> Double {  
    func calculateEuclideanDistanceSquared(start: Point, end:  
Point) -> Double {  
        let deltaX = start.x - end.x  
        let deltaY = start.y - end.y  
  
        return deltaX * deltaX + deltaY * deltaY  
    }  
}
```

```

func calculateEuclideanDistance(start: Point, end: Point) ->
Double {
    sqrt(calculateEuclideanDistanceSquared(start: start, end:
end))
}

```

```

func calculateManhattanDistance(start: Point, end: Point) ->
Double {
    abs(start.x - end.x) + abs(start.y - end.y)
}

```

```

switch technique {
case .euclidean:
    return calculateEuclideanDistance
case .euclideanSquared:
    return calculateEuclideanDistanceSquared
case .manhattan:
    return calculateManhattanDistance
}
}

```

Note that each of the three functions needs to accept parameters now, because they will be called later on like this:

```

let distanceAlgorithm =
createDistanceAlgorithm(technique: .euclidean)
let distance = distanceAlgorithm(Point(x: 10, y: 10), Point(x:
100, y: 100))

```

Documentation markup

Swift has special syntax that lets you embed Markdown-formatted text into your source code, which gets parsed by Xcode and displayed in the Quick Help system pane.

Using specially formatted code comments, you can document what parameters should be passed in, what the return value will contain, any errors that can be thrown, and more.

This documentation is *not* the same as the regular inline comments you add to particular code. These special comments are placed before your functions and classes and are used to show information in the Quick Help pane, as well as in the autocomplete popup, and are formatted so that both humans and Xcode can read them.

Let's get the easy stuff out of the way: unless you use one of the special keywords covered later, everything you write in a Markdown comment is shown as description text in the Quick Help pane. If you just start typing text, what you write will be used as a brief description in the autocomplete popup. Xcode can usually fit 20-30 words in the autocomplete space, but that's too long for real world use – aim for about 10 words of concise description.

Markdown comments start with `/**` and end with `*/`, like this:

```
/**
Call this function to grok some globs.
*/
func myGreatFunction() {
    // do stuff
}
```

In this text you can use a selection of Markdown formatting, as shown below:

Place text in ``backticks`` to mark code; on your keyboard these usually share a key with tilde, `~`.

* You can write bullets by starting with an asterisk then a space.

* **Indent** your asterisks to create sublists

1. You can write numbered lists by starting with 1.
1. Subsequent items can also be numbered 1. and Xcode will renumber them automatically.

If you want to write a link, [place your text in brackets](and your link in parentheses)

Headings start with a # symbol

Subheadings start with

Sub-subheadings start with ### and are the most common heading style you'll come across

Write a **single asterisk** around words to make them italic

Write ****two asterisks**** around words to make them bold

Documentation keywords

Beyond using text to describe your functions, Swift lets you add special keywords that get displayed in the Quick Help pane. There are quite a few of these, but most do nothing other than show a title and some text in Quick Help. There are six that I generally recommend as being useful, and you can learn them in a few minutes.

First: the **Returns** keyword lets you specify what value the caller can expect back when the function runs successfully. Remember that autocomplete already reports the data type of the return value, so this field is there to describe what the data actually means – we know it's a string, but how will it be formatted?

– **Returns:** A string containing a date formatted as RFC-822

Next is the **Parameter** keyword. This lets you specify the name of a parameter and describe what it contains. Again, autocomplete will say what data type must be used, so this is your chance to provide some detail: “The name of a Taylor Swift album”. You can include as many **Parameter** lines as you have parameters.

Syntax

- **Parameter** album: The name of a Taylor Swift album
- **Parameter** track: The track number to load

Third is the **Throws** keyword, which lets you specify a comma-separated list of the error types that can be thrown by the function:

- **Throws**: `LoadError.networkFailed, LoadError.writeFailed`

Fourth is **Precondition**, which should be used to describe the correct state of your program before your function is called. If you're using pure functions, this precondition ought to rely only on the parameters being passed into the function, for example that an array must not be empty:

- **Precondition**: `inputArray.isEmpty == false`

Fifth is **Complexity**, which is popular in the Swift standard library. This is often useful information for others working with your code. This should be written using Big O notation, for example:

- **Complexity**: `O(1)`

Finally there is the **Authors** keyword, which sounds useful at first but I remain dubious. As you might imagine, this is used to write the name of a function's author into the Quick Help pane, which is helpful when you need to figure out who to complain to and/or praise for their work:

- **Authors**: Paul Hudson

If you include more freeform text between the documentation keywords, it will just be flowed into the correct position in Quick Help.

Chapter 3

Types

Useful initializers

Understanding how initializers work isn't easy in Swift, but it's also something you probably learned quite a while ago so I'm not going to go over it again here. Instead, I want to focus on some interesting initializers that might help you use common Swift types more efficiently.

Repeating values

Easily my favorite initializer for strings and arrays is **repeating:count:**, which lets you create large values quickly. For example, you can create headings in the Markdown text format by writing equals signs underneath some text, like this:

```
This is a heading
=====
```

Markdown is a useful format because it can be parsed by computers while also being visually attractive to humans, and that underlining gives us a great example for **repeating:count:**.

To use this initializer, give it a string for its first parameter and the number of times to repeat for its second parameter, like this:

```
let heading = "This is a heading"
let underline = String(repeating: "=", count: heading.count)
```

You can do the same for arrays like this:

```
let equalsArray = Array(repeating: "=", count: heading.count)
```

This array initializer is flexible enough that you can use it to create multi-dimensional arrays very easily. For example, this creates a 10x10 array ready to be filled:

```
var board = Array(repeating: Array(repeating: "", count: 10),
count: 10)
```


Converting to and from numbers

It hurts my brain when I see code like this:

```
let str1 = "\(someInteger)"
```

It's wasteful and unnecessary, and yet string interpolation is such a neat feature that you'd be forgiven for using it – and in fact I'm pretty sure I've used it a few times without thinking!

But Swift has a simple, better way to create a string from an integer using an initializer:

```
let str2 = String(someInteger)
```

Things are slightly harder when converting the other way, because you might try to pass in an invalid number, like this:

```
let int1 = Int("elephant")
```

So, this initializer returns **Int?**: if you gave it a valid number you'll get back an **Int**, otherwise you'll get back nil.

If you don't want an optional, you should unwrap the result:

```
if let int2 = Int("1989") {
    print(int2)
}
```

Alternatively, use the nil coalescing operator (??) to provide a sensible default, like this:

```
let int3 = Int("1989") ?? 0
print(int3)
```

Swift has variations on these two initializers that deal with variable radices. For example, if you want to work with hexadecimal (base 16), you can ask Swift to give you the string

Types

representation of a number in hex like this:

```
let str3 = String(28, radix: 16)
```

That will set **str3** to “1c”. If you preferred “1C” – i.e., uppercase – try this instead:

```
let str4 = String(28, radix: 16, uppercase: true)
```

To convert that back into an integer - remembering that it’s optional! – use this:

```
let int4 = Int("1C", radix: 16)
```

Unique arrays

If you have an array with repeated values and you want to find a fast way to remove duplicates, you’re looking for **Set**. This is a built-in data type that has initializers to and from generic arrays, which means you can de-dupe an array quickly and efficiently just using initializers:

```
let scores = [5, 3, 6, 1, 3, 5, 3, 9]
let scoresSet = Set(scores)
let uniqueScores = Array(scoresSet)
```

That’s all it takes – it’s no wonder I’m such a big fan of sets!

Important: This approach does not retain the order of your original array.

Capacities

One simple initializer to end on: if you’re adding items individually to an array, set, or dictionary but know how many items you intend to add overall, create them using the **minimumCapacity:** initializer like this:

```
var dictionary = Dictionary<String, String>(minimumCapacity:
```

```
100)  
print(dictionary.capacity)
```

This helps Swift optimize execution by allocating enough space up front.

Note: Swift is free to create and adjust capacities as it sees fit, so you'll likely find that requesting a minimum capacity of 100 actually provides a larger capacity. Remember, we're requesting a *minimum* capacity – if you add more objects, it's not a problem.

Strings

I know you probably don't think of strings being a particularly advanced Swift feature, but there are four specific things I want to cover to make sure you have fully rounded out your knowledge of them.

Creating raw strings

Raw strings allow us to make strings by placing one or more hash symbols before and after the quote marks, like this:

```
let message = #"Hello, world!"#
```

Once made, the string is otherwise identical to a regular string.

That might seem pointless, but consider code like this:

```
let quote = "The \"rain\" in \"Spain\" falls \"mainly\" on the  
\"plain\"."
```

That uses quote marks inside the string as part of the text, which means we need to escape every single one to make sure Swift doesn't think we're ending the string early.

This is where raw strings come in: if you start a string with #" then Swift will ignore a quote mark by itself and instead only consider the string ending once it reads "#.

So, we could rewrite our string to this:

```
let quote = #"The "rain" in "Spain" falls "mainly" on the  
"plain"."#
```

The same effect applies to \. For example, this kind of code is not allowed in Swift:

```
let tip = "Use \ to start a key path, e.g. \.id, \.name, or  
\.self."
```

That considers each `\` to be the start of an escape sequence – Swift is looking for `\n` or string interpolation, for example.

So, we could either escape the escape character, like this:

```
let tip = "Use \\ to start a key path, e.g. \\.id, \\.name, or \\.self."
```

Or use raw strings, like this:

```
let tip = #"Use \ to start a key path, e.g. \.id, \.name, or \.self."#
```

As you might imagine, escaping the escape characters is a path that leads to migraines, because you end up having to escape the escape that escaped the escape, and so on – you find yourself writing code like this:

```
let tip2 = "Use \\\ to escape \\ in Swift."
print(tip2)
```

That will print `Use \\ to escape \ in Swift`, but it's really hard to tell just from the code!

Using raw strings instead allows us to make strings read more easily, and also *search* for them more easily – if you were running a search on your project to look for the string `Use \ to start a key path`, you wouldn't expect to have to escape it.

isEmpty vs count

If you need to know whether a string is empty, you should use this:

```
let string = "The rain in Spain"
print(string.isEmpty)
```

Types

...and not this:

```
let string = "The rain in Spain"
print(string.count == 0)
```

I know the latter comes far more naturally to many people, likely for two reasons:

1. We often use **count** to look for a specific number, such as at least 3 letters, or no more than 50 letters.
2. Using **count** or similar is the only approach in many other languages.

However, in Swift, using **count** is an $O(n)$ operation, where "n" is the number of letters in the string. In plainer terms, it means the longer a string is, the longer it takes to count.

This happens because Swift calculates the length of your string every time you ask, whereas most other languages just store the size separately.

So, when we use **string.count**, Swift needs to go over every character one by one, which is particularly slow because it goes to great lengths to make sure emoji only count as one character even they are made up of several. That's a little inefficient when checking short strings, but *viciously* inefficient if your string is long.

In comparison, using **string.isEmpty** has one comparison: is the start index of the string equal to the end index? It's instant, whether you have an empty string or the complete works of Shakespeare.

Substrings and memory

Swift has a few different string types, but the ones you'll use most are **String** and **Substring**. Although they are similar, they carry a small but important performance consideration: substrings refer to part of a string elsewhere, and can often keep that original string's memory allocated long after you think.

What matters is this: for as long as your substring remains alive, the entire string it refers to

also stays alive.

I want to demonstrate this with some code you can step through with a debugger, so you can track the memory usage yourself.

First, here's a function that generates a large string, then sends back the first 100 letters of it as a **Substring**:

```
func generateBigString() -> Substring {
    let input = String(repeating: UUID().uuidString, count:
10_000_000)
    let endPosition = input.index(input.startIndex, offsetBy:
100)
    return input[...endPosition]
}
```

Second, this function calls the first one, printing its returned value:

```
func runTest() {
    let substring = generateBigString()
    print(substring)
}
```

And finally we can write a little code to run our test, adding some extra space for breakpoint steps:

```
print("Starting")
runTest()
print("Done")
```

Now place a breakpoint on the "Starting" line and run your code.

1. Use the Step Into button until you reach the **let substring = generateBigString()** line – you should notice Xcode's debug navigator says memory usage is pretty low.

Types

2. Press Step Into again and you'll move into **generateBigString()**, and memory will still be low.
3. As soon as the **String(repeating:count:)** line runs you'll see memory jumps up, because the string being made is big.
4. Now press Step Into a few more times until you exit **generateBigString()** and go back to **runTest()** – you'll see memory usage remains high, because although the big string is no longer in scope, our substring still references it.
5. Now press Step Into a few more times until you're back at the top-level code again, at which point you'll see memory drops down again – as soon as the substring is destroyed, the original string is destroyed too.

This same behavior happens with other slice types such as **ArraySlice** – if you load a huge array of strings then return only a handful, the entire original array stays in memory.

If you need to keep substrings around for a long time, it's almost certainly better to promote them to full strings, so that Swift can remove the original, big string fully.

You can see the difference it makes in our little memory sandbox just by adding a string conversion to **runTest()**:

```
func runTest() {  
    let substring = String(generateBigString())  
    print(substring)  
}
```

When that runs you'll see memory drops while **runTest()** is still executing because Swift can see the original string isn't needed.

Enums

I already talked about enum associated values in the pattern matching chapter, but here I want to focus on enums themselves because they are deceptively powerful.

Let's start with a very simple enum to track some basic colors:

```
enum Color {  
    case unknown  
    case blue  
    case green  
    case pink  
    case purple  
    case red  
}
```

If you prefer, you can write all the cases on one line, like this:

```
enum Color {  
    case unknown, blue, green, pink, purple, red  
}
```

For testing purposes, let's wrap that in a simple struct that represents toys:

```
struct Toy {  
    let name: String  
    let color: Color  
}
```

Swift's type inference can see that the **color** property of a **Toy** is a **Color** enum, which means you don't need to write **Color.blue** when creating a toy. For example, we can create two toys like this:

```
let barbie = Toy(name: "Barbie", color: .pink)
```

Types

```
let raceCar = Toy(name: "Lightning McQueen", color: .red)
```

Raw values

Let's start off with raw values: a data type that underlies each enum case. Enums don't have a raw value by default, so if you want one you need to declare it. For example, we could give our colors an integer raw value like this:

```
enum Color: Int {  
    case unknown, blue, green, pink, purple, red  
}
```

Just by adding **: Int** Swift has given each color a matching integer, starting from 0 and counting upwards. That is, Unknown is equal to 0, Blue is equal to 1, and so on.

Sometimes the default values aren't useful to you, so you can specify individual integers for every raw value if you want. Alternatively, you can specify a different starting point to have Xcode count upwards from there.

For example, we could create an enum for the four inner planets of our solar system like this:

```
enum Planet: Int {  
    case mercury = 1  
    case venus  
    case earth  
    case mars  
    case unknown  
}
```

By specifically giving Mercury the value of 1, Xcode will count upwards from there: Venus is 2, Earth is 3, and Mars is 4.

Now that the planets are numbered sensibly, we can find the raw value of any planet like this:

```
let marsNumber = Planet.mars.rawValue
```

Going the other way isn't quite so easy: yes, you can create a **Planet** enum from a number now that we have raw values, but doing so creates an *optional* enum. This is because you could try to create a planet with the raw value 99, which doesn't exist – at least not *yet*.

Fortunately, I added an Unknown case to the planets for times when an invalid planet number has been requested, so we can use nil coalescing to create a **Planet** enum from its raw value with a sensible default:

```
let mars = Planet(rawValue: 556) ?? .unknown
```

Having numbers is fine for planets, but when it comes to colors you might find it easier to use strings instead. Unless you have very specific needs, just specifying **String** as the raw data type for your enum is enough to give them meaningful names – Swift automatically maps your enum name to a string. For example, this will print “pink”:

```
enum Color: String {
    case unknown, blue, green, pink, purple, red
}
```

```
let pink = Color.pink.rawValue
print(pink)
```

Regardless of the data type of your raw values, or even if you have one or not, Swift will automatically stringify your enums when they are used as part of string interpolation.

Used in this way doesn't make them a string, though, so if you want to call any string methods you need to create a string from them. For example:

```
let barbie = Toy(name: "Barbie", color: .pink)
let raceCar = Toy(name: "Lightning McQueen", color: .red)
```

Types

```
// regular string interpolation
print("The \(barbie.name) toy is \(barbie.color)")

// get the string form of the Color then call a method on it
print("The \(barbie.name) toy is \(
(barbie.color.rawValue.uppercased())")
```

Computed properties and methods

Enums aren't quite as powerful as structs and classes, but they do let you encapsulate some useful functionality inside them.

For example, you can't give enums stored properties unless they are static because doing so wouldn't make sense, but you can add *computed* properties that return a value after running some code.

To put this in some useful context for you, let's add a computed property to the **Color** enum that prints a brief description of the color. Apologies ahead of time for mixing Terry Pratchett, Prince and Les Misérables references in one piece of code:

```
enum Color {
    case unknown, blue, green, pink, purple, red

    var description: String {
        switch self {
        case .unknown:
            "the color of magic"
        case .blue:
            "the color of the sky"
        case .green:
            "the color of grass"
        case .pink:
            "the color of carnations"
```

```

        case .purple:
            "the color of rain"
        case .red:
            "the color of desire"
    }
}
}

```

```

let barbie = Toy(name: "Barbie", color: .pink)
print("This \(barbie.name) toy is \(barbie.color.description)")

```

Of course, computed properties are just syntactic sugar around good old methods, so it should come as no surprise that you can add methods directly to enums too.

Let's do that now by adding two new methods to the **Color** enum, **forBoys()** and **forGirls()**, to determine whether a toy is for girls or boys based on its color – just add this below the **description** computed property we just added:

```

func forBoys() -> Bool {
    true
}

func forGirls() -> Bool {
    true
}

```

In case you were wondering, deciding which toys are for boys or girls based on their color is a bit 1970s: those functions both return true for a reason!

Enum protocols

Enums can conform to protocols just like other types can. In fact, they automatically conform to some out of the box, including **Hashable**, as long as they don't have an associated value.

Types

However, with enums there are three that are particularly useful: **CaseIterable**, **Comparable**, and **Identifiable**.

The first of these is one of those wonderful protocols that causes the compiler to add extra functionality on our behalf, which in this case is a static array called **allCases**, which contains each of the enum's cases in the order you defined them.

For example, here's a **CaseIterable** enum defining various weather conditions:

```
enum Weather: CaseIterable {  
    case rainy, snowy, sunny, windy  
}
```

Just adding that **CaseIterable** conformance is enough to create the **allCases** array, allowing us to loop over it:

```
for condition in Weather.allCases {  
    print(condition)  
}
```

Again, they are ordered in the same way they appear in the enum's definition.

That automatic conformance only works if you don't use any associated values. If you want an associated value – and there are sensible associated values you could put into an array – you need to make your own, like this:

```
enum Weather: CaseIterable {  
    case rainy, snowy(isHeavy: Bool), sunny, windy  
  
    static var allCases: [Weather] {  
        [.rainy, .snowy(isHeavy: false), .snowy(isHeavy:  
true), .sunny, .windy]  
    }  
}
```

The second useful conformance is **Comparable**, and it's another one the compiler adds for us based on the order of our cases from left to right.

As an example, this creates an enum suitable for clothing sizes:

```
enum Size: Comparable {  
    case extraSmall, small, medium, large, extraLarge  
}
```

Because we've marked that as **Comparable**, Swift will automatically generate a < operator to compare two sizes, like this:

```
print(Size.extraSmall < Size.extraLarge)
```

Unlike **CaseIterable**, Swift's automatic conformance for **Comparable** *does* work when associated values are attached, because Swift extends the comparison to whatever associated value you're storing – it compares the cases first, then the associated values second.

However, you need to be careful that you don't get into meaningless comparisons by accident. For example, we could add an extra case to our **Size** enum to track other special sizes, like this:

```
enum Size: Comparable {  
    case extraSmall, small, medium, large, extraLarge,  
    other(name: String)  
}
```

That's helpful for adding in cases we hadn't explicitly coded in, such as a size for infants:

```
let baby = Size.other(name: "Baby")
```

But now we've made a problem for ourselves: that new **other** case comes *after* the **extraLarge** case, which means code like this prints nonsense:

Types

```
print(baby > Size.extraLarge)
```

That will confidently tell us that baby sizes are bigger than extra large, because that's how we made the enum.

So, by all means use **Comparable** with your enums, just be mindful of how you order your cases!

Finally, I want to touch on the **Identifiable** protocol, which is important if you want to bind your enum cases in things like pickers.

For enums that don't have an associated value, this is as simple as adding an **id** property that returns **self**, like this:

```
enum SpendingType: Identifiable {
    case food, home, leisure, travel

    var id: SpendingType {
        self
    }
}
```

If you're feeling fancy, you could return **Self** from **id** like this:

```
enum SpendingType, Identifiable {
    case food, home, leisure, travel

    var id: Self {
        self
    }
}
```

It does the same thing here – **Self** refers to **SpendingType**.

If we added an **other** case that has a string associated type, our simple solution won't work any more. So, this code won't compile:

```
enum SpendingType: Identifiable {
    case food, home, leisure, travel, other(name: String)

    var id: SpendingType {
        self
    }
}
```

To fix this we need to re-add **Hashable** conformance to our enums – it's there by default for simple enums, but automatically gets removed for enums with associated values to avoid accidentally hashing things in a way you aren't happy with. This is no different from a struct with properties, which also don't conform to **Hashable** or **Equatable** unless you ask for it.

If we re-add the **Hashable** conformance (which will automatically re-add **Equatable**, because **Hashable** inherits from **Equatable**), then Swift can use the final, hashed identifier for the object's identifier automatically:

```
enum SpendingType: Hashable {
    case food, home, leisure, travel, other(name: String)

    var id: SpendingType {
        self
    }
}
```

So: our enum now has a raw value, a computed property, methods, and protocol conformances too. I hope you can see why I described enums as “deceptively powerful” – they can do a lot!

Arrays

Arrays are one of the true workhorses of Swift. Sure, they are important in most apps, but their use of generics makes them type-safe while adding some useful features.

In this chapter I want to walk through a handful of the most important features of arrays, to make sure you're using them effectively.

First: sorting. As long as the element type your array stores adopts the **Comparable** protocol, you get the methods **sorted()** and **sort()** – the former returns a sorted array, whereas the latter modifies the array you call it on. If you don't adopt the **Comparable** protocol, you can use alternate versions of **sorted()** and **sort()** that let you specify how items should be sorted.

To demonstrate the examples below we'll be using these two arrays:

```
var names = [ "Taylor", "Timothy", "Tyler", "Thomas", "Tobias",  
             "Tabitha" ]  
let numbers = [ 5, 3, 1, 9, 5, 2, 7, 8 ]
```

To sort the **names** array alphabetically, use either **sort()** or **sorted()** depending on your needs:

```
let sorted = names.sorted()
```

Once that code runs, **sorted** will contain ["Tabitha", "Taylor", "Thomas", "Timothy", "Tobias", "Tyler"].

If you want to write your own sorting function – which is required if you don't adopt **Comparable** but optional otherwise – write a closure that accepts two strings and returns true if the first string should be ordered before the second.

For example, we could write a string sorting algorithm that behaves identically to a regular alphabetical sort, with the exception that it always places the name “Taylor” at the front. It's what Ms Swift would want, I'm sure:

```
names.sort { first, second in
    print("Comparing \(first) and \(second)")
    if (first == "Taylor") {
        return true
    } else if second == "Taylor" {
        return false
    } else {
        return first < second
    }
}
```

That code uses **sort()** rather than **sorted()**, which causes the array to be sorted in place rather than returning a new, sorted array. I also added a **print()** call in there so you can see exactly how **sort()** works. Here's the output:

```
Comparing Timothy and Taylor
Comparing Tyler and Timothy
Comparing Thomas and Tyler
Comparing Thomas and Timothy
Comparing Thomas and Taylor
Comparing Tobias and Tyler
Comparing Tobias and Timothy
Comparing Tabitha and Tyler
Comparing Tabitha and Tobias
Comparing Tabitha and Timothy
Comparing Tabitha and Thomas
Comparing Tabitha and Taylor
```

As you can see, names can appear as **\$0** or **\$1** as the algorithm progresses, which is why I compare against both possibilities in the custom sort function.

Sorting is easy enough, but adopting **Comparable** also enables two more useful methods: **min()** and **max()**. Just like **sort()** these have alternatives that accept a closure if you don't

Types

adopt **Comparable**, but the code is identical because the operation is identical: should item A appear before item B?

Using the **numbers** array from earlier, we can find the highest and lowest number in the array in two lines of code:

```
let lowest = numbers.min()  
let highest = numbers.max()
```

For strings, **min()** returns whichever string comes first after sorting, and **max()** returns whichever string comes last. If you try re-using the same closure I gave for custom sorting, including the **print()** statement, you'll see that **min()** and **max()** are actually more efficient than using **sort()** because they don't need to move every item.

Conforming to Comparable

Working with **sort()**, **min()**, and **max()** is easy enough for basic data types like strings and integers. But how do you sort something else entirely, like types of cheese or breeds of dog? I've already shown you how to write custom closures, but that approach becomes cumbersome if you have to sort more than once – you end up duplicating code, which creates a maintenance nightmare.

The smarter solution is to implement the **Comparable** protocol, which in turn requires you to use operator overloading. We'll get into more detail on that later, but for now I want to show you just enough to get **Comparable** working.

First, here's a basic **Dog** struct that holds a couple of pieces of information:

```
struct Dog {  
    var breed: String  
    var age: Int  
}
```

For the purposes of testing, we'll create three dogs and group them in an array:

```
let luna = Dog(breed: "Samoyed", age: 7)
let arya = Dog(breed: "Samoyed", age: 8)
let rusty = Dog(breed: "Golden Retriever", age: 9)
var dogs = [luna, arya, rusty]
```

Because the **Dog** struct doesn't conform to **Comparable**, we don't get simple **sort()** and **sorted()** methods on the **dogs** array, we only get ones that require a custom closure to run.

To begin the process of making **Dog** adopt **Comparable** we need to declare it as conforming, like this:

```
struct Dog: Comparable {
    var breed: String
    var age: Int
}
```

You'll get errors now, and that's okay.

The next step is what confuses people when they try it for the first time: you need to implement a new function, but it has an unusual name that takes a little getting used to when working with operator overloading, which is what we need to do.

Add this method inside the **Dog** struct:

```
static func <(lhs: Dog, rhs: Dog) -> Bool {
    lhs.age < rhs.age
}
```

To be clear, your code should look like this:

```
struct Dog: Comparable {
    var breed: String
    var age: Int
```

Types

```
static func <(lhs: Dog, rhs: Dog) -> Bool {  
    lhs.age < rhs.age  
}  
}
```

That function name is unusual if you haven't worked with operator overloading before, but I hope you can see exactly what it does: the `<` function is used when you write **dog1 < dog2**.

Those two steps are enough to implement **Comparable** fully, so you can now sort the **dogs** array easily:

```
dogs.sort()
```

Adding and removing items

You're almost certainly already using the **append()**, **insert()** and **remove(at:)** methods of arrays, but I want to make sure you're aware of other ways to add and remove items.

If you want to add two arrays together, you can use `+`, or `+=` to add in place. For example:

```
let beethoven = Dog(breed: "St Bernard", age: 8)  
dogs += [beethoven]
```

When it comes to removing items, there are two interesting ways of removing the last item: **removeLast()** and **popLast()**. They both remove the final item in an array and return it to you, but **popLast()** is optional whereas **removeLast()** is not.

Think that through for a moment: **dogs.removeLast()** must return an instance of the **Dog** struct. What happens if the array is empty? Well, the answer is “bad things” – your app will crash.

If it's even slightly possible your array might be empty when you try to remove an item, use **popLast()** instead so you can safely check the return value:

```
if let dog = dogs.popLast() {  
    // do stuff with `dog`  
}
```

Note: **removeLast()** has a counterpart called **removeFirst()** to remove and return the initial item in an array. Sadly, **popLast()** has no equivalent.

Emptiness and capacity

Here are two more small tips I want to demonstrate: **isEmpty** and **reserveCapacity()**. The first of these, **isEmpty**, returns true if the array has no items added to it. This is both shorter and more efficient than using **someArray.count == 0**, but for some reason gets less use.

The **reserveCapacity()** method lets you tell iOS how many items you intend to store in the array. This isn't a hard limit – i.e., if you reserve a capacity of 10 you can go ahead and store 20 if you want – but it does allow iOS to optimize object storage by ensuring you have enough space for your suggested capacity.

Warning: using **reserveCapacity()** is *not* a free operation. Behind the scenes, Swift will create a new array that contains the same values, with space for the capacity you asked for. It will *not* just extend the existing array.

Dictionaries

The unique nature of dictionaries give them a few special methods that are worth covering in detail.

For example, it's not safe to use **map()** on dictionary keys because you might create duplicates by accident – and dictionaries can't have duplicate keys.

As a result, there's a method called **mapValues()** that is much more useful because it lets you transform *values* and place them back into a dictionary using the original keys.

For example, this code will round and stringify all city populations, then put them back into a new dictionary with the same keys of Shanghai, Karachi, and Seoul:

```
let cities = [ "Shanghai": 24_256_800, "Karachi": 23_500_000,
               "Beijing": 21_516_000, "Seoul": 9_995_000 ]
let roundedCities = cities.mapValues { "\( $0 / 1_000_000 )
million people" }
```

Easily my favorite dictionary feature is a grouping initializer, which converts a sequence into a dictionary of sequences that are grouped by whatever you want. Continuing our cities example, we could use **cities.keys** to get back an array of city names, then group them by their first letter, like this:

```
let groupedCities = Dictionary(grouping: cities.keys)
{ $0.first! }
print(groupedCities)
```

That will output the following:

```
[ "B": [ "Beijing" ], "S": [ "Shanghai", "Seoul" ], "K":
[ "Karachi" ] ]
```

Alternatively, we could group the cities based on the length of their names like this:


```
let groupedCities = Dictionary(grouping: cities.keys)
{ $0.count }
print(groupedCities)
```

That will output the following:

```
[5: ["Seoul"], 7: ["Karachi", "Beijing"], 8: ["Shanghai"]]
```

Finally, it's possible to access a dictionary key and provide a default value to use if the key is missing:

```
let person = ["name": "Tom", "city": "Detroit"]
let name = person["name", default: "Anonymous"]
```

Now, any experienced developer will probably argue that could equally well be written using nil coalescing, and I agree. You could write this line instead using the current version of Swift:

```
let name = person["name"] ?? "Anonymous"
```

However, that doesn't work when you're modifying the dictionary value rather than just reading it. You can't modify a dictionary value in place because accessing its key returns an optional – the key might not exist, after all. With Swift's default dictionary values you can write much more succinct code, such as this:

```
var favoriteTVShows = ["Red Dwarf", "Blackadder", "Fawlty Towers", "Red Dwarf"]
var favoriteCounts = [String: Int]()

for show in favoriteTVShows {
    favoriteCounts[show, default: 0] += 1
}
```

That loops over every string in **favoriteTVShows**, and uses a dictionary called

Types

favoriteCounts to keep track of how often each item appears. We can modify the dictionary in one line of code because we know it will always have a value: either the default value of 0, or some higher number based on previous counting.

Sets

Understanding the difference between sets and arrays – and knowing when each one is the right choice – is an important skill in any Swift developer’s toolbox.

Sets can be thought of as unordered arrays that cannot contain duplicate elements. If you add the same element more than once, it will appear only once in the set.

The combination of lacking duplicates and not tracking order allows sets to be significantly faster than arrays, because items are stored according to a hash rather than an incrementing integer index.

To put this into context, checking to see whether an array contains an item has the complexity $O(n)$, which means “it depends on how many elements you have in the array.” This is because `Array.contains()` needs to check every element from 0 upwards, so if you have 50 elements it needs to do 50 checks.

Checking whether a *set* contains an item has the complexity $O(1)$, which means “it will always run at the same speed no matter how many elements you have.” This is because sets work like dictionaries: a key is generated by creating a hash of your object, and that key points directly to where the object is stored.

The basics

The best way to experiment with sets is with some code. Try this:

```
var set1 = Set<Int>([1, 2, 3, 4, 5])
print(set1)
```

When that runs, you’ll see `[5, 2, 3, 1, 4]` in the output window. Like I said, sets are unordered so you might see something different in your Xcode window.

That creates a new set from an array, but Swift is capable of making them using array literal syntax too, like this:

Types

```
var set1: Set = [1, 2, 3, 4, 5]
```

You can even create them from ranges:

```
var set2 = Set(1...100)
```

You can add items to them individually, although the method is named **insert()** rather than **append()** to reflect its unordered nature:

```
set1.insert(6)
set1.insert(7)
```

To check whether an item exists in your set, use the lightning fast **contains()** method:

```
if set1.contains(3) {
    print("Number 3 is in there!")
}
```

And use **remove()** to remove items from the set:

```
set1.remove(3)
```

Arrays and sets

Arrays and sets work well when used together, so it's no surprise they are almost interchangeable.

First, both arrays and sets have initializers that take the other type, like this:

```
var set1: Set = [1, 2, 3, 4, 5]
var array1 = Array(set1)
var set2 = Set(array1)
```

In fact, converting an array into a set and back is the fastest way to remove all its duplicates,

and it's just two lines of code.

Second, several methods of sets return arrays rather than sets because doing so is more useful. For example, the **sorted()** and **map()** methods on sets return an array.

So, while you can loop over sets directly like this:

```
for number in set1 {  
    print(number)  
}
```

...you can also sort the set into a sensible order first, like this:

```
for number in set1.sorted() {  
    print(number)  
}
```

Sets, like arrays, have the method **removeFirst()** to remove an item from the front of the set. Its use is different, though: because sets are unordered you really have no idea what that first item is going to be, so **removeFirst()** effectively means “give me any object so I can process it.”

Cunningly, sets have a **popFirst()** method, which arrays do not – a real shame!

Set operations

Sets come with a number of methods that allow you to manipulate them in interesting ways. For example, you can create the union of two sets – i.e. the merger of two sets – like this:

```
let spaceships1 = Set(["Serenity", "Nostromo", "Enterprise"])  
let spaceships2 = Set(["Voyager", "Serenity", "Executor"])  
  
let union = spaceships1.union(spaceships2)
```

Types

When that code runs, **union** will contain five items because the duplicate “Serenity” will appear only once.

Two other useful set operations are **intersection()** and **symmetricDifference()**. The former returns a new set that contains only elements that exist in both sets, and the latter does the opposite: it returns only elements that do not exist in both. In code it looks like this:

```
let intersection = spaceships1.intersection(spaceships2)
let difference = spaceships1.symmetricDifference(spaceships2)
```

When that runs, **intersection** will contains Serenity and **difference** will contains Nostromo, Enterprise, Voyager and Executor.

Note: **union()**, **intersection()**, and **symmetricDifference()** all have in-place alternatives that modify the set directly, called by adding **form** to the method. So, **formUnion()**, **formIntersection()**, and **formSymmetricDifference()**.

Sets have several query methods that return true or false depending on what you provide them, and in particular they distinguish between subsets and *strict* (or “proper”) subsets, with the difference being that the latter necessarily excludes identical sets.

That is, Set A is a subset of Set B if every item in Set A is also in Set B. On the other hand, Set A is a strict subset of Set B if every element in Set A is also in Set B, but Set B contains at least one item that is missing from Set A.

The code below demonstrates them all individually, and I’ve marked each method’s return value in a comment:

```
let spaceships1 = Set(["Serenity", "Nostromo", "Enterprise"])
let spaceships2 = Set(["Voyager", "Serenity", "Executor"])
let spaceships3 = Set(["Galactica", "Sulaco", "Minbari"])
let spaceships1and2 = spaceships1.union(spaceships2)

print(spaceships1.isSubset(of: spaceships1and2)) // true
```

```

print(spaceships1.isSubset(of: spaceships1)) // true
print(spaceships1.isSubset(of: spaceships2)) // false

print(spaceships1.isStrictSubset(of: spaceships1and2)) // true
print(spaceships1.isStrictSubset(of: spaceships1)) // false

```

NSCountedSet

The Foundation framework has a specialized set called **NSCountedSet**, and it is a set with a twist: items can still appear only once, but if you try to add them more than once it will keep track of the count as if they were actually there. This means you get all the speed of non-duplicated sets, but you also get the ability to count how many times items would appear if duplicates were allowed.

You can create an **NSCountedSet** from Swift arrays or sets depending on your need. In the example below I create one large array (with a duplicated item), add it all to the counted set, then print out the counts for two values:

```

var spaceships = ["Serenity", "Nostromo", "Enterprise"]
spaceships += ["Voyager", "Serenity", "Star Destroyer"]
spaceships += ["Galactica", "Sulaco", "Minbari"]

let countedSet = NSCountedSet(array: spaceships)
print(countedSet.count(for: "Serenity")) // 2
print(countedSet.count(for: "Sulaco")) // 1

```

As you can see, you can use **count(for:)** to retrieve how many times (in theory) an element appears in the counted set. You can pull out an array of all objects using the **countedSet.allObjects** property, but be warned: **NSCountedSet** does *not* support generics, so you'll need to typecast it back to **[String]**.

Tuples

Tuples are like simplified, anonymous structs: they are value types that carry distinct fields of information, but they don't need to be formally defined. This lack of formal definition makes them easy to create and throw away, so they are commonly used when you need a function to return multiple values.

In the chapters on pattern matching and destructuring, I covered how tuples are used in other ways – they really are pervasive little things. How pervasive? Well, consider this code:

```
func doNothing() { }  
let result = doNothing()
```

Consider this: what data type does the **result** constant have? As you might have guessed given the name of this chapter, it's a tuple: **()**. Behind the scenes, Swift maps the **Void** data type (the default for functions with no explicit return type) to an empty tuple.

Now consider this: every type in Swift – integers, strings, and so on – is effectively a single-element tuple of itself. Take a look at the code below:

```
let int1: (Int) = 1  
let int2: Int = (1)
```

That code is perfectly correct: assigning an integer to a single-element tuple and assigning a single-element tuple to an integer both do exactly the same thing. As the Apple documentation says, “if there is only one element inside the parentheses, the type [of a tuple] is simply the type of that element.” They are literally the same thing.

So, if you return nothing from a function you get a tuple, if you return several values from a function you get a tuple, and if you return a single value you effectively also get a tuple. I think it's safe to say you're already a frequent tuple user whether you knew it or not!

Now, I'm going to cover a few interesting aspects of tuples below, but first you should know there are a couple of downsides to tuples. Specifically, you can't add methods to tuples or

make them implement protocols – if this is what you want to do, you’re looking for structs instead.

Tuples have types

It’s easy to think of tuples as some sort of open dumping ground for data, but that’s not true: they are strongly typed, just like everything else in Swift. This means you can’t change the type of a tuple once it’s been created – code like this will simply not compile:

```
var singer = ("Taylor", "Swift")
singer = ("Taylor", "Swift", 26)
```

If you don’t name the elements of your tuple, you can access them using numbers counting from 0, like this:

```
var singer = ("Taylor", "Swift")
print(singer.0)
```

If you have tuples within tuples – which is not uncommon – you need to use **0.0** and so on, like this:

```
var singer = (first: "Taylor", last: "Swift", address: ("555
Taylor Swift Avenue", "No, this isn't real", "Nashville"))
print(singer.2.2) // Nashville
```

This is built-in behavior, but that doesn’t mean it’s recommended. You can – and usually should – name your elements so you can access them more sensibly:

```
print(singer.last)
```

These names form part of the type, so code like this will not compile:

```
var singer = (first: "Taylor", last: "Swift")
singer = (first: "Justin", fish: "Trout")
```

Tuples and closures

You can't add methods to tuples, but you can add closures. The distinction is very fine, I agree, but it is important: adding a closure to a tuple is just like adding any other value, you're literally attaching the code as a data type to the tuple. Because it's not a regular function the declaration is a little different, but here's an example to get you started:

```
var singer = (first: "Taylor", last: "Swift", sing: { (lyrics:
String) in
    print(lyrics)
})

singer.sing("Haters gonna hate")
```

Note: these closures can't access sibling elements, which means code like this won't work:

```
print("My name is \(first): \(lyrics)")
```

Returning multiple values

Tuples are commonly used to return multiple values from a function. In fact, if this was the only thing that tuples gave us, they'd still be a huge feature of Swift compared to other languages – including Objective-C.

Here's an example of a Swift function returning several values in a single tuple:

```
func fetchWeather() -> (type: String, cloudCover: Int, high:
Int, low: Int) {
    ("Sunny", 50, 32, 26)
}

let weather = fetchWeather()
```

```
print(weather.type)
```

You don't have to name the elements, of course, but it is certainly good practice so that other developers know what to expect.

If you prefer to destructure the results of a tuple-returning function, that's easy to do too:

```
let (type, cloud, high, low) = fetchWeather()
```

In comparison, if Swift *didn't* have tuples then we'd have to rely on returning an array and typecasting as needed, like this:

```
import Foundation

func fetchWeather() -> [Any] {
    ["Sunny", 50, 32, 26]
}

let weather = fetchWeather()
let weatherType = weather[0] as! String
let weatherCloud = weather[1] as! Int
let weatherHigh = weather[2] as! Int
let weatherLow = weather[3] as! Int
```

Or – worse – using **inout** variables, like this:

```
func fetchWeather(type: inout String, cloudCover: inout Int,
high: inout Int, low: inout Int) {
    type = "Sunny"
    cloudCover = 50
    high = 32
    low = 26
}
```

Types

```
var weatherType = ""
var weatherCloud = 0
var weatherHigh = 0
var weatherLow = 0

fetchWeather(type: &weatherType, cloudCover: &weatherCloud,
high: &weatherHigh, low: &weatherLow)
```

Seriously: if **inout** is the answer, you're probably asking the wrong question.

Optional tuples

Tuples can contain optional elements, and you can also have optional tuples. That might sound similar, but the difference is huge: optional elements are individual items inside a tuple such as **String?** and **Int?**, whereas an optional tuple is where the whole structure may or may not exist.

A tuple with optional elements must exist, but its optional elements may be nil. An optional tuple must either have all its elements filled, or be nil. An optional tuple with optional elements may or may not exist, and each of its optional elements may or may not exist.

Swift can't use type inference when you work with optional tuples because each element in a tuple has its own type. So, you need to declare exactly what you want, like this:

```
let optionalElements: (String?, String?) = ("Taylor", nil)
let optionalTuple: (String, String)? = ("Taylor", "Swift")
let optionalBoth: (String?, String?)? = (nil, "Swift")
```

Broadly speaking, optional elements are common, optional tuples less so.

Comparing tuples

Swift lets you compare tuples up to arity six as long as they have identical types. That means you

can compare tuples that contain up to six items using `==`, and it will return true if all six items in one tuple match their counterpart in a second tuple.

For example, the code below will print “No match”:

```
let singer = (first: "Taylor", last: "Swift")
let person = (first: "Justin", last: "Bieber")

if singer == person {
    print("Match!")
} else {
    print("No match")
}
```

Be warned, though: tuple comparison ignores element labels and focuses only on types, which can have unexpected results. For example, the code below will print “Match!” even though the tuple labels are different:

```
let singer = (first: "Taylor", last: "Swift")
let bird = (name: "Taylor", breed: "Swift")

if singer == bird {
    print("Match!")
} else {
    print("No match")
}
```

Typealiases

You’ve seen how powerful, flexible, and useful tuples can be, but sometimes you will want to formalize things just a bit. To give you a Swift-themed example, consider these two tuples, representing Taylor Swift’s parents:

Types

```
let father = (first: "Scott", last: "Swift")
let mother = (first: "Andrea", last: "Finlay")
```

(No, I don't have a collection of Taylor Swift facts, but I *can* use Wikipedia!)

When they married, Andrea Finlay became Andrea Swift, and they became husband and wife. We could write a simple function to represent that event:

```
func marryTaylorsParents(man: (first: String, last: String),
woman: (first: String, last: String)) -> (husband: (first:
String, last: String), wife: (first: String, last: String)) {
    (man, (woman.first, man.last))
}
```

Note: I've used "husband" and "wife", and also had the wife take her husband's surname, because that's what happened with Taylor Swift's parents.

The **father** and **mother** tuples seemed nice enough in isolation, but that **marryTaylorsParents()** function looks pretty grim. Repeating **(first: String, last: String)** again and again makes it hard to read and hard to change. Swift's solution is simple: the **typealias** keyword. This is *not* specific to tuples, but it's certainly most useful here: it lets you attach an alternate name to a type. For example, we could create a **typealias** like this:

```
typealias Name = (first: String, last: String)
```

Using that, the **marryTaylorsParents()** function becomes significantly shorter:

```
func marryTaylorsParents(man: Name, woman: Name) -> (husband:
Name, wife: Name) {
    (man, (woman.first, man.last))
}
```

Using tuples with a type alias is a nice middle ground between regular tuples and structs – you

get a slightly more formal definition, without needing to make any major changes to your code.

Ranges and RangeSet

Swift has multiple types of ranges available for us to use depending on our goal. They are implemented with slightly different types internally, but their fundamentals are very similar.

The most common two are **Range** and **ClosedRange**, which are made with the half-open range operator (`..<`) and the closed range operator (`...<`) like this:

```
let childAges1 = 0..<18
let childAges2 = 0...17
```

Although those are two different types internally, both represent all the values between 0 and 17 inclusive on both sides.

Tip: It's common to refer to these ranges using slightly different phrasing: "0 to 17" means "0 up to 17 but excluding 17 itself," whereas "0 *through* 17" includes that final number.

A second range type is a *partial* range, created using one-sided ranges. For example, we could make a range of numbers from 9001 upwards to the maximum value integers can store

```
let over9000 = 9001...
```

That includes 9001 itself, but goes upwards to the value of **Int.max**.

Partial ranges are particularly useful when slicing arrays, where you can read some parts of an array trivially:

```
let characters = ["Dr Horrible", "Captain Hammer", "Penny",
"Bad Horse", "Moist"]
let bigParts = characters[..<3]
let smallParts = characters[3...]
print(bigParts)
print(smallParts)
```


Dealing with bounds

Where things get a little trickier is when dealing with the bounds of these ranges. You see, closed ranges and half-open ranges both tell you their upper and lower bounds, but they don't tell you whether those bounds include or exclude the final number.

Try this to see the problem in action:

```
let childAges1 = 0..<18
let childAges2 = 0...17
print(childAges1.upperBound)
print(childAges2.upperBound)
```

That will print 18 and 17, but in the first case 18 is *excluded* and 17 is *included*.

For added fun, partial ranges *don't* have either a **lowerBound** or **upperBound** property at all, depending on which part of the range you miss off. For example, this is allowed:

```
let over9000 = 9001...Int.max
print(over9000.upperBound)
```

But this is not:

```
let over9000 = 9001...
print(over9000.upperBound)
```

That second option still includes **Int.max**, you just can't check for its upper bound directly.

Now, you might think code like this would work:

```
print(over9000.max(!))
```

...however, that's accidentally terrible code: it will manually loop over every integer in the range (9001 up to **Int.max**, which is over nine quintillion!), only to return the final number.

Types

When given a partial range, the best you can do is use its **relative(to:)** method, which creates a new range that is bounded on both sides by the bounds of another sequence. This might be an array, a closed range, or something else.

For example, we could take a one-sided range and get its bounds relative to our **characters** array from earlier:

```
let characters = ["Dr Horrible", "Captain Hammer", "Penny",  
"Bad Horse", "Moist"]  
let smallParts = 3..  
let bounded = smallParts.relative(to: characters)  
print(bounded.upperBound)
```

That now has a valid upper bound, because Swift knows the array has the upper bound of 5.

Over to RangeSet

Whether they are closed, half-open, or partial, all three ranges we've looked at so far always include their values contiguously. For example, the range of 3 through 6 will always include 3, 4, 5, and 6, and can't miss out numbers.

If you *do* want a non-contiguous values, they are provided through another range type called **RangeSet** – the "set" part in that name is there because **RangeSet** supports a variety of functions that come from Swift's **SetAlgebra** protocol, including **union()**, **intersection()**, and **isSuperset(of:)**. This also means that inserting one range into another will merge overlapping ranges rather than creating duplicates.

To demonstrate this, let's start with a simple array of values:

```
struct ExamResult {  
    var student: String  
    var score: Int  
}
```

```
let results = [
    ExamResult(student: "Eric Effiong", score: 95),
    ExamResult(student: "Maeve Wiley", score: 70),
    ExamResult(student: "Otis Milburn", score: 100)
]
```

We can get a **RangeSet** containing the indices of all students who score 85% or higher like this:

```
let topResults = results.indices { student in
    student.score >= 85
}
```

Similar to partial ranges, range sets don't have upper *or* lower bounds available to read, but you can still apply them to arrays as a subscript like this:

```
for result in results[topResults] {
    print("\(result.student) scored \(result.score)%")
}
```

This subscript is slightly different from when using the other range types: they all return array slices, but this returns a **DiscontiguousSlice**, meaning that the elements are necessarily adjacent in the original collection.

Checking a range contains a value

All our range types have a built-in **contains()** method that returns true if a value is contained inside the range. However, there also the pattern match operator `~=`, which lets you write code like this:

```
let range = 1...100
let i = 42
```

Types

```
if range ~= i {  
    print("Match!")  
}
```

That operator isn't strictly needed because you can code this using the built-in **contains()** method of ranges. However, it does have a slight syntactic sugar advantage over **contains()** because it doesn't require an extra set of parentheses:

```
let test1 = (1...100).contains(42)  
let test2 = 1...100 ~= 42
```

Ranges of everything comparable

All my examples so far have used integers for ranges, but actually you can make ranges from anything that conforms to the **Comparable** protocol.

So, both of these examples are perfectly valid:

```
let earlyAlphabet = "A"... "E"  
let next24Hours = Date.now...Date.now.addingTimeInterval(86400)
```

Be careful, though: ranges of things like characters rely on case sensitivity. You can see the problem with this sample code:

```
let uppercaseRange = "A"...  
let lowercaseRange = "a"...  
print(uppercaseRange.contains("A"))  
print(uppercaseRange.contains("a"))  
print(lowercaseRange.contains("A"))  
print(lowercaseRange.contains("a"))
```

That will print true, true, false, true, because a capital A comes before a lower case A in ASCII ordering. So, when we make a range from lowercase A onwards that will immediately exclude

capital A, but the same isn't true the other way around.

Generics

Even though generics are an advanced topic in Swift, you use them all the time: **[String]** is an example of you using the **Array** structure to store strings, which is an example of generics.

The truth is that *using* generics is straightforward, but *creating* them takes a little getting used to. In this chapter I want to demonstrate how (and why!) to create your own generics, starting with a function, then a struct, and finally wrapping a Foundation type.

Let's start with a simple generic function to square an integer. I'll show you the code first, then break it down:

```
func square<T: Numeric>(_ value: T) -> T {  
    value * value  
}
```

First, generic code in Swift uses angle brackets to mark out placeholders: places where we know some kind of data will be used, but we don't know what. I often call these Pulp Fiction brackets after John Travolta's dance moves in that movie!

These placeholders are formally called *generic type parameters*. **T** is one, and in this case means "I expect some kind of type to be used with this function, and it should conform to the **Numeric** protocol used by numbers in Swift.

T is better than using **Any** as a parameter and return type for two reasons. First, because we're using it in two places: we're saying the value we want to square will be of type **T**, and the return type from the function will also be of type **T**. And second, we get to add the restrictions that **T** must be something that conforms to Swift's **Numeric** protocol.

Using the same placeholder in both places tells Swift the same type must be used in both places, which makes sense here – if we passed in an integer, we would like an integer back as opposed to a **Double**.

We can use these placeholders as they are, or as part of other types. For example, if we had the

placeholder **T** we could use **[T]** to get arrays of that element, **T?** to get an optional of that element, and more.

To demonstrate this, here's a **mostFrequentElement()** method that takes an array of **T** and returns an optional **T?**, using a dictionary with **T** as its keys:

```
func mostFrequentElement<T: Hashable>(in array: [T]) -> T? {
    var frequencies = [T: Int]()

    for element in array {
        frequencies[element, default: 0] += 1
    }

    let item = frequencies.max { $0.value < $1.value }
    return item?.key
}
```

Let's break that down:

1. We add the constraint that **T** must conform to **Hashable**, because we need to be able to use objects of that type as dictionary keys.
2. Internally we make a new dictionary with **T** for keys and **Int** for values – the values will store how often each **T** instance appears.
3. We loop over the array, adding one to each **T** value every time we see it.
4. We then use **max()** to find the item with the highest count, and send it back.

In each of those instances, Swift enforces the rule that **T** refers to the same data type – we can't take an array of numbers and return an optional Boolean, for example.

Creating a generic data type

Now that you've mastered generic functions, let's take it up a notch with a fully generic data type: we're going to create a generic struct.

Types

When you're creating a generic data type you need to declare your placeholder data type as part of the struct's name, and that placeholder is then available to use in every property and method as needed.

The struct we're going to build is called "deque", which is a common abstract data type that means "double-ended queue." A regular queue is one where you add things to the end of the queue, and remove them from the front.

A *deque* (pronounced "deck") is a queue where you can add things to the beginning or the end, and remove things from the beginning or end too. I chose to use a deque here because it's very simple to do by re-using Swift's built-in arrays – the concept is the key here, not the implementation!

To create a deque struct, we're going to give it a stored array property that will itself be generic, because it needs to hold whatever data type our deque stores. We'll also add four methods: **pushBack()** and **pushFront()** will accept a parameter of type **T** and add it to the right place, and **popBack()** and **popFront()** will return a **T?** (a placeholder optional data type) that will return a value from the back or front if one exists.

There's only one tiny complexity, which is that arrays don't have a **popFirst()** method that returns **T?**, so we need to add some extra code to run when the array is empty. Here's the code:

```
struct Deque<T> {
    var array = [T]()

    mutating func pushBack(_ obj: T) {
        array.append(obj)
    }

    mutating func pushFront(_ obj: T) {
        array.insert(obj, at: 0)
    }
}
```



```

mutating func popBack() -> T? {
    array.popLast()
}

mutating func popFront() -> T? {
    if array.isEmpty {
        nil
    } else {
        array.removeFirst()
    }
}
}

```

With that struct in place, we can start using it immediately:

```

var testDeque = Deque<Int>()
testDeque.pushBack(5)
testDeque.pushFront(2)
testDeque.pushFront(1)
testDeque.popBack()

```

Working with Cocoa types

Cocoa data types – **NSArray**, **NSDictionary**, and so on – have been available to use since the earliest releases of Swift, but they can be difficult to work with because Objective-C’s support for generics is both recent and limited.

In the case of **NSCountedSet**, one of my favorite Foundation types, generics aren’t supported at all. This means you lose the automatic type safety bestowed by the Swift compiler, which in turn makes you one step closer to being a JavaScript programmer.

Fortunately, I’m going to demonstrate to you how to create your own generic data type by creating a generic wrapper around **NSCountedSet**. This is just like a regular **Set** in that each

Types

item is only stored once, but it has the added benefit that it keeps count of how many times you tried to add or remove objects. This means it can say “you added the number 5 twenty times” even though it’s actually only in there once.

The basic code for this isn’t too hard:

```
struct CustomCountedSet<T: Any> {
    let internalSet = NSCountedSet()

    mutating func add(_ obj: T) {
        internalSet.add(obj)
    }

    mutating func remove(_ obj: T) {
        internalSet.remove(obj)
    }

    func count(for obj: T) -> Int {
        internalSet.count(for: obj)
    }
}
```

With that new data type in place, you can use it like so:

```
var countedSet = CustomCountedSet<String>()
countedSet.add("Hello")
countedSet.add("Hello")
print(countedSet.count(for: "Hello"))

var countedSet2 = CustomCountedSet<Int>()
countedSet2.add(5)
print(countedSet2.count(for: 5))
```

All our struct does is wrap **NSCountedSet** with type safety, but that's always a welcome improvement. Given Apple's direction of travel with Swift, I wouldn't be surprised if they re-implement **NSCountedSet** as a generic struct-based **CountedSet** in the future – we'll see!

Key paths

Key paths are uninvoked references to properties, which is a fancy way of saying they refer to the property itself rather than the actual value of the property in a particular object.

For example, `\String.count` refers to the **count** property of any string as opposed to the **count** property of one specific string. We can store that in a key path constant like this:

```
let countPath = \String.count
```

Now we can make a couple of example strings:

```
let name1 = "Benia"  
let name2 = "Marcus"
```

And now we can use the **keyPath** subscript to read the count of those strings dynamically:

```
print(name1[keyPath: countPath])  
print(name2[keyPath: countPath])
```

That will print 5 then 6, reflecting the lengths of our two names – it reads the **count** property of each string by invoking our key path against each name we provide.

The type of **countPath** is important: it's a **KeyPath<String, Int>**, which means it's a key path that operates on strings and returns an integer when invoked. This makes it type safe to use, because we'll always get back an integer.

Key paths can access properties of any type, such as Booleans:

```
let emptyPath = \String.isEmpty
```

They can also dig into optionals if needed, although the result will automatically also be optional:

```
let asciiPath = \String.first?.isASCII
```

Where key paths get interesting is when they combine with the `@dynamicMemberLookup` attribute to power generic subscripts.

To show why this is so useful, I first want to show the *problem*. Consider this code, which creates a trivial `User` struct and `ViewModel` class:

```
struct User {
    var name: String
    var job: String
    var city: String
}

class ViewModel {
    var user: User

    init(user: User) {
        self.user = user
    }
}
```

This is quite a common set up, although in practice of course you'd have significantly more functionality in the view model!

Creating a user and view model is simple enough:

```
let marcus = User(name: "Marcus W", job: "Fashion Designer",
city: "Detroit")
let viewModel = ViewModel(user: marcus)
```

However, it gets a bit annoying trying to access the `user` object through the view model, because you end up writing code like this:

```
print(viewModel.user.name)
```

Types

```
print(viewModel.user.job)
```

@dynamicMemberLookup and key paths combine to let us do better. First, add this directly before the **ViewModel** class:

```
@dynamicMemberLookup
```

And now add this subscript:

```
subscript<T>(dynamicMember keyPath: KeyPath<User, T>) -> T {  
    user[keyPath: keyPath]  
}
```

I'll explain it a moment, but first I want you to see the impact – change your two **print()** calls to this:

```
print(viewModel.name)  
print(viewModel.job)
```

As you can see, we can access **name** and **job** directly on our view model, rather than always having to spell out the **user** property directly.

Let's go back to the subscript, because it's quite dense. There are only two actual lines of code there, so let's start with the first one:

```
subscript<T>(dynamicMember keyPath: KeyPath<User, T>) -> T {
```

This defines a new subscript, which normally takes the form of **something[someValue]** but with **@dynamicMemberLookup** gets translated to **something.someValue**.

The subscript has one generic type parameter, which I've called **T**. This is important, because we're saying that we expect to be given a key path that accesses our **User** struct and returns a **T**, and that our whole subscript returns the same **T**. So, if we're accessing the key path **\User.name**, a string, then our subscript also returns a string.

Inside the subscript, we forward whatever key path we were given straight onto the **user** property we're wrapping:

```
user[keyPath: keyPath]
```

The magic here is that Swift automatically handles bridging so that it looks like a direct property access – we can access every property from **User** directly from **ViewModel**, and it works great with code completion in Xcode too.

Chapter 4

References and Values

What's the difference between structs and classes?

Understanding and exploiting the difference between reference types and value types is a critically important skill for any serious Swift developer. It's not "useful knowledge," or "good to know," but critically important – I'm not saying that lightly.

Later on I'm going to go into detail about references and values so that you can learn for yourself, but first I want to start by explaining the difference between them. I've met too many developers who either don't understand or don't care – either of which is a mistake – so this is a sensible place to start.

I'm a big Star Wars fan, and you might remember that the end of the original Star Wars film (later titled "A New Hope") featured an attack on the Death Star. One of the Rebel pilots – Wedge Antilles – flew to attack the Death Star, but was damaged and had to fly back to base. Another Rebel pilot – Luke Skywalker – was also attacking the Death Star, but used the Force to save the day, at least until the next movie.

To illustrate the difference between references and values, I want to recreate this movie scene in simple Swift code. It's not complete, but it's enough you can follow along:

```
// create a target
var target = Target()

// set its location to be the Death Star
target.location = "Death Star"

// tell Luke to attack the Death Star
luke.target = target

// oh no – Wedge is hit! We need to
// tell him to fly home
```

References and Values

```
target.location = "Rebel Base"
wedge.target = target

// report back to base
print(luke.target.location)
print(wedge.target.location)
```

Now, the question is this: when those pilots report back to base, what will they say? The answer is, “it depends.” And what it depends on is – you guessed it – the difference between references and values.

You see, if **Target** was a class, which is a reference type in Swift, it would look like this:

```
class Target {
    var location = ""
}
```

And if **Target** was a struct, which is a value type in Swift, it would look like this:

```
struct Target {
    var location = ""
}
```

The code is almost identical, with just one word different. But the result is big: if **Target** was a struct then Luke would target the Death Star and Wedge would target the Rebel Base, but if it was a class then both Luke and Wedge would target the Rebel Base – which would make a very unhappy ending for the Rebels, and some deeply confused cinemagoers.

The reason for this behavior is that when you assign a reference type like a class in more than one place, all those places point to the same piece of data. So even though Luke has a **target** property and Wedge has a **target** property, they are both pointing to the same instance of the **Target** class – changing one means the other changes too.

What's the difference between structs and classes?

On the other hand, value types like structs always have only one owner. When you assign a value type in more than one place, all those places point to individual copies of that value. When the code `luke.target = target` is run, Luke gets his own unique copy of the **Target** instance. So, when that gets changed to “Rebel Base”, he doesn’t care – he continues his attack on the Death Star.

Swift is an aggressively value-oriented language, meaning that most of its data types are values rather than references. Booleans, integers, strings, tuples, and enums are all value types. Even arrays and dictionaries are value types, so the code below will print 3 and not 4:

```
var a = [1, 2, 3]
var b = a
a.append(4)
print(b.count)
```

Value types are simpler than reference types, which is no bad thing. When you’re giving a value to work with, you can be sure its value won’t change by surprise because you have your own unique copy.

You can also compare value types easily, and it doesn’t matter *how* they got their value – as long as two values look the same, they *are* the same. For example, the code below will print “Equal” even though **a** and **b** were created very differently:

```
let a = [1, 2, 3]
let b = Array(1...3)
if a == b { print("Equal") }
```

In short, references are shared on assignment and so can have multiple owners, whereas values are copied on assignment and therefore only ever have one owner.

Why use structs?

If you're faced with the choice between a class or a struct – i.e. a reference or a value – there are good reasons for choosing structs. This comparison applies to all reference and value types, but the most common time you need to make a choice is when it comes down to a class or a struct, so that's the context I'll be using here.

I already mentioned that value types are copied rather than shared, which means if your code has three different things pointing at the same struct they will each have their own copy – there's no chance of them treading on each others' toes.

This greatly reduces the number of relationships that exist in your code, and also means data tends to flow in one way – it looks more like a tree and less like an explosion in a spaghetti factory.

Structs also come with significant ease of use benefits over classes: Swift is able to provide structs with default conformances to **Equatable**, **Codable**, and **Hashable** if we ask for them, and it can also synthesize memberwise initializers for them.

For the protocols, Swift is able to make the whole struct conform to a protocol if each of its properties already conforms to the protocol.

As an example, this **Person** struct can conform to **Codable**, **Equatable**, and **Hashable** because its properties also conform to those protocols:

```
struct Person: Codable, Hashable {  
    var name: String  
    var age: Int  
    var greeting: String  
}
```

Tip: The **Hashable** protocol inherits from **Equatable**, so conforming to one includes the other automatically.

Swift isn't able to do the same for classes because of class inheritance – it would get really messy to automatically add **Codable** conformance, only to find that a subclass or parent class later changes to include a non-**Codable** property.

As for the synthesized memberwise initializer, this means the compiler automatically produces an initializer that provides default values for each of the struct's properties. So, we can create **Person** instances like this:

```
let taylor = Person(name: "Taylor Swift", age: 26, greeting:
"Hi, I'm Taylor.")
```

It's a small thing, I know, but it's immensely practical and makes structs that little bit easier to use.

This automatic initializer is removed if you add your own custom initializer, because Swift takes that as a signal that your initializer does important custom work. If you want to add a custom initializer while also retaining the memberwise initializer, add yours as an extension like this:

```
extension Person {
    init(name: String, age: Int) {
        self.name = name
        self.age = age
        self.greeting = "Hi! My name is \(name)"
    }
}
```

Now you can create a new **Person** using **Person(name:age:greeting:)** as provided by Swift, or use the custom **Person(name:age:)** initializer.

This behavior makes sense if you think about it: if adding a new initializer in an extension disabled the original initializer, it would break all sorts of code if we tried to make custom initializers for things like **String** and **Array**!

References and Values

I would recommend you choose structs over classes where possible. If you specifically need inheritance or shared ownership of data such as **@Observable** classes, then go with classes, but otherwise structs should always be your default starting point.

Why use classes?

There *are* reasons for preferring classes to structs, although at least one can come back to bite you. Reference types are a bit like names or handles, or even pointers if you're an old-school developer: when you pass them around, everyone points at the same value.

In practice, this means you can create a resource such as a database connection or a data model object and share it across lots of places in your app.

As a simple example, we could create a **File** class that is able to write data to a filename, but does so by building up its contents over time then writing it all out to disk at once:

```
class File {
    var filename: String
    var contents = ""

    init(filename: String) {
        self.filename = filename
    }

    func write(_ text: String) {
        contents += text
    }

    func save() {
        print("Writing \(contents) to \(filename)")
    }
}
```

Because that's a class, we could open the file once, sharing it and adding to it from many parts of our app, then call **save()** and have all changes from everywhere written out – although of course if you use Swift concurrency you'll need to make sure it's all shared safely!

References and Values

One feature that classes give you that structs don't is inheritance: the ability to take an existing class and build upon it. For many years now – easily one or even two generations of developers – inheritance has been a fundamental concept in software development, and now is deeply ingrained in our industry. It allows you to take an existing class, large or small, and build upon it in any way you want.

Finally, one major reason for using classes is that so many of Apple's older frameworks use them – UIKit, SpriteKit, MapKit, Core Location, and more are all object-oriented.

Even some new API such as **@Observable** and `SwiftData` require classes. In fact, if you think about it something like `SwiftData` would be rather hard to implement *without* classes – we need *some* way to share data while also keeping our UI synchronized.

So, if you're writing Swift code that needs to work with those frameworks, you're going to rely on classes much more heavily whether you like it or not.

Tip: It's good practice to mark classes as **final** by default. Unless you have specifically thought, “yes, this class is safe to be subclassed by others” then it's a mistake to allow it to happen. Do not underestimate the complexity of robust subclassability!

Immutability in structs and classes

Value and reference types differ in the way they handle immutability, but it's quite a subtle difference and it's easy to get confused.

Let me back track a little: one thing I love about Swift is that it's aggressively focused on immutability. That is, it's very easy to say “don't let this value ever change.” Not only does this mean that you're encouraged to use **let** rather than **var** when writing your code, but the Swift compiler will scan your code and warn when it finds places where variables could be made into constants.

This is very different from Objective-C, where mutability was enforced only as part of your class name – **NSString** had an **NSMutableString** counterpart, **NSArray** had an **NSMutableArray** counterpart, and so on.

Immutability and value types might seem to go hand in hand. After all, if a value is just a value, how can it change? Integers are value types in Swift, and you can't exactly say “hey, I'm changing the number 5 so that 5 is actually equal to 6 now.” But when you start comparing classes and structs, immutability is more complicated, and I want to explain why.

Consider the following code:

```
struct PersonStruct {  
    var name: String  
    var age: Int  
}  
  
var taylor = PersonStruct(name: "Taylor Swift", age: 26)  
taylor.name = "Justin Bieber"  
print(taylor.name)
```

When that code runs, the final name value of the **taylor** instance is “Justin Bieber,” which is going to be a huge surprise for any Swifties attending a concert!

References and Values

If we change just one line of code, the result is very different:

```
let taylor = PersonStruct(name: "Taylor Swift", age: 26)
taylor.name = "Justin Bieber"
```

With just one line different, the code won't even compile because changing the **name** property is disallowed. Even though the **name** and **age** properties are marked as variable, the **taylor** struct is marked as a constant so Swift will not allow any part of it to change.

This is why in almost all my code, I use variables for struct properties rather than constants, unless I really need a value to be constant – an **id** would be constant, for example. With this approach, if I want those to be constant, I use **let** for the whole struct, but otherwise I have the flexibility to change individual values.

Here's where things get a bit complicated, but please knuckle down and bear with me: this is really important. Consider the following code:

```
final class PersonClass {
    var name: String
    var age: Int

    init(name: String, age: Int) {
        self.name = name
        self.age = age
    }
}
```

```
let taylor = PersonClass(name: "Taylor Swift", age: 26)
taylor.name = "Justin Bieber"
```

That example uses a **PersonClass** class rather than a struct, but it leaves the **taylor** instance as being a constant. Now the code compiles, which is very different from when it was a struct.

When **taylor** is a constant struct, you can't change its value or its properties. When it's a

constant class, you can't change its value but you *can* change its properties, as long as they aren't marked individually as constant. This means using structs allows you to enforce immutability far more strongly than using classes, which can help simplify your code even further.

The code below shows off all possible options – I've commented out the code that won't work:

```
// variable struct: changing property and changing value okay
var taylor1 = PersonStruct(name: "Taylor Swift", age: 26)
taylor1.name = "Justin Bieber"
taylor1 = PersonStruct(name: "Justin Bieber", age: 22)

// constant struct: changing property or value not allowed
let taylor2 = PersonStruct(name: "Taylor Swift", age: 26)
//taylor2.name = "Justin Bieber"
//taylor2 = PersonStruct(name: "Justin Bieber", age: 22)

// variable object: changing property and reference okay
var taylor3 = PersonClass(name: "Taylor Swift", age: 26)
taylor3.name = "Justin Bieber"
taylor3 = PersonClass(name: "Justin Bieber", age: 22)

// constant object: changing property okay, changing reference
not allowed
let taylor4 = PersonClass(name: "Taylor Swift", age: 26)
taylor4.name = "Justin Bieber"
//taylor4 = PersonClass(name: "Justin Bieber", age: 22)
```

As you can see, the difference occurs where constants are used: a constant class instance cannot be changed to point to a new instance but you can change any of its properties, whereas a constant struct is totally fixed.

The most common place where mutability is a concern is when dealing with collections, such

References and Values

as arrays. These are value types in Swift (hurray!) but they can also be mutable – you can add and remove elements freely if you declare them as **var**. If you are able to, declare them as immutable so that Swift can optimize their use and provide extra safety.

Closures are reference types

As you know, Booleans, numbers, strings, arrays, dictionaries, structs, and more are all value types in Swift. Classes and actors are reference types, but so are closures. For simple closures, this doesn't really matter.

For example, the code below stores a trivial closure in **printGreeting**, calls it, assigns it to **copyGreeting**, then calls *that*:

```
let printGreeting = { print("Hello!") }
printGreeting()

let copyGreeting = printGreeting
copyGreeting()
```

In that code, the closure could be a value type and it wouldn't affect the output. Where things become complicated is when closures capture values, because any captured values are shared amongst any variables pointing at the same closure.

To give you a practical example, the code below is a **createIncrementer()** function that accepts no parameters and returns a closure:

```
func createIncrementer() -> () -> Void {
    var counter = 0

    return {
        counter += 1
        print(counter)
    }
}
```

Inside the **createIncrementer()** function is a single variable, **counter**, which is initialized to 0. Because that variable is used inside the closure that gets returned, it will be captured. So, we

References and Values

can follow that function with code like this:

```
let incrementer = createIncrementer()  
incrementer()  
incrementer()
```

The first line calls **createIncrementer()** and stores its returned closure in **incrementer**. The following two lines call **incrementer()** twice, triggering the closure, so you'll see 1 then 2 being printed out – the counter is moving upwards as expected.

Now for the important bit: because closures are references, if we create another reference to **incrementer**, they will share the same closure, and thus will also share the same **counter** variable. For example:

```
let incrementerCopy = incrementer  
incrementerCopy()  
incrementer()
```

When that code is run, you'll see 1, 2, 3, 4 being printed out, because both **incrementer** and **incrementerCopy** are pointing at the exact same closure, and therefore at the same captured value.

If you use closures infrequently this is not likely to be a problem, and if you aren't using them to capture values then you're safe. Otherwise, step carefully: working with reference types can be hard enough without also introducing shared closure capturing.

Mixing classes and structs

Once you understand value types and reference types, it's much easier to choose between a class and a struct – at least at first. But as your application grows, you might find your situation becomes less black and white: your app might work 95% of the time with value types, but maybe once or twice everything would have been a whole lot easier if you had a reference type instead.

All is not lost: bearing in mind everything I've said about the relative advantages and disadvantages of both, there is a way to share value types in several places if you're convinced it's the right solution.

The technique is called boxing – not the punchy, sweaty sort of boxing, but as in “placing something inside a box.” This approach wraps a value type inside a reference type so you can share it more easily, and is commonly seen in languages such as C# and Java.

Note: I'm not going to continue repeating the warning that sharing rather than copying values raises the complexity of your program; please just take it as read!

I want to walk you through a practical example so you can see for yourself how it works. First, here's a **Person** struct we can use:

```
struct Person {  
    var name: String  
    var age: Int  
    var favoriteIceCream: String  
}  
  
let taylor = Person(name: "Taylor Swift", age: 26,  
favoriteIceCream: "Chocolate")
```

If we want to share that **taylor** struct across multiple objects, we'd create a **PersonBox** class like this:

References and Values

```
final class PersonBox {  
    var person: Person  
  
    init(person: Person) {  
        self.person = person  
    }  
}  
  
let box = PersonBox(person: taylor)
```

That is a class container around the **Person** struct, and as a reference type will be shared rather than copied.

Finally, let's create a **TestContainer** class that simulates some parts of your app, such as different view controllers:

```
final class TestContainer {  
    var box: PersonBox!  
}  
  
let container1 = TestContainer()  
let container2 = TestContainer()  
  
container1.box = box  
container2.box = box
```

That creates two containers, each of which point at the same **PersonBox** object, which in turn means they are pointing at the same **Person** struct. To prove this works, we could write code like this:

```
print(container1.box.person.name)  
print(container2.box.person.name)
```



```

box.person.name = "Not Taylor"

print(container1.box.person.name)
print(container2.box.person.name)

```

That will print “Taylor Swift” twice, then “Not Taylor” twice, proving that changing the value in one container changes the value in the other.

If you intend to make extensive use of boxing and unboxing (warning warning blah blah you could write this yourself, I hope!), you might want to consider creating a generic **Box** class like this:

```

final class Box<T> {
    var value: T

    init(value: T) {
        self.value = value
    }
}

final class TestContainer {
    var box: Box<Person>!
}

```

That way you can share other types of structs without having to create lots of different box classes.

It’s undeniable that this approach weakens the power and safety of value types a little, but at least it makes explicit which situations give you the safety of value types and which don’t – you’re stating “this bit is explicitly shared” rather than implicitly sharing everything.

There’s one more thing you should know, and it’s something that’s fairly obvious if you’re coming from an Objective-C background: boxing and unboxing can be helpful if you’re facing

References and Values

significant referencing counting performance issues.

Swift, like modern Objective-C, uses a system called Automatic Reference Counting (ARC), which keeps track of how many times an object is referenced. When that count reaches 0, the object is automatically destroyed.

Swift's structs are *not* reference counted, because they are always uniquely referenced. But if a struct contains a class object as one of its properties, that object *is* reference counted. For something small this isn't a problem, but if your struct has lots of objects as properties – let's say 10 – then suddenly ARC has to do 10 reference increments and decrements each time your struct is copied.

In this situation, boxing your struct in a wrapper object simplifies things dramatically, because ARC would only need to manipulate the reference count for the box, not for all the individual properties.

Chapter 5

Functions and Closures

Variadic functions

Variadic functions are functions of indefinite arity, which is a fancy way of saying that they take as many parameters as you send. This is used in some basic functions – even **print()** – to make your code easier and safer to write.

Let's work with **print()**, because it's a function you know well. You're used to seeing code like this:

```
print("I'm Commander Shepard and this is my favorite book")
```

But **print()** is a variadic function, which means you can pass it any number of things to print:

```
print(1, 2, 3, 4, 5, 6)
```

That will produce different output to calling **print()** once for each number: using one call prints the number on a single line, whereas using multiple calls would print the numbers one per line.

The variadic nature of **print()** becomes more useful once you add in the optional extra parameters: **separator** and **terminator**. The first one places a string between each of the values you passed, and the second places a string once all values have been printed. For example, this will print “1, 2, 3, 4, 5, 6!”:

```
print(1, 2, 3, 4, 5, 6, separator: ", ", terminator: "!")
```

So, that's how to *call* variadic functions. Let's now talk about making them, which I think you'll find is quite slick in Swift.

Consider the code below:

```
func add(numbers: [Int]) -> Int {  
    var total = 0
```

```

    for number in numbers {
        total += number
    }

    return total
}

let result = add(numbers: [1, 2, 3, 4, 5])
print(result)

```

That function accepts an array of integers, then adds each number together to form a total. There are more efficient ways of doing this, but that's not the point of this chapter!

To make that function variadic – i.e., so that it accepts any number of individual integers rather than a single array – two changes are needed. First, rather than writing `[Int]` we need to write `Int...`. Second, rather than writing `add(numbers: [1, 2, 3, 4, 5])` we need to write `add(numbers: 1, 2, 3, 4, 5)`. And that's it. So, the final code is this:

```

func add(numbers: Int...) -> Int {
    var total = 0

    for number in numbers {
        total += number
    }

    return total
}

add(numbers: 1, 2, 3, 4, 5)

```

You can place your variadic parameter anywhere in your function's parameter list, and you're even allowed more than one if you want.

Operator overloading

Operators are the little symbols we use to check or transform values, such as `+`, `*`, or `==`, and operator overloading is the ability to implement your own operators or even adjust the existing ones.

The main reason for using operator overloading is that it gives you very clear, natural, and expressive code. You already understand that `5 + 5` equals 10 because you understand basic maths, so it's a logical extension to allow `myShoppingList + yourShoppingList`, i.e. the addition of two custom structs.

There are several downsides to operator overloading. First, its meaning can be opaque: if I say `horse1 + horse2`, is the result a baby horse, a horse race, or some weird, eight-legged mutant?

Second, it does nothing that methods can't do: `horse1.race(against: horse2)` would have the same result, and is significantly clearer.

Third, it hides complexity: `5 + 5` is a trivial operation, but perhaps `horse1 + horse2` involves a whole bunch of calculations to create and run a horse race.

Fourth, and perhaps most seriously, operator overloading can produce unexpected results, particularly because you can adjust the existing operators with impunity.

The basics of operators

Operator overloading is done using functions, which makes them easy to try out.

For example, you might write a custom `==` operator that compares two instances of a custom struct like this:

```
struct Horse {  
    var name: String  
  
    static func ==(lhs: Horse, rhs: Horse) -> Bool {
```

Functions and Closures

```
        lhs.name == rhs.name
    }
}

let first = Horse(name: "BoJack")
let second = Horse(name: "Sad Ken")

if first == second {
    print("Same name")
} else {
    print("Different name")
}
```

When that code runs, it will print "Different name" because the horses have different names, but let's break what's happening here:

1. The name of the function is the operator itself – the function is literally called `==`.
2. In Swift, the `==` operator is an *infix* operator, which means it becomes between two values, known as *operands*.
3. Because those operands are placed either side of the operator, e.g. `3 == 5`, we usually name them **lhs** and **rhs** for "left-hand side" and "right-hand side".
4. The function must return a Boolean that reports whether they are considered equal.

So, the operator is `==`, and usually you'll hear the left-hand and right-hand values being referred to as the *operands* – the things our operator is working on.

As well as a function that does the actual work, operators also have precedence and associativity, both of which affect the result of the operation.

When multiple operators are used together without brackets, Swift uses the operator with the highest precedence first. You probably learned this as PEMDAS (Parentheses, Exponents, Multiply, Divide, Add, Subtract), or BODMAS, or similar depending on where you went to school.

If precedence alone is not enough to decide the order of operations, associativity is used: does this operator work on things to its left, or to its right?

Swift lets you control both precedence and associativity, and it's important you understand what they mean. So, let's try an experiment now: what value should the following operation result in?

```
let i = 1 + 10 * 5
```

Following PEMDAS, the multiplication should be executed first ($5 * 10 = 50$), then the addition ($50 + 1 = 51$), so the result is 51.

This precedence is baked right into Swift – if you visit <https://github.com/swiftlang/swift> you can find the source code for Swift itself, and in `stdlib/public/core/Policy.swift` you'll find this code:

```
precedencegroup AdditionPrecedence {
    associativity: left
    higherThan: RangeFormationPrecedence
}

precedencegroup MultiplicationPrecedence {
    associativity: left
    higherThan: AdditionPrecedence
}
```

That declares two operator precedence groups, which are things that are evaluated in the same order. It doesn't actually define what's *in* those groups yet, just their relative order.

You can see that **MultiplicationPrecedence** is marked as being higher than **AdditionPrecedence**, which is what makes multiplication be evaluated before addition.

If you scroll a little further down you'll see these:

Functions and Closures

```
infix operator * : MultiplicationPrecedence
infix operator + : AdditionPrecedence
infix operator - : AdditionPrecedence
```

Now Swift is explicitly saying which operators belong to which groups.

Remember, these three operators are called “infix” because they are placed inside two operands, i.e. $5 + 5$. This is as opposed to *prefix* operator ("comes before one value") or a *postfix* operator ("comes after one value").

Swift doesn't have many postfix operators, but one it *does* have is `!` – the force unwrap. This isn't declare in the current file because it's baked into a lower level of Swift, but it's placed after a single operand. Cunningly, `!` is also a prefix operator, placed before its operand, except there it means "not".

Swift lets us redefine the precedence for existing operators by assigning them to new groups. You can create your own precedence groups if you want, or just re-use an existing one.

If you wanted to introduce a little chaos into your workplace, you could cause all sorts of curious behavior by rewriting the `*` operator like this:

```
infix operator * : RangeFormationPrecedence
```

That redefines `*` to have a lower precedence than `+`, which means this code will now return 55:

```
let i = 1 + 10 * 5
```

That's the same line of code as before, but now it will be executed with the addition first ($10 + 1 = 11$) followed by the multiplication ($5 * 11$) to give 55.

When two operators have the same precedence, associativity comes into play. For example, consider the following:

```
let i = 10 - 5 - 1
```

Look again how Swift's own code declares the **AdditionPrecedence** group, to which the `-` operator belongs:

```
precedencegroup AdditionPrecedence {
    associativity: left
    higherThan: RangeFormationPrecedence
}
```

As you can see, it's defined as having left associativity, which means **10 - 5 - 1** is executed as **(10 - 5) - 1** and *not* **10 - (5 - 1)**. The difference is subtle, but important: unless we change it, **10 - 5 - 1** will give the result 4. Of course, if you want to cause more havoc, you can change it like this:

```
precedencegroup AdditionPrecedence {
    associativity: right
    higherThan: RangeFormationPrecedence
}
```

```
infix operator - : AdditionPrecedence
let i = 10 - 5 - 1
```

That modifies the existing **AdditionPrecedence** group, then updates the `-` with the change – now the sum will be interpreted as **10 - (5 - 1)**, which is 6.

Adding to an existing operator

Now that you understand how operators work, let's modify the `*` operator so that it can multiply arrays of integers like this:

```
let result = [1, 2, 3] * [4, 5, 6]
```

Once we're done, that will return a new array containing [4, 10, 18], which is 1x4, 2x5 and

3x6.

The `*` operator already exists, so we don't need to declare it. Instead, we just need to create a new **func** `*` that takes our new data types. This function will create a new array made of multiplying each item in the two arrays it is provided. Here's the code:

```
func *(lhs: [Int], rhs: [Int]) -> [Int] {
    let limit = min(lhs.count, rhs.count)
    var result = [Int]()

    for i in 0..
```

Note that I added a **min()** call at the beginning to ensure we only use as many numbers as the smaller of the two arrays.

Because the `*` operator already exists, the important thing is the **lhs** and **rhs** parameters, both of which are integer arrays: these parameters are what ensure this new function is selected when two integer arrays are multiplied.

Of course, thanks to Swift's generics we can make this more flexible by replacing **Int** with some kind of **Numeric** value:

```
func *<T: Numeric>(lhs: [T], rhs: [T]) -> [T] {
    let limit = min(lhs.count, rhs.count)
    var result = [T]()

    for i in 0..
```

```
    return result  
}
```

And, just to give you an idea of what lies ahead, later in this book you'll learn a great deal about functional programming, where you'll also see how that same function can be boiled down to this:

```
func *<T: Numeric>(lhs: [T], rhs: [T]) -> [T] {  
    zip(lhs, rhs).map(*)  
}
```

Beautiful!

Adding a new operator

When you add a new operator, you need to give Swift enough information to use it.

At the very least you need to specify the new operator's position (prefix, postfix, or infix), but if you don't specify a precedence or associativity Swift will provide default values that make it a low-priority, non-associative operator.

Let's add a new operator, `**`, which returns one value raised to the power of another. That is, `2 ** 4` should equal `2 * 2 * 2 * 2`, i.e. 16.

The first step is to tell Swift that `**` will be an infix operator, because we'll have one operand on its left and another on its right:

```
infix operator **
```

That *doesn't* specify a precedence or associativity, so the defaults will be used.

Finally, the new `**` function itself. I've made it accept doubles for maximum flexibility, and Swift is smart enough to infer 2 and 4 as doubles when used with this operator:

```
func ** (lhs: Double, rhs: Double) -> Double {  
    pow(lhs, rhs)  
}
```

As you can see, the function itself is a cinch thanks to `pow()`. Try it yourself:

```
let result = 2 ** 4  
print(result)
```

So far, so good. However, expressions like this one won't work:

```
let result = 4 ** 3 ** 2
```

In fact, even something like this won't work either:

```
let result = 2 ** 3 + 2
```

This is because we're using default precedence and associativity. To fix that, we need to decide where `**` should rank compared to other operators, and for that you can either go back to PEMDAS (it's the E!) or just look at what other languages do.

Haskell, for example, places it before multiplication and division, following PEMDAS. Haskell also declares exponentiation to be right associativity, meaning that `4 ** 3 ** 2` would be parsed as `4 ** (3 ** 2)`.

We can make our own `**` operator behave the same way by modifying its declaration to this:

```
precedencegroup ExponentiationPrecedence {
    higherThan: MultiplicationPrecedence
    associativity: right
}

infix operator **: ExponentiationPrecedence
```

With that change, you can now use `**` twice in the same expression, and also combine it with other operators – much better!

Modifying an existing operator

Now for something more complicated: modifying an existing operator. I've chosen a slightly more complicated example because if you can watch me solve it here I hope it will help you solve your own operator overloading problems.

The operator I'm going to modify is `...`, which exists already as the closed range operator. So, you can write `1...10` and get back a range covering 1 through 10 inclusive.

By default this is an infix operator with the low end of the range on the left and the high end of the range on the right, but I'm going to modify it so that it also accepts a range on the left and

Functions and Closures

another integer on the right, like this:

```
let range = 1...10...1
```

When that code is run, it will return an array containing the numbers 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 – it counts up then down.

This is possible because the operator appears twice: the first time it will see **1...10**, which is the closed range operator, but the second time it will see **CountableClosedRange<Int>...1**, which is going to be our new operation.

In this function, the **CountableClosedRange<Int>** is the left-hand side operand and the **Int 1** is the right-hand side operand.

The new **...** function needs to do two things:

1. Calculate a new range by going from the right-hand integer up to the highest point in the left-hand range, then reversing this range.
2. Append the left-hand range to the newly created downward range, and return that as the result of the function.

In code, it looks like this:

```
func ... (lhs: CountableClosedRange<Int>, rhs: Int) -> [Int] {  
    let downwards = (rhs..<lhs.upperBound).reversed()  
    return Array(lhs) + downwards  
}
```

If you try that code out, you'll see it *won't* work – at least not yet. To see why, take a look at Swift's definition of the **...** operator:

```
infix operator ... : RangeFormationPrecedence  
  
precedencegroup RangeFormationPrecedence {
```



```
    higherThan: CastingPrecedence
}
```

And now take a look at our code again:

```
let range = 1...10...1
```

You can see we're using the ... operator twice, which means Swift needs to know whether we mean **(1...10)...**1 or **1...(10...1)**. As you can see above, Swift's definition of ... doesn't mention its associativity, so Swift doesn't know what to do in this situation. So, as things stand our new operator will work only with code like this:

```
let range = (1...10)...1
```

If we want that same behavior without the user having to add the parentheses, we need to tell Swift that ... has left associativity, like this:

```
precedencegroup RangeFormationPrecedence {
    associativity: left
    higherThan: CastingPrecedence
}

infix operator ... : RangeFormationPrecedence
```

That's it: the code now works correctly without parentheses, and we have a useful new operator.

```
precedencegroup RangeFormationPrecedence {
    associativity: left
    higherThan: CastingPrecedence
}

infix operator ... : RangeFormationPrecedence
```

Functions and Closures

```
func ... (lhs: CountableClosedRange<Int>, rhs: Int) -> [Int] {  
    let downwards = (rhs..    return Array(lhs) + downwards  
}
```

```
let range = 1...10...1  
print(range)
```

Of course, on the flip side we also just changed a well-known operator in Swift, so I'd be very wary about dropping this into big projects!

Closures: the basics

Like tuples, closures are endemic in Swift: global functions are closures, nested functions are closures, functional methods like **sort()** and **map()** accept closures, lazy properties use closures, and that's just the tip of the iceberg.

You *will* need to use closures during your Swift career, and if you want to advance to a senior development position then you'll need to be comfortable *creating* them too.

I know some people have unusual ideas of what closures are, so let's start with a simple definition: a closure is a block of code that can be passed around and stored like a variable, which also has the ability to capture any values it uses. This capturing is really what makes closures hard to understand, so we'll come back to it later.

Let's create a trivial closure to get things moving:

```
let greetPerson = {  
    print("Hello there!")  
}
```

That creates a closure called **greetPerson**, which can then be used like a function:

```
greetPerson()
```

Because functions like closures are first-class data types – i.e., just like integers, strings, and others – you can copy them and use them as parameters to other functions. Here's copying in action:

```
let greetCopy = greetPerson  
greetCopy()
```

When copying closures, remember that closures are a reference type – both those “copies” in fact point to the same shared closure.

Functions and Closures

To pass a closure into a function as a parameter, specify the closure's own parameter list and return value as its data type. That is, rather than writing **param: String**, you would write something like **param: () -> Void** to accept a closure that has no parameters and returns no value. Yes, the **-> Void** is required, otherwise **param: ()** would mean an empty tuple.

If we wanted to pass our **greetPerson** closure into a function and call it there, we'd use code like this:

```
func runSomeClosure(_ closure: () -> Void) {
    closure()
}

runSomeClosure(greetPerson)
```

Of course, there isn't much point to that code: why is a closure even needed? Well, in that example it isn't, but we could adjust our function so that it calls a closure only 50% of the time by using **Bool.random()**:

```
func randomly(_ closure: () -> Void) {
    if Bool.random() {
        closure()
    }
}
```

And now this *might* print a message, or it might not:

```
randomly {
    print("Hello, world!")
}
```

We could also pass in an existing closure directly:

```
randomly(greetPerson)
```

Similarly, what if we wanted to call the closure only after some number of seconds had passed?

This would need to use **async throws** because we need to use **Task.sleep(for:)** like this:

```
func delay(by seconds: Double, closure: () -> Void) async
throws {
    try await Task.sleep(for: .seconds(seconds))
    closure()
}
```

And now we can print our message after a delay:

```
try await delay(by: 1) {
    print("Hello, world!")
}
```

This is where closures become useful: they are pieces of functionality that your app can store away to be used later on if needed.

Where closures start to get confusing is when they accept their own parameters, partly because their parameter list is placed in an unusual location, but also because the type syntax for these closures can look quite messy!

First: how to make a closure accept parameters. To do this, write your parameter list *inside* the closure's braces, followed by the keyword **in**:

```
let greetPerson = { (name: String) in
    print("Hello, \(name)!")
}
```

```
greetPerson("Taylor")
```

This is also where you specify a capture list if you need one. This is most commonly used to

Functions and Closures

avoid strong reference cycles with **self** by making it **weak** like this:

```
let greetPerson = { [weak self] (name: String) in
    print("Hello, \(name)!")
}
```

```
greetPerson("Taylor")
```

Now, how to pass parameter using closures to functions. This is complicated for two reasons:

1) it can start to look like a sea of colons and parentheses, and 2) the calling convention varies depending on what you're trying to do.

Let's go back to our **runSomeClosure()** function, so that it accepts a closure that itself accepts one parameter. We need to define it like this:

```
func runSomeClosure(_ closure: (String) -> Void)
```

So, **closure** is a function that accepts a string and returns nothing. Here is that new function in action:

```
let greetPerson = { (name: String) in
    print("Hello, \(name)!")
}
```

```
func runSomeClosure(_ closure: (String) -> Void) {
    closure("Taylor")
}
```

```
runSomeClosure(greetPerson)
```

Closure capturing

You should already know that closures are reference types, which has cunning implications for captured values: when two variables point at the same closure, they both use the same captured data.

Let's start with the basics: when a closure references a value, it needs to ensure that value will still be around when the closure is run. This can seem like the closure is copying the data, but it's actually more subtle than that. The process is called capturing, and it allows a closure to refer to and modify values that it refers to, even when the original values no longer exist.

The distinction is important: if the closure *copied* its values, value type semantics would apply and any changes to a value type inside a closure would be on a unique copy that would not affect the original caller. Instead, closures *capture* data.

I realize this all sounds hypothetical, so let me give you a practical example:

```
func testCapture() -> () -> Void {
    var counter = 0

    return {
        counter += 1
        print("Counter is now \(counter)")
    }
}
```

```
let greetPerson = testCapture()
greetPerson()
greetPerson()
greetPerson()
```

```
let greetCopy = greetPerson
greetCopy()
```

Functions and Closures

```
greetPerson()  
greetCopy()
```

That declares a function called **testCapture()**, which has the return value **() -> Void** – i.e., it returns a function that accepts no parameters and returns nothing.

Inside **testCapture()** I created a new variable called **counter**, giving it the initial value of 0. However, nothing else happens to that variable inside the function. Instead, it returns a closure that adds one to **counter** and prints out its new value. It doesn't *call* that closure, it just returns it.

Where things get interesting is after the function: **greetPerson** is set to be the function returned by **testCapture()**, and it gets called three times. That closure references the **counter** value that was created inside **testCapture()**, which is clearly out of scope now because that function finished.

So, Swift *captured* the value: that closure now has its own independent reference to **counter** that can be used when ever it's called. Each time the **greetPerson()** function is called, you'll see **counter** increment by one.

Where things get *doubly* interesting is with **greetCopy**. This is what I was saying about closures being references and using the same captured data.

When **greetCopy()** is called, it increments the *same* **counter** value as **greetPerson** because they are both pointing at the same captured data. This means the **counter** value will move from 1 to 6 as the closure is called again and again.

I've talked about this quirk twice now, so if it hurts your brain don't worry: it's not covered again!

Closure shorthand syntax

Before you progress onto more advanced features of closures, I want to quickly go over the closure shorthand syntax in full so that we're definitely on the same wavelength.

When you pass an inline closure to a function, Swift has several techniques in place so you don't have to write as much code.

To give you a good example, I'm going to use the **filter()** method of arrays, which accepts a closure that takes one element from the array as its only parameter, and returns true if that element should be in a new array.

The code below filters an array so that we end up with a new array containing everyone whose name starts with Michael:

```
let names = ["Michael Jackson", "Taylor Swift", "Michael Caine", "Adele Adkins", "Michael Jordan"]

let result1 = names.filter({ (name: String) -> Bool in
    if name.hasPrefix("Michael") {
        true
    } else {
        false
    }
})

print(result1.count)
```

From that you can see that **filter()** expects to receive a closure that accepts a string parameter named **name** and returns true or false. The closure then checks whether the name has the prefix "Michael" and returns a value.

Swift knows that the closure being passed to **filter()** must accept a string and return a Boolean, so we can remove it and just use the name of a variable that will be used for each item to filter:

Functions and Closures

```
let result2 = names.filter({ name in
    if name.hasPrefix("Michael") {
        true
    } else {
        false
    }
})
```

Next, we can return the result of **hasPrefix()** directly, like this:

```
let result3 = names.filter({ name in
    name.hasPrefix("Michael")
})
```

Trailing closures allow us to remove one set of parentheses, which is always welcome:

```
let result4 = names.filter { name in
    name.hasPrefix("Michael")
}
```

Many people stop here, and with good reason: the next step can be quite confusing at first.

You see, when this closure is called, Swift automatically creates anonymous parameter names made up of a dollar sign then a number counting up from 0. So, **\$0**, **\$1**, **\$2**, and so on. You aren't allowed to use names like that in your own code so these tend to stand out!

These shorthand parameter names map to the parameters that the closure accepts. In this case, that means **name** is available as **\$0**. You can't mix explicit arguments and anonymous arguments: either you declare a list of the parameters coming in, or you use **\$0** and friends. Both of these two do exactly the same thing:

```
let result5 = names.filter { name in
    name.hasPrefix("Michael")
}
```

```

}

let result6 = names.filter {
    $0.hasPrefix("Michael")
}

```

Notice how you have to remove the **name in** part when using the anonymous names? Yes, it means even less typing, but at the same time you give up a little bit of readability.

If you choose to go with shorthand names, it's common at this point to put the whole method call on a single line, like this:

```
let result7 = names.filter { $0.hasPrefix("Michael") }
```

When you compare that to the size of the original closure, you have to admit it's a big improvement!

Choosing to use short-hand names vs explicit names is personal preference, but over the years I've found that short-hand names work best only when two things are true:

1. The function I'm using is short – maybe 1-3 lines. Anything more than that means the saving of using shorthand names is negated by the rest of the code.
2. It's one of Swift's built-in methods, e.g. **filter()**, **map()**, and **sort()**, where the meanings of the shorthand names are well known.

Closures, operators, and functions

Swift really blurs the lines between functions, methods, operators, and closures, which is marvelous because it hides all the compiler complexity from you and leaves developers to do what we do best: make great apps.

This blurry behavior can be hard to understand at first, and is even harder to use in day-to-day coding, but I want to show you two examples that I hope will demonstrate just how clever Swift is.

My first example is this: given an array of strings called **words**, how can you find out whether any of those words exist in a string named **input**? One possible solution is to break up **input** into its own array, then loop over both arrays looking for matches. But Swift gives us a better solution, and I want to look at it now.

First, you'll know that strings have a method called **contains()** that accepts another string and returns a Boolean. So, this code will return true:

```
let input = "My favorite album is Fearless"
print(input.contains("album"))
```

Swift arrays also have two **contains()** methods: one where you specify an element directly (a string, for example), but another that accepts a closure using a **where** parameter. That closure needs to accept a string and return a Boolean, like this:

```
let words = ["1989", "Fearless", "Red"]

words.contains { (str) -> Bool in
    true
}
```

The brilliant design of Swift's compiler lets us put these two things together: even though the string's **contains()** is a Foundation method that comes from **NSString**, we can pass it into the

array's **contains(where:)** in place of a closure. So, the entire code becomes this:

```
words.contains(where: input.contains)
```

The last line is the key. **contains(where:)** will call its closure once for every element in the array until it finds one that returns true. Passing it **input.contains** means that Swift will call **input.contains("1989")** and get back false, then it will call **input.contains("Fearless")** and get back true – then stop there. Because **contains()** has the exact same signature that **contains(where:)** expects (take a string and return a Boolean), this works like a charm.

My second example uses the **reduce()** method of arrays: you provide a starting value, then give it a function to apply to every item in an array. Each time the function is called you will be given two parameters: the previous value when the function was called (which will be the starting value initially), and the current value to work with.

To demonstrate this in action, here's an example of calling **reduce()** on an array of integers to calculate the sum of them all:

```
let numbers = [1, 3, 5, 7, 9]

numbers.reduce(0) { (int1, int2) -> Int in
    int1 + int2
}
```

When that code runs, it will add the starting value and 1 to produce 1, then 1 and 3 (running total: 4), then 4 and 5 (9), then 9 and 7 (16), then 16 and 9, giving 25 in total.

This approach is perfectly fine, but Swift has a much simpler and more efficient solution:

```
let numbers = [1, 3, 5, 7, 9]
let result = numbers.reduce(0, +)
```

When you think about it, **+** is a function that accepts two integers and returns their sum, so we can remove our whole closure and replace it with a single operator.

Functions and Closures

We'll look at **reduce()** more later on, when discussing functional programming.

Escaping closures

When you pass a closure into a function, Swift considers it non-escaping by default. This means that the closure must be used immediately inside the function, and cannot be stored away for later – the Swift compiler will refuse to build if you try to use the closure after the function returns.

This works great for many types of functions, such as **sort()**, where you know for sure the closure will be used in the method then never again. The **sort()** method accepts a non-escaping closure as its only parameter because **sort()** won't try to store a copy of that closure for use later – it uses the closure immediately, then is finished with it.

On the other hand, *escaping* closures are ones that *will* be called after the method has returned. These exist in lots of places that need the closure to be called asynchronously.

For example, previously we wrote a small function that delays running a closure by some amount of time:

```
func delay(by seconds: Double, closure: () -> Void) async
throws {
    try await Task.sleep(for: .seconds(seconds))
    closure()
}
```

Calling that means using **try await** everywhere it's used, because the function is marked both **async** and **throws**:

```
try await delay(by: 1) {
    print("Hello, world!")
}
```

If you run that code, you'll see it complains about Swift concurrency because of the weird hacks Swift uses with `main.swift` in command line apps.

Functions and Closures

So, to make this work better I'd like you to rename `main.swift` to `App.swift` to disable the compiler hacks, then change your code to place the **delay()** function inside an **App** struct like this:

```
@main
struct App {
    static func main() async throws {
        func delay(by seconds: Double, closure: () -> Void) async
throws {
            try await Task.sleep(for: .seconds(seconds))
            closure()
        }

        try await delay(by: 1) {
            print("Hello, world!")
        }
    }
}
```

If you haven't seen this before, **@main** lets Swift create and use our **App** struct to run its code, rather than simply running things line by line in `main.swift`. It looks for a static **main()** method to run, which is exactly what we have.

Now that works, you'll find that if we place the body of the function into a **Task**, our code won't build any more:

```
func delay(by seconds: Double, closure: () -> Void) {
    Task {
        try await Task.sleep(for: .seconds(seconds))
        closure()
    }
}
```


And the reason is *escaping*: in order to put the closure inside a **Task**, we must mark it as escaping the current function, like this:

```
func delay(by seconds: Double, closure: @escaping () -> Void) {
    Task {
        try await Task.sleep(for: .seconds(seconds))
        closure()
    }
}
```

So now we'd call it like this:

```
delay(by: 1) {
    print("Hello, world!")
}

try await Task.sleep(for: .seconds(1.5))
```

Tip: We still need that last **sleep()** call because otherwise your little Command Line App project will end before the task finishes; it wouldn't be needed in a real project.

The distinction between escaping and non-escaping closures might sound small, but it matters because closures are *reference types*. Once Swift knows the closure won't be used once the function is finished – that it's *non-escaping* – it won't need to worry about referencing counting, so it can save some work.

As a result, *non-escaping* closures are fractionally more efficient, and are therefore the default in Swift. That is, unless you specify otherwise, all closure parameters are considered to be non-escaping.

Helpfully, this rule is impossible to get wrong, because Swift will tell you to add the **@escaping** keyword when it's required – it's another keyword meant solely as a signal to developers, because the compiler already knows it's needed.

Functions and Closures

We could look at this with some code that won't compile, all of which can go directly inside **main()**:

```
var queuedClosures: [() -> Void] = []

func queueClosure(_ closure: () -> Void) {
    queuedClosures.append(closure)
}

queueClosure { print("Running closure 1") }
queueClosure { print("Running closure 2") }
queueClosure { print("Running closure 3") }
```

That creates an array of closures to run and a function that accepts a closure to queue. The function doesn't do anything other than append whatever closure it was given to the array of queued closures. Finally, it calls **queueClosure()** three times with three simple closures, each one printing a message.

To finish off that code, we just need to create a new method called **executeQueuedClosures()** that loops over the queue and executes each closure:

```
func executeQueuedClosures() {
    for closure in queuedClosures {
        closure()
    }
}

executeQueuedClosures()
```

Let's examine the **queueClosure()** method more closely:

```
func queueClosure(_ closure: () -> Void) {
    queuedClosures.append(closure)
```

```
}
```

It takes a single parameter, which is a closure with no parameters or return value. That closure is then appended to the **queuedClosures** array. What this means is that the closure we pass in can be used later, in this case when the **executeQueuedClosures()** function is called.

Because the closures can be called later, Swift considers them to be *escaping* closures, and so it will refuse to build this code. Remember, non-escaping closures are the default for performance reasons, so we need to explicitly add the **@escaping** keyword to make our intention clear:

```
func queueClosure(_ closure: @escaping () -> Void) {
    queuedClosures.append(closure)
}
```

So: if you write a function that calls a closure immediately then doesn't use it again, it will be non-escaping by default and you can forget about it. But if you intend to store the closure for later use, **@escaping** is required.

Note: Optional closures are inherently always escaping, because they are wrapped in another type.

@autoclosure

The **@autoclosure** attribute is similar to **@escaping** in that you apply it to a closure parameter for a function, but it's used much more rarely. Well, no, that's not strictly true: it's common to *call* functions that use **@autoclosure** but uncommon to *write* functions with it.

When you use this attribute, it automatically creates a closure from an expression you pass in. When you call a function that uses this attribute, the code you write *isn't* a closure, but it *becomes* a closure, which can be a bit confusing – even the official Swift reference guide warns that overusing autoclosures makes your code harder to understand.

To help you understand how it works, here's a trivial example:

```
func printTest(_ result: () -> Void) {  
    print("Before")  
    result()  
    print("After")  
}
```

```
printTest { print("Hello") }
```

That code creates a **printTest()** function, which accepts a closure and calls it. As you can see, the **print("Hello")** is inside a closure that gets called between “Before” and “After”, so the final output is “Before”, “Hello”, “After”.

If we used **@autoclosure** instead, it would allow us to rewrite the **printTest()** call so that it doesn't need braces, like this:

```
func printTest(_ result: @autoclosure () -> Void) {  
    print("Before")  
    result()  
    print("After")  
}
```

```
printTest(print("Hello"))
```

These two pieces of code produce identical results thanks to **@autoclosure**. In the second code example, the **print("Hello")** won't be executed immediately because it gets wrapped inside a closure for execution later.

This behavior might seem rather trivial: all this work just removes a pair of braces, and makes the code harder to understand. However, there is one specific place where you'll use them: **assert()**. This is a Swift function that checks whether a condition is true, and causes your app to halt if not.

That might sound awfully drastic: why would you *want* your app to crash? You wouldn't, obviously, but when you're *testing* your app, adding calls to **assert()** helps ensure your code is behaving as expected. What you really want is for your assertions to be active in debug mode but disabled in release mode – when you build your app for release on the App Store – which is exactly how **assert()** works.

Take a look at the three examples below:

```
assert(1 == 1, "Maths failure!")
assert(1 == 2, "Maths failure!")
assert(myReallySlowFunction() == false, "The slow function
returned false!")
```

The first will return true, so nothing will happen. The second will return false, so the app will halt. The third is an example of **assert()**'s power: because it uses **@autoclosure** to wrap your code in a closure, the Swift compiler simply doesn't run the closure when it's in release mode. This means you get all the safety of assertions while debugging, with none of the performance cost in release mode.

You might be interested to know that autoclosures are also used to handle the **&&** and **||** operators. If you open the Swift source code at <https://github.com/swiftlang/swift> then browse to `stdlib/public/core/Bool.swift`, you'll find the following code in there:

Functions and Closures

```
public static func && (lhs: Bool, rhs: @autoclosure () throws
-> Bool) rethrows -> Bool {
    return lhs ? try rhs() : false
}
```

Yes, that has try/catch, throws and rethrows, operator overloading, the ternary operator, and **@autoclosure** all in one tiny function. Still, I hope you can see through all that cleverness to understand what the code actually does: if **lhs** is true it returns the result of **rhs()**, otherwise false. This is short-circuit evaluation in action: Swift doesn't need to run the **rhs** closure if the **lhs** code already returned false.

Reminder: Use autoclosures with care. They can make your code harder to understand, so don't use them just because you want to avoid typing some braces.

Chapter 6

Errors

Error fundamentals

Swift has a unique take on error handling that is extremely flexible as long as you understand the full extent of what's on offer. I'm going to start off relatively simply and work through the full range of error handling techniques.

Tip: Apple's Swift reference guide says "the performance characteristics of a **throw** statement are comparable to those of a **return** statement," which means they are *fast* – we have no reason to avoid them.

Let's start off with a simple example. All errors you want to throw must be an enum that conforms to the **Error** protocol. So, we define an error enum like this:

```
enum PasswordError: Error {  
    case empty  
    case short  
}
```

This is a regular enum just like any other, so we can add an associated value like this:

```
enum PasswordError: Error {  
    case empty  
    case short  
    case obvious(message: String)  
}
```

To mark a function or method as having the potential to throw an error, add **throws** before its return type, like this:

```
func encrypt(_ str: String, with password: String) throws ->  
String {  
    // complicated encryption goes here  
    let encrypted = password + str + password  
    return String(encrypted.reversed())  
}
```



```
}
```

You then use a mix of **do**, **try**, and **catch** to run the risky code. At the absolute minimum, calling our current code would look like this:

```
do {
    let encrypted = try encrypt("Secret!", with: "T4y10r")
    print(encrypted)
} catch {
    print("Encryption failed")
}
```

That either prints the result of calling **encrypt()**, or an error message. Using **catch** by itself catches all possible errors, which is required in Swift. This is sometimes called Pokémon error handling because “you gotta catch ‘em all.”

Note: this restriction does *not* apply to top-level code in an Xcode playground or a Command Line App using `main.swift`. If you’re using one of those, you should put your **do** block inside a function for testing purposes:

```
func testCatch() {
    do {
        let encrypted = try encrypt("Secret!", with: "T4y10r")
        print(encrypted)
    } catch {
        print("Encryption failed")
    }
}

testCatch()
```

Handling all errors in a single **catch** block might work for you sometimes, but more often you’ll want to catch individual cases. To do this, list each case individually, ensuring the

Errors

generic **catch** is last so that it will only be used when nothing else matches:

```
do {
    let encrypted = try encrypt("secret information!", with:
    "T4ylorSwift")
    print(encrypted)
} catch PasswordError.empty {
    print("You must provide a password.")
} catch PasswordError.short {
    print("Your password is too short.")
} catch PasswordError.obvious {
    print("Your password is obvious")
} catch {
    print("Error")
}
```

To work with an associated value, you need bind it to a constant inside your **catch** block:

```
catch PasswordError.obvious(let message) {
    print("Your password is obvious: \(message)")
}
```

If you want to test that in action, amend the **encrypt()** function to this:

```
func encrypt(_ str: String, with password: String) throws ->
String {
    // complicated encryption goes here
    if password == "12345" {
        throw PasswordError.obvious(message: "I have the same
number on my luggage")
    }

    let encrypted = password + str + password
}
```

```
    return String(encrypted.reversed())
}
```

When working with associated values you can also use pattern matching. To do this, first use **let** to bind the associated value to a local constant, then use a **where** clause to filter.

For example, we could modify our **PasswordError.short** case to return how many characters should have been provided:

```
case short(minChars: Int)
```

With that change, we can catch variations on **short** by filtering by the **minChars** associated value:

```
catch PasswordError.short(let minChars) where minChars < 5 {
    print("We have a lax security policy here: passwords must be
at least \(minChars) characters")
} catch PasswordError.short(let minChars) where minChars < 8 {
    print("We have a moderate security policy here: passwords
must be at least \(minChars) characters")
} catch PasswordError.short(let minChars) {
    print("We have a serious security policy here: passwords must
be at least \(minChars) characters")
}
```

One last thing I want to mention before we move on: Swift already knows exactly which functions can throw errors and which can't, so strictly speaking it doesn't need us to use **try** to call them. Instead, that keyword is required for *our* benefit – it's a visual marker for humans that the function might throw errors, in just the same way that **await** a visual reminder that an async function might trigger a suspension.

Error propagation

When you use `try` to call a function, Swift forces you to handle any errors somewhere, but it *doesn't* need to be immediately. If function A calls function B, and function B calls function C, who should handle errors thrown by C?

If your answer is “B” then your existing error handling knowledge is enough for you.

If your answer is “A” – i.e., that one caller should handle some or all errors in any functions that it calls as well as any errors in functions that *those* functions call, and so on, you need to learn about *error propagation*.

Let's model the `A()`, `B()`, `C()` function calls along with a trimmed version of the `PasswordError` enum we already used:

```
enum PasswordError: Error {
    case empty
    case short
    case obvious
}

func functionA() {
    functionB()
}

func functionB() {
    functionC()
}

func functionC() {
    throw PasswordError.short
}
```

That code won't compile as-is because **functionC()** throws an error but isn't marked with **throws**. If we add that, the code is as follows:

```
func functionC() throws {
    throw PasswordError.short
}
```

But now the code still won't compile, because **functionB()** is calling a throwing function without using **try**. Now we are presented with several options, and I want to explore them individually.

The first option is to catch all errors inside **functionB()**. This is the option you'll use if you want **functionA()** to be oblivious to any errors that happen below it, and looks like this:

```
func functionA() {
    functionB()
}
```

```
func functionB() {
    do {
        try functionC()
    } catch {
        print("Error!")
    }
}
```

```
func functionC() throws {
    throw PasswordError.short
}
```

You could add individual **catch** blocks to **functionB()**, but the principle is still the same.

The second option is for **functionB()** to ignore the errors and let them bubble upwards to its

Errors

own caller, which is called *error propagation*. To do this, we need to move the **do/catch** code out of **functionB()** and into **functionA()**. We then just need to mark **functionB()** with **throws** to have it work, like this:

```
func functionA() {
    do {
        try functionB()
    } catch {
        print("Error!")
    }
}

func functionB() throws {
    try functionC()
}

func functionC() throws {
    throw PasswordError.short
}
```

Before we look at the third option, I want you to take a close look at the current code for **functionB()**: it's a function that uses **try without** having a **do/catch** block around it. This is perfectly fine, as long as the function itself is marked as **throws** so any errors can continue to propagate upwards.

The third option is to delegate parts of error handling to whichever function is most appropriate. For example, you might want **functionB()** to catch empty passwords, but **functionA()** to handle all other errors. Swift normally wants all **try/catch** blocks to be exhaustive, but if you're in a throwing function that requirement is waived – any errors you don't catch will just propagate upwards.

The code below has **functionB()** handle empty passwords, and **functionA()** handle everything else:

```
func functionA() {  
    do {  
        try functionB()  
    } catch {  
        print("Error!")  
    }  
}  
  
func functionB() throws {  
    do {  
        try functionC()  
    } catch PasswordError.empty {  
        print("Empty password!")  
    }  
}  
  
func functionC() throws {  
    throw PasswordError.short  
}
```

Ultimately all error cases must be caught, so at some point you need to have a generic catch all statement or use typed throws.

Throwing functions as parameters

I'm now going to walk through a really useful feature of Swift, and I'm saying this up front because if you find yourself questioning why it's useful I want to make sure you power through – trust me, it's worth it!

First, here's an important quote from the Swift reference guide: “nonthrowing functions are subtypes of throwing functions. As a result, you can use a nonthrowing function in the same places as a throwing one.”

Think about that for a moment. Let it sink in: nonthrowing functions are subtypes of throwing functions, so you can use them anywhere that expects a throwing function. You could even write code as shown below if you wanted to, although you'll get a compiler warning because it's unnecessary:

```
func definitelyWontThrow() {  
    print("Shiny!")  
}  
  
try definitelyWontThrow()
```

Tip: In case you weren't already aware, a nonthrowing function is a regular function like the one above, whereas a throwing function has the **throws** keyword in its signature to mark that it can throw errors.

Where this starts to really matter is when you use a throwing function as a parameter, and I want to give you a working example so you can learn all this in a practical context.

Imagine an app that must fetch user data, either remotely or locally, then act upon it. There's a function to fetch remote user data, which might throw an error if there's a network problem. There's a second function to fetch local user data, which is guaranteed to work and so doesn't throw. Finally, there's a third function that calls one of those two fetch functions, then acts upon the result.

Putting that final function to one side, the initial code might look like this:

```
enum Failure: Error {
    case badNetwork(message: String)
    case broken
}

func fetchRemote() throws -> String {
    // complicated, failable work here
    throw Failure.badNetwork(message: "Firewall blocked port.")
}

func fetchLocal() -> String {
    // this won't throw
    "Taylor"
}
```

The third function is where things get interesting: it needs to call either **fetchRemote()** or **fetchLocal()** and do something with the data that was fetched. Both those functions accept no parameters and return a string, however one is marked **throws** and the other is not.

Think back to what I wrote a few minutes ago: you can use nonthrowing functions anywhere that expects a throwing function. So, we can write a function like this: **fetchUserData(using closure: () throws -> String)**. Let's break that down:

- It's called **fetchUserData()**
- It accepts a closure as a parameter
- That closure must accept no parameters and return a string.

But the closure isn't *required* to throw: we've said that it *can*, but it doesn't *have* to.

With that in mind, a first pass at the **fetchUserData()** function might look like this:

```
func fetchUserData(using closure: () throws -> String) {
```

Errors

```
do {  
    let userData = try closure()  
    print("User data received: \(userData)")  
} catch Failure.badNetwork(let message) {  
    print(message)  
} catch {  
    print("Fetch error")  
}  
}
```



```
fetchUserData(using: fetchLocal)
```

As you can see, we can move from local fetching to remote fetching by changing only the last line:

```
fetchUserData(using: fetchRemote)
```

So, it doesn't matter if the closure we pass in does or does not throw, as long as we state that it *might* and handle it appropriately.

This by itself is a great source of flexibility in our code, but where things get interesting – and by interesting I mean thoroughly awesome – is when you want to use error propagation with a throwing closure parameter. Stick with me – we're almost through!

A simple solution might declare **fetchUserData()** as throwing then catch errors at the call site, like this:

```
func fetchUserData(using closure: () throws -> String) throws {  
    let userData = try closure()  
    print("User data received: \(userData)")  
}  
  
do {
```

```

    try fetchUserData(using: fetchLocal)
} catch Failure.badNetwork(let message) {
    print(message)
} catch {
    print("Fetch error")
}

```

That works fine for this exact situation, but Swift has a much smarter solution. This **fetchUserData()** function could be called elsewhere in our app, perhaps more than once – it's going to get awfully messy having all that **try/catch** code in there, particularly the times we use **fetchLocal()** and know for a fact it won't throw.

Swift's solution is the **rethrows** keyword, which we can use to replace the regular **throws** in the **fetchUserData** function like this:

```

func fetchUserData(using closure: () throws -> String) rethrows
{
    let userData = try closure()
    print("User data received: \(userData)")
}

```

So, the closure *throws*, but the **fetchUserData()** function *rethrows*. The difference might seem subtle, but this code will now create a warning in Xcode:

```

do {
    try fetchUserData(using: fetchLocal)
} catch Failure.badNetwork(let message) {
    print(message)
} catch {
    print("Fetch error")
}

```

If you replace **try fetchUserData(using: fetchLocal)** with **try fetchUserData(using:**

Errors

fetchRemote) then the warning goes away. What's happening is that the Swift compiler is now checking each call to **fetchUserData()** individually, and now only requires **try/catch** to be used when the closure you pass in throws.

So, when you use **fetchUserData(using: fetchLocal)** the compiler can see **try/catch** isn't necessary, but when you use **fetchUserData(using: fetchRemote)** Swift will ensure you catch errors correctly.

So, with **rethrows** you get the best of both worlds: when you pass in a closure that will throw you get all the safety you'd expect from Swift, but when you pass in a closure that won't throw you don't need to add pointless **try/catch** code.

Armed with this knowledge, take another look at the code for the short-circuit logical **&&** operator, taking from the Swift source code. As a reminder, you can find this at <https://github.com/swiftlang/swift>, in the `stdlib/public/core/Bool.swift`:

```
public static func && (lhs: Bool, rhs: @autoclosure () throws
-> Bool) rethrows -> Bool {
    return lhs ? try rhs() : false
}
```

You should now be able to break down exactly what that does: the right-hand side of the **&&** is an autoclosure so that it gets executed only if the left-hand side evaluates to true. **rhs** is marked as throwing even though it might not, and the whole function is marked as **rethrows** so that the caller is required to use **try/catch** only when necessary.

I hope you'll agree that Swift's error handling approach is fractally beautiful: the more you dig into it, the more you come to appreciate its subtle genius.

Typed throws

When we mark a function with **throws**, we're saying it might throw any kind of error at all, which, while *flexible*, can be a bit annoying because you need to have that final **catch** block to handle all other possible errors even when you know it won't be reached.

From Swift 6 onwards, we have an alternative called *typed throws*: rather than just writing **throws** by itself, we can write **throws(MyError)**, and three things change:

1. Swift will ensure we only ever throw errors of that type in the function.
2. We no longer need a generic catch all section if we already caught the other cases for our error type.
3. When we catch the error, it will automatically have the type **MyError** rather than just **Error**.

That sounds great, right? Well, before I show you some example code, it's important I say that the Swift Evolution proposal for this feature – the official language change document from the Swift team – says explicitly that, "even with the addition of typed throws to Swift, untyped throws are better for most scenarios."

This is primarily because they know a lot of folks have been bitten in the past by saying "I'll definitely only throw errors of type X," only to change their minds later.

Still here? Okay, let's look at some sample code. Here's a **UserError** type, that allows us to flag up bad usernames and bad passwords:

```
enum UserError: Error {  
    case badUsername  
    case badPassword  
}
```

And now we can write a little test function to validate an example username and password, but we'll make this only throw a **UserError**:

Errors

```
func authenticate(username: String, password: String) throws
(UserError) {
    if username == "Taylor" {
        if password == "swift" {
            print("Success")
        } else {
            throw .badPassword
        }
    } else {
        throw .badUsername
    }
}
```

Notice how we can write just **.badPassword** and **.badUsername** here, because Swift knows the errors thrown must be **UserError**.

Now we can go ahead and call our function, but here things are a little... well, *imperfect*, at least at the time I'm writing this.

You see, what I want to do is write code like his:

```
func runTest() {
    do {
        try authenticate(username: "Taylor", password: "fr0stles")
    } catch .badUsername {
        print("Please check your username.")
    } catch .badPassword {
        print("Please check your password.")
    }
}
```

However, right now at least that isn't allowed – Swift complains it wants a generic **catch** block at the end, but if you add one it says it won't be reached.

I'm hoping it's just a temporary bug that gets resolved by the time you read this, but in the meantime we need to write this instead:

```
func runTest() {
    do {
        try authenticate(username: "Taylor", password: "fr0st1es")
    } catch {
        switch error {
        case .badUsername:
            print("Please check your username.")
        case .badPassword:
            print("Please check your password.")
        }
    }
}
```

It's still better than having a generic **catch** block that definitely won't be reached, and you can see how the **switch** block is considered exhaustive because we catch both its cases.

That's all there is to using typed throws, but I want to add three other pieces of information just to round out your knowledge:

1. Functions that don't throw errors are equivalent to **throws(Never)**.
2. Functions that use untyped **throws** are equivalent to **throws(any Error)**.
3. Using **rethrows** can be expressed using a generic type parameter, e.g. **E**, along with a typed throw such as **throws(E)**.

I know it's tempting to rush out and use typed errors everywhere, but please think carefully – particularly if you're working on libraries that others will use.

try vs try? vs try!

Swift's error handling comes in three forms, and all three have their uses. All are used when calling a function that is marked with **throws**, but have subtly different meaning:

1. When using **try** you must have a **catch** block to handle any errors that occur.
2. When using **try?** the function you call will automatically return **nil** if any errors are thrown. You don't need to catch them, but you do need to be aware that your return value becomes optional.
3. When using **try!** the function will crash your app if any errors are thrown.

I've numbered them because that's the order you should use them in: regular **try** is by far the most common and behaves as we've seen so far; **try?** is a safe and useful fallback that, when used wisely, will do a lot to improve your code's readability; **try!** means "throwing an error is so unlikely – or so undesirable – that I'm willing to accept a crash," and is very uncommon.

Now, you might wonder why **try!** even exists: if you're certain it won't throw an error, why is the function marked with **throws** in the first place?

Well, consider code that reads a file from your app bundle. Loading a file like this can throw an error if the file doesn't exist or isn't readable, but if that situation arises your app is clearly in a very broken state, because a file is missing from the app's bundle! So, forcing a crash here might well be a desirable outcome rather than allowing the user to continue with a corrupted app.

That being said, I think I'd still rather use a regular **try** backed up by **fatalError()** in this situation, because at least it issues a helpful message.

Of the three, only the regular **try** requires a **do/catch** block, so if you're looking for concise code you are likely to want to use **try?** on occasion.

Using **try?** means safely unwrapping its optional return value, like this:

```
if let savedText = try? String(contentsOfFile: "saved.txt"),
```



```
encoding: .utf8) {
    print(savedText)
} else {
    print("Nothing found")
}
```

You can also use the nil coalescing operator `??` to use a default value if `nil` is returned, thus removing optionality altogether:

```
let savedText = (try? String(contentsOfFile: "saved.txt",
encoding: .utf8)) ?? "Hello, world!"
```

On rare occasions, I use `try?` bit like the internet UDP protocol: try this thing, but I don't care if it fails. Two of the most important protocols behind the internet are called TCP and UDP. TCP guarantees that all packets will arrive, and will keep trying to re-send until a certain time has expired; it's used for downloads, for example, because if you're missing part of a zip file then you have nothing.

UDP sends all data packets once and hopes for the best: if they arrive, great; if not, just wait until the next one arrives. UDP is useful for things like video streaming, where you don't care if you lost a split second of a live stream – it's more important that new video keeps coming in.

So, `try?` can be used a bit like UDP: if you don't care what the return value is and you just want to make an attempt to do something, `try?` is for you:

```
_ = try? string.write(toFile: somePathHere, atomically: true,
encoding: .utf8)
```

Assertions and preconditions

Assertions allow you to state that some condition must be true. The condition is down to you, and can be as complicated as you like, but if it evaluates to **false** your program will halt immediately. Assertions are cleverly designed in Swift, to the point where reading its source code is a valuable exercise.

When you write **assert** in Xcode, the code completion will give you two options:

```
assert(condition: Bool)
assert(condition: Bool, message: String)
```

In the first option you need to provide a condition to be tested; in the second, you also provide a message to display if the condition evaluates to false. In the code below, the first assertion will evaluate to true and so nothing will happen; in the second, it will fail because the condition is false, so a message will be printed:

```
assert(1 == 1)
assert(1 == 2, "Danger, Will Robinson: mathematics failure!")
```

Obviously asserting basic arithmetic isn't much use, so you'll usually write code like this:

```
let success = runImportantOperation()
assert(success == true, "Important operation failed!")
```

Assertions are extremely useful when writing complicated apps, because you can scatter them throughout your code to ensure that everything is as it should be. You can test as much as you want, as often as you want, and it means that any unexpected state in your app – when you say “how did **someVar** get set to *that*?” – gets caught early in development.

To explore how assertions work, take a look at the actual type signature from the Swift source code:

```
public func assert(_ condition: @autoclosure () -> Bool, _
```

```
message: @autoclosure () -> String = String(), file:
StaticString = #file, line: UInt = #line)
```

The last two parameters have defaults provided by the Swift compiler, and it's unlikely you'll want to change them: **#file** is replaced by the name of the current file, and **#line** is replaced by the line number of the code that triggered the assertion. These are passed in as parameters with default values (as opposed to being specified inside the function) so that Swift uses the filename and line number of the call site, not some line inside **assert()**.

The first two parameters are much more interesting: both the **condition** and **message** parameters use **@autoclosure**, which means they aren't immediately executed. This is important, because internally Swift will execute assertions only if it's in debug mode. This means you can assert hundreds or even thousands of times in your app, but all that work will only get done if you're debugging. When the Swift compiler runs in release mode, the work is skipped.

You can see this for yourself in Swift's source code – go to <https://github.com/swiftlang/swift> then browse to `stdlib/public/core/Assert.swift`. Here's the body of **assert()**, straight from the Swift source code:

```
if _isDebugAssertConfiguration() {
    if !_fastPath(condition()) {
        _assertionFailure("Assertion failed", message(), file:
file, line: line, flags: _fatalErrorFlags())
    }
}
```

The functions prefixed with an underscore are internal, but you might to be able to guess what they do:

- **_isDebugAssertConfiguration()** returns false if we're not in debug mode. This is what causes the assertion to disappear when built for release.
- **!_fastPath(condition())** runs the condition closure that was created by **@autoclosure**.

Errors

It tells the compiler to presume the condition will evaluate successfully (which ought to be the case most of the time), which allows the code to run a little faster. This only affects debugging, but it helps your assertions run a little faster.

- If we're in debugging mode and calling **condition()** returned false, then **_assertionFailure()** is called to terminate your program. At this point, the **message()** closure is called to print a useful error.

The use of **@autoclosure** is perfect for the condition, because this way it gets run only if we're in debug mode. But you might wonder why the **message** parameter is also an autoclosure – isn't it just a string?

This is where **assert()** gets *really* clever: because **message** is a closure, you can run any other code you like before finally returning a string, and none of it will get called unless your assertion fails.

The most common use for this is when using a logging system: your **message** closure can write an error to your log, before returning that message back to **assert()**. Below is a simplified example that writes your message to a file before returning it – it's basically just a pass-through that adds some extra functionality:

```
func saveError(message: String, file: String = #file, line: Int
= #line) -> String {
    _ = try? message.write(toFile: pathToDebugFile, atomically:
true, encoding: .utf8)
    return message
}

assert(1 == 2, saveError(message: "Fatal error!"))
```

Preconditions

Assertions are checked only when your app is running in debug mode, which is useful when you're developing but automatically deactivated in release mode. If you wish to make

assertions in release mode – bearing in mind that a failure causes your app to terminate immediately – you should use **precondition()** instead.

This takes identical parameters to **assert()** but is compiled differently: under normal circumstances, a failed precondition will cause your app to terminate, even in release mode. However, if you change the “Disable Safety Checks” build option to Yes, failed preconditions are ignored.

Just like using **try!**, there is a good reason why you might want to crash your app in release mode: if something has gone fatally wrong in such a way that it suggests your app is in an unstable, unknown, or perhaps even dangerous state, it’s better to bail out than continue and risk serious data loss.

In the chapter on operator overloading, I presented a modification to the ***** operator that allows us to multiply two arrays:

```
func *<T: Numeric>(lhs: [T], rhs: [T]) -> [T] {
    let limit = min(lhs.count, rhs.count)
    var result = [T]()

    for i in 0..

```

As you can see, the function kicks off with a **min()** call to figure the maximum number of numbers we can safely multiply.

In many cases this is safe programming, but it’s also possible that if our program ended up with two arrays of different sizes then something serious has gone wrong and we should halt execution.

Errors

In this situation, using **precondition()** might be preferable:

```
func *<T: Numeric>(lhs: [T], rhs: [T]) -> [T] {
    precondition(lhs.count == rhs.count, "Arrays were not the
same size")
    var result = [T]()

    for i in 0..
```

Remember, disabling safety checks will effectively disable your preconditions, but it also disables other bounds checking too – that's why it's so fast, and that's why it's so dangerous!

Never and fatalError()

One particular feature of Swift that can help you write simpler functions is the **Never** return type, which means “this function will never return.” That’s different from **Void**, which means “will return nothing” – a **Never** function will literally never come back.

Now, that might sound pointless, but there’s a particular use case which more developers need to be aware of: the **fatalError()** function. This immediately terminates your app, optionally writing out an error message, a bit like failing an **assert()** call except **fatalError()** remains in your code no matter what compiler optimizations you enable.

What makes **fatalError()** special is its use of the **Never** return type – this function is guaranteed to terminate your app, so it will never return. This means the Swift compiler will *not* force you to return anything from the function, even if normally that function has a return type.

This is important, because it lets you simplify your code. For example, we could write a function that finds and loads a named file from an app bundle, but if the file can’t be found or loaded we trigger a fatal error:

```
func fetchBundleString(from filename: String) -> String {
    guard let url = Bundle.main.url(forResource: filename,
withExtension: nil) else {
        fatalError("Unable to locate \(filename)")
    }

    guard let contents = try? String(contentsOf: url,
encoding: .utf8) else {
        fatalError("Unable to load \(filename)")
    }

    return contents
}
```

Errors

This makes perfect sense, even in production code: if your app relies on a file that is included in your app bundle, there's no point trying to crawl on without it – it *must* be there, and if it isn't your app bundle has gotten corrupted somehow.

Using **`fatalError()`** means you aren't returning something just for the sake of making your code compile. Instead, you're making your intent crystal clear: this needs to work, and failure is not an option.

Chapter 7

Functional Programming

What is functional programming?

Depending on where you're coming from, functional programming might be a perfectly normal way of writing code, it might be an academic tool that somehow snuck out of university campuses, or it might be something you use to scare kids into staying in bed.

The truth is that functional programming usage covers a spectrum, from “I’m basically reading algebra” to “I’m mostly using object-oriented code, with a few functional techniques where I want them.”

You'll be pleased to know that functional programming isn't too hard to pick up and use in a practical way, although it's easy to feel overwhelmed with talk of monads and functors.

I aim to teach you the benefits of functional programming while also cutting out the parts you're less likely to benefit from. I make no apologies: this whole book is about teaching you hands-on techniques to improve your coding immediately, not explaining what referential transparency is.

I want to teach you some functional programming using what Andy Matuschak describes as “lightweight encounters.” This means we'll focus on finding simple, small benefits that you can understand and use straight away. Andy is an experienced functional developer, but he's also pragmatic about learning approaches – I hope he won't frown too much on me simplifying (and even jettisoning) so much theory!

Before I jump into functional code, I want to give you a broad idea of why we might want to change the current way of working. Chances are that you have used object orientation extensively in the past: you've created classes, made subclasses from those classes, added methods and properties, and more.

If you've already learned about reference and value types, you'll also know that instances of classes are reference types, which means a property could be changed by any of its multiple owners.

We consider object orientation easy because it's what we know. If I tell you that **Poodle**

inherits from **Dog**, has a **barkVolume** property and a **biteStrength** property, as well as an **isBarkWorseThanBite()** method, you immediately understand what all that means.

But it's not *simple*. It's complicated: that one "easy" class blends state, functionality, inheritance, and more, so you need to keep a lot in your head to follow along.

Functional programming – or at least the slightly simplified definition I'll be working with here – can dramatically simplify your code.

At first, this will cause problems because you're effectively fighting object-oriented muscle memory: your instinct to solve every problem by creating a new class needs to be put on hold, at least for the time being.

Instead, we're going to apply five principles that help deliver the benefits of functional programming without the algebra.

First, functions are first-class data types. That means they can be created, copied, and passed around like integers and strings.

Second, because functions are first-class data types, they can be used as parameters to other functions.

Third, in order to allow our functions to be re-used in various ways, they should always return the same output when given specific input, and not cause any side effects.

Fourth, because functions always return the same output for some given input, we should prefer to use immutable data types rather than using functions to change mutable variables.

Fifth and finally, because our functions don't cause side effects and variables are all immutable, we can reduce how much state we track in our program – and often eliminate it altogether.

I know that's a lot to take in all at once, so let me try to break down each of those into more detail.

Functional Programming

You should already know that functions are first-class data types in Swift – after all, you can copy closures and pass them around. So, that’s one down.

Next up, passing functions as parameters to other functions is also something you maybe have done already, such as calling **sort()** with a closure. You might come across the name “higher-order function”, which is the name given to a function that accepts another function as a parameter.

Where things get a bit more complicated – but a lot more interesting – is when we write functions that always produce the same output for a given input. This means if you write a function **length(of strings:)** that accepts an array of strings and returns back an array of integers based on the length of each string, that function will return **[6, 4, 5]** when given **["Taylor", "Paul", "Adele"]**. It doesn’t matter what else has happened in your program, or how often the function has been called: the same input must return the same output.

A corollary is that functions ought not to create side effects that might affect other functions. For example, if we had another function **fetchSystemTime()** that returned the time, it should not be the case that calling **fetchSystemTime()** could affect the result of **lengthOf(strings:)**.

A function that always returns the same result for a given input without causing side effects is often called a *pure* function. I guess that makes all other functions *impure*, and you don’t want to write *dirty* functions, do you?

One source of confusion about pure functions revolves around the meaning of “side effect.” If a function does something like writing to disk, is that a side effect or in fact just the main point of the function?

There’s a lot of argument about this, and I’m not going to wade into it. Instead, I’ll say that functional programmers should *aspire* to create pure functions, but when it comes to the crunch favor predictable output for known input over avoiding side effects.

That is, if you want to write a function that writes some data to disk (a side effect? the actual effect? call it what you want!) then go for it, but at least make sure it writes exactly the same thing when given the same data.

You should already appreciate the importance of immutable data types and values rather than references, so I shan't go over them again other than to say that classes are about as welcome in functional code as a hedgehog in a hemophilia convention.

Finally, the lack of state can be tricky, because it's so deeply baked into our projects. "State" is a series of values stored by your program, which isn't always a bad thing – it includes caching things to increase performance, important things like user settings, and, well, everywhere you write `@State` in SwiftUI. The problem comes when this state gets used inside a function, because it means the function is no longer predictable.

Using the `length(of strings:)` example from earlier, consider what would happen if we had a Boolean setting called `returnLengthsAsBinary` – the function that would always return `[6, 4, 5]` when given `["Taylor", "Paul", "Adele"]` could now also return `[110, 10, 101]` depending on the value of something completely external.

Being pragmatic, don't avoid state at all costs, but never let it pollute your functional code, and work to minimize its use more generally.

When all five of these principles combine, you get a number of immediate, valuable benefits. When you write functions that produce predictable output, you can write unit tests for them trivially. When you use immutable value types rather than reference types, you remove unexpected dependencies in your app and make your code easier to reason about. When you build small, composable functions that can be combined with higher-order functions and re-used in any number of ways, you can build hugely powerful apps by putting together many small, simple pieces.

Note: I refer to these five principles a lot. Rather than repeat myself again and again, I'll often just say the Five Functional Principles, and hopefully you'll remember 1) first-class data types, 2) higher-order functions, 3) pure functions, 4) immutability, and 5) reduce state.

Okay, enough theory. I hope I've managed to convince you that functional programming has something to offer everyone, and even if you take only a few concepts from the following sections that's a big improvement.

map()

Let's start with the easiest way into functional programming: the **map()** method. This takes a value out of a container, applies a function to it, then puts the result of that function back into a new container that gets returned to you.

Swift's arrays, dictionaries, and sets have **map()** built in, and it's commonly used to loop through every item in an array while applying a function to each value.

Let's start in a non-functional way, with a **length(of strings:)** function that accepts an array of strings and returns an array of integers based on the size of the input strings.

You might write it something like this:

```
func length(of strings: [String]) -> [Int] {
    var result = [Int]()

    for string in strings {
        result.append(string.count)
    }

    return result
}
```

That function takes an array of strings and returns an array of integers based on those strings. This is a perfect use for **map()**, and in fact we can replace all that code with this:

```
func length(of strings: [String]) -> [Int] {
    strings.map { $0.count }
}
```

It's clear that the functional approach is shorter, but its benefit is not just about writing less code.

Functional Programming

Instead, the functional version conveys significantly more meaning to the compiler: it's now clear we want to apply some code to every item in an array, and it's down to Swift to make that happen efficiently.

Using **map()** also makes clear our intention to other coders: it will go over every item in an array and apply a function to it. With the traditional **for** loop you might have a **break** in there half-way through – something that is not possible with **map()** – and the only way to find that out is to read all the code.

If you're following the Five Functional Principles I already laid down (specifically using pure functions and avoiding state), a human reading your code can know immediately that a closure used with **map()** won't try to access global state along the way.

This simplification is important, and it's a change of focus. Javier Soto – well-known functional proponent and Swift hacker extraordinaire at OpenAI – sums up the usefulness of **map()** like this: it “allows us to express what we want to achieve, rather than how this is implemented.” That is, we just say “here's what we want to do with the items in this array” is easier to read, write, and maintain than hand-writing loops and creating arrays by hand.

One other thing: notice how the type signature hasn't changed. That means we write **func length(of strings: [String]) -> [Int]** whether or not we're using a functional approach internally.

This means you can change the internals of your functions to adopt functional approaches without affecting how they interact with the rest of your app – you can upgrade your code bit by bit rather than all at once.

Examples

To help you get more comfortable using **map()**, here are some examples.

This snippet converts strings to uppercase:

```
let fruits = ["Apple", "Cherry", "Orange", "Pineapple"]
```



```
let uppercaseFruits = fruits.map { $0.uppercased() }
```

This snippet converts an array of integer scores into formatted strings:

```
let scores = [100, 80, 85]
let formatted = scores.map { "Your score was \($0)" }
```

These two snippets use the ternary conditional operator to create an array of strings, matching each item against specific criteria. The first checks that each score is above 85, and the second checks that each position is within a the range 45 to 55 inclusive:

```
let scores = [100, 80, 85]
let passOrFail = scores.map { $0 >= 85 ? "Pass" : "Fail" }

let position = [50, 60, 40]
let averageResults = position.map { 45...55 ~= $0 ? "Within
average" : "Outside average" }
```

Third, this example uses the `sqrt()` function to calculate the square roots of numbers:

```
import Foundation
let numbers: [Double] = [4, 9, 25, 36, 49]
let result = numbers.map(sqrt)
```

As you can see, `map()` has that name because it specifies the mapping from one array to another. That is, if you pass it an array `[a, b, c]` and function `f()`, Swift will give you the equivalent of `[f(a), f(b), f(c)]`.

And fourth, I want to return to a function I mentioned briefly in the chapter on operator overloading. This multiplies two arrays of `Numeric` values:

```
func *<T: Numeric>(lhs: [T], rhs: [T]) -> [T] {
    zip(lhs, rhs).map(*)
}
```

Swift's **zip()** function has the job of combining two arrays into one by making pairs, ignoring any values if one sequence grows beyond the other. For example, if we zipped the arrays **[1, 2, 3]** and **[4, 5]**, we'd get back an array containing the pairs **(1, 4)** and **(2, 5)**.

We then map those pairs, which means Swift will give us two numbers from the zipped array, such as 1 and 4. And then for the fun part: we pass those two numbers directly to *****, which as you saw back in the operator overloading chapter is just a function that accepts two numbers and returns their product.

So, that one-line function is identical in behavior to the original one I showed you previously:

```
func *<T: Numeric>(lhs: [T], rhs: [T]) -> [T] {
    let limit = min(lhs.count, rhs.count)
    var result = [T]()

    for i in 0..
```

Optional map

To repeat what I said earlier, **map()** “takes a value out of a container, applies a function to it, then puts the result of that function back into a new container that gets returned to you.” We’ve been using an array so far, but if you think about it a value inside a container is exactly what optionals are. They are defined like this:

```
enum Optional<Wrapped> {
    case none
    case some(Wrapped)
```

```
}
```

Because they are just simple containers for a single value, we can use **map()** on optionals too. The principle is identical: take value out of container, apply function, then place value back in the container again.

Let's start with a simple example so you can see how optional map works:

```
let i: Int? = 10
let j = i.map { $0 * 2 }
print(j)
```

That will print **Optional(20)**: the value 10 was taken out of its optional container, multiplied by 2, then placed back into an optional. If **i** had been nil, **map()** would just return nil. This behavior makes **map()** useful when manipulating optional values, particularly when combined with the nil coalescing operator.

To give you an example, consider the following function:

```
func fetchUsername(id: Int) -> String? {
    if id == 1989 {
        "Taylor Swift"
    } else{
        nil
    }
}
```

That returns an optional string, so we'll either get back "Taylor Swift" or nil. If we wanted to print out a welcome message – but only if we got back a username – then optional map is perfect:

```
var username: String? = fetchUsername(id: 1989)
let formattedUsername = username.map { "Welcome, \($0)!" } ??
"Unknown user"
```

```
print(formattedUsername)
```

To write that in a non-functional approach, the alternative is much longer:

```
let username = fetchUsername(id: 1989)
let formattedUsername: String

if let username {
    formattedUsername = "Welcome, \(username)!"
} else {
    formattedUsername = "Unknown user"
}

print(formattedUsername)
```

forEach

map() has a close relation called **forEach()**, which also loops over an array and executes a function on each item.

The main difference lies in the return value: **map()** returns a new array of items, whereas **forEach()** returns nothing at all – it's just a functional way to loop over each item.

Honestly, in over 10 years of writing Swift code, I have never once needed this method.

Whenever I see someone who *has* used it, they've often abused it by bringing in external state or even *changing* external state, which makes it a thoroughly counter-productive use of functional programming.

Other than the return value, **forEach()** is used the same as **map()**:

```
["Taylor", "Paul", "Adele"].forEach { print($0) }
```

Behind the scenes, **forEach()** literally boils down to a regular **for-in** loop – there's nothing

map()

special about it, so maybe just use a **for** loop.

compactMap()

As you saw, **map()** takes an item out of a container such as an array, applies a function to it, then puts it back in the container. The immediate use case for this is arrays, but optionals work just as well.

Swift has a specialized form of **map()** called **compactMap()**, which applies a compacting step once the **map()** operation has completed: all optionals get unwrapped, and any that contained `nil` are discarded.

First, let's look again at our friend **map()**:

```
let albums = [ "Fearless", nil, "Speak Now", nil, "Red" ]
let result = albums.map { $0 }
print(result)
```

Mapping using **\$0** just means “return the existing value,” so that code will print the following:

```
[ Optional( "Fearless" ), nil, Optional( "Speak Now" ), nil,
Optional( "Red" ) ]
```

That's a lot of optionals, with some **nil**s scattered through. Switching to **compactMap()** rather than **map()** can help:

```
let albums: [String?] = [ "Fearless", nil, "Speak Now", nil,
"Red" ]
let result = albums.compactMap { $0 }
print(result)
```

Just moving from **map { \$0 }** to **compactMap { \$0 }** changes the result dramatically:

```
[ "Fearless", "Speak Now", "Red" ]
```

The optionals are gone *and* the **nil**s are removed.

The reason for this magic lies in **compactMap()**'s return value: whereas **map()** will retain the optionality of the items it processes, **compactMap()** will strip it out and discard any nil items. So, in the code below, **mapResult** is of type **[String?]** and **compactMapResult** is of type **[String]**:

```
let mapResult = albums.map { $0 }
let compactMapResult = albums.compactMap { $0 }
```

Failable operations

If the usefulness of **compactMap()** isn't quite clear yet, please stick with me! Let's look at this example again:

```
let albums: [String?] = ["Fearless", nil, "Speak Now", nil,
"Red"]
let result = albums.compactMap { $0 }
print(result)
```

The **albums** array is of type **[String?]**, so it holds an array of optional strings. Using **compactMap()** here strips out the optionality, but really that's just the effect of unwrapping the optionals – it doesn't do any sort of mapping transformation.

Now, here's where **compactMap()** becomes awesome: because the compacting happens after the mapping, you can effectively say “do something interesting with these items (map), then remove any items that returned nil (compact).”

To give you a practical example, imagine a school grade calculator: students are asked to enter their scores from various exams, and it will output their estimated grade. This needs to convert user-entered numbers into integers, which is problematic because they can make mistakes or enter nonsense. As a result, creating an integer from a string returns an optional integer – it's nil if the input was “Fish”, but 100 if the input was “100”.

This is a perfect problem for **compactMap()**: we can take an array like **["100", "90", "Fish",**

"85"], map over each value to convert it to an optional integer, then compact the resulting array to remove the optionality and any invalid values:

```
let scores = ["100", "90", "Fish", "85"]
let compactMapScores = scores.compactMap { Int($0) }
print(compactMapScores)
```

That will output **[100, 90, 85]** – perfect!

But we can do better. You see, **compactMap()** wants a function that takes one item from our sequence, converts it to an optional value somehow, and returns the result to be compacted.

In our current code, we're passing in a closure that calls the **Int** initializer with a string, which returns an **Int?**. Initializers are just functions, so rather than passing a closure that calls the function, we can just send in the initializer directly:

```
let compactMapScores = scores.compactMap(Int.init)
```

This works great in so many places, such as converting an array of strings into an array of **URL** objects. Those also have a failable initializer, which is perfect for **compactMap()**:

```
let sites = ["https://apple.com", "https://swift.org"]
let urls = sites.compactMap(URL.init)
```

So, if you have an array of input items, and a transformation function that will return either a transformed item or nil if the transformation failed, this technique is ideal.

Remember: any throwing function can be used with **try?**, which translates it to a function that returns nil on failure – exactly what this approach needs.

Here's an example so you can see what I mean:

```
let files = (1...10).compactMap { try? String(contentsOfFile:
"someFile-\($0).txt", encoding: .utf8) }
```


`compactMap()`

```
print(files)
```

That will load into an array the contents of `someFile-1.txt`, `someFile-2.txt`, and so on. Any files that don't exist will throw an error, which **try?** converts into `nil`, and **compactMap()** will ignore – all with just one line of code!

So, **try?** effectively turns any throwing call into a candidate for **compactMap()** by introducing `nil`. A similar trick is possible using the **as?** typecast, which also returns `nil` when it fails.

As you can see, **compactMap()** is extremely useful for dealing with failable operations, and serves as a useful counterpart to a regular **map()**.

flatMap()

The **flatMap()** method is another variation of **map()**, and it also has variants for both sequences and optionals. Both variants have the same goal, and the clue is in the name: it's a flattening map operation.

As you know, **map()** takes an item out of a container such as an array, applies a function to it, then puts it back in the container. The most common use case for this is sequences, but optionals work just as well.

When you have nested sequences – e.g., an array of arrays – you get access to a version of the **joined()** method that converts that array of arrays into a single array, like this:

```
let numbers = [[1, 2], [3, 4], [5, 6]]
let joined = Array(numbers.joined())
// [1, 2, 3, 4, 5, 6]
```

So, **joined()** flattens the containers by one level: a two-dimensional array becomes a single-dimensional array by concatenating items.

The **flatMap()** method on sequences is effectively the combination of using **map()** and **joined()** in a single call, in that order. It maps items in array A into array B using a function you provide, then joins the results using concatenation.

For example, let's say we had an array of strings like this:

```
let words = ["the rain in", "Spain falls mainly", "on the Spaniards"]
```

We can use **flatMap()** to create one array of all the words, like this:

```
let result = words.flatMap { $0.split(separator: " ") }
print(result)
```

That's identical to using **map()** then **joined()** and converting the result to an array:

```
let result2 = Array(words.map { $0.split(separator: "
") }.joined())
```

Where **flatMap()** gets more interesting is its use with optionals. Using **flatMap()** with optionals is similar to using **map()**, with one important difference: if your transformation closure returns an optional, **flatMap()** will combine that optional with the existing optional, whereas **map()** will keep them both. So, here the flattening/joining operation is to combine two identical containers together into one.

Here's a practical example so you can see the difference:

```
let stringNumber: String? = "5"
let intNumber = stringNumber.map { Int($0) }
```

Or we can also pass in the **Int** initializer directly, like this:

```
let intNumber = stringNumber.map(Int.init)
```

When that code runs, **intNumber** will be an **Int??** – an optional optional integer. This is because we already have optionality from **stringNumber**, and the **Int** initializer from a string also returns an optional, so **map()** just puts them together.

In comparison, **flatMap()** acts differently:

```
let flatMapNumber = stringNumber.flatMap(Int.init)
```

That will return a regular **Int?**, meaning that either the whole thing exists or nothing exists – it's easier to work with.

If you're not sure whether to use **map()** or **flatMap()** with your optional value, the simple rule is this: if your transformation closure returns an optional you should use **flatMap()**, to avoid optional optionals. Or if you want all your optionals removed entirely, use **compactMap()**.

Functional Programming

Either way if you ever find yourself working with optional optionals, you've definitely taken a wrong turn somewhere!

filter()

The **filter()** method loops over every item in a collection, and passes it into a function that you write. If your function returns true for that item it will be included in a new array, which is the return value for **filter()**.

For example, the code below creates an array of the first 10 numbers in the Fibonacci sequence, then filters that array to include only items that are even:

```
let fibonacciNumbers = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
let evenFibonacci = fibonacciNumbers.filter { $0.isMultiple(of: 2) }
```

So, the code will run, and pass the number 1 into the closure, making **1.isMultiple(of: 2)**, which is false. As a result, 1 will not be included in the result array. When it reaches 2, that *will* be included because **2.isMultiple(of: 2)** is true.

As with **map()**, the way items are filtered is out of our control – Swift is free to apply any optimizations it wants to make filtering go faster.

However, there is *one* optimization we can do ourselves: if the reason you're filtering is because you want to count how many items match a test, you should use the **count(where:)** method instead. This works identically to filter, except it returns a count of the result rather than array – it's much more efficient.

For example, this counts our even numbers rather than creating a new array of them:

```
let evenFibonacciCount = fibonacciNumbers.count
{ $0.isMultiple(of: 2) }
```

Examples

Here are three examples that demonstrate using **filter()** in different ways, starting with two I've used elsewhere:

```
let names = ["Michael Jackson", "Taylor Swift", "Michael Caine", "Adele Adkins", "Michael Jordan"]
let result = names.filter { $0.hasPrefix("Michael") }
```

That will create an array of strings using only names that begin with Michael.

This next example returns a subset of the **words** array based on which words exist in the string **input**:

```
import Foundation
let words = ["1989", "Fearless", "Red"]
let input = "My favorite album is Fearless"
let result = words.filter { input.contains($0) }
```

We could also pass the **contains()** method direct into **filter()**, like this:

```
let result = words.filter(input.contains)
```

I have shown elsewhere **compactMap()**'s ability to strip nil values from an array, but it also removes optionality from the remaining elements. If you want to strip nil but retain optionality, use **filter()** like this:

```
let words: [String?] = ["1989", nil, "Fearless", nil, "Red"]
let result = words.filter { $0 != nil }
print(result)
```

reduce()

The **reduce()** method condenses an array into a single value by applying a function to every item. Each time the function is called, you get passed the previous value from your function as well as the next item in your array.

So, you'll often see code like this:

```
let scores = [100, 90, 95]
let sum = scores.reduce(0, +)
```

That will make **sum** equal to 285 by adding 0, 100, 90, and 95.

The **0** is the initial value, which is used the first time your function – in this case **+** is called. Once you become more experienced with **reduce()** you'll start to realize why functional developers lean on it so heavily!

To make the behavior of **reduce()** a little clearer, let's start by writing out our example explicitly:

```
let sum = scores.reduce(0) { previousTotal, nextNumber in
    print("Total: \(previousTotal), Next: \(nextNumber)")
    return previousTotal + nextNumber
}
```

That does the same thing as our short version, except with an added **print()** so you can see exactly what happens:

- Total starts as 0, which is the starting value we passed into **reduce()**.
- We add 100 (the first number), so next time the closure is called the total is 100.
- We add 90 (the second number), so next time the total is 190.
- We add 95 (the last number), so the final value returned is 285.

Reducing to a different type

Just like **map()** and **compactMap()**, we can feed a sequence of one type into **reduce()** and get a different type back.

For example, we could use **reduce()** to sum the number of characters used in all the names below:

```
let names = ["Taylor", "Paul", "Adele"]
// your code here
print(count)
```

That should print “15”, which is the sum of the lengths of “Taylor”, “Paul”, and “Adele”. This is a straight integer operation, but you can’t just use + because you want to add the character count rather than appending strings.

You're welcome to give it a try yourself, if you like. Alternatively, you can continue reading to see my solution.

Still here? Okay, here's how it's done:

```
let names = ["Taylor", "Paul", "Adele"]
let count = names.reduce(0) { $0 + $1.count }
print(count)
```

For comparison, here’s the non-functional variant:

```
let names = ["Taylor", "Paul", "Adele"]
var count = 0

for name in names {
    count += name.count
}

print(count)
```


Again, it's not really about the difference in the number of lines of code. Instead, about it's about clarity of intent: with the non-functional variant we're manipulating each item directly, and you need to read the contents of the loop to see what happens to **count** – it's a variable now, so it could be changed in any number of ways.

Alternatives to reduce()

Now that you've met **reduce()**, I want to do something a little odd: I want to give three examples of where **reduce()** is the *wrong* choice for a coding problem.

I'm doing this because I often see reduce squeezed into places where it's a poor choice – maybe folks don't know any better or maybe they are just trying to flex their functional knowledge, it's hard to tell.

First, a common programming task is to check whether all items in an array evaluate to true for a given condition: “is everyone over 18?” or “are all addresses in Canada?” This is converting an array into a single value, but although **reduce()** is possible here it's not a great fit.

As an example, we can check that our array of names contains strings that are over four characters in length:

```
let names = [ "Taylor", "Paul", "Adele" ]
let longEnough = names.reduce(true) { $0 && $1.count > 4 }
print(longEnough)
```

That function starts with an initial value of **true**, then uses **&&** to compare the existing value against a new one. As a reminder:

- false && false == false
- true && false == false
- false && true == false
- true && true == true

Functional Programming

Each time the function runs, it checks whether `$1.count > 4`, for example `"Taylor".count > 4`. That will return true, and our initial value was true, so Swift will run `true && true` to make **true**. The next time it runs, `"Paul".count > 4` will return false, and our previous value was true, so Swift will run `true && false` to make **false**. Finally, `"Adele".count > 4` will return true, and our previous value was false, so Swift will run `false && true` to make **false**.

So, once the code runs `longEnough` will be set to false: we've successfully checked that all strings are at least a certain length.

There's an obvious problem here, which is that if we're checking 1000 items and item 2 fails the test, we don't need to continue. With this functional approach all 1000 items will be checked, but with a regular **for** loop you can **break** as needed.

In this situation, a better choice is Swift's `allSatisfy()` method, which will return false as soon as any item fails the test:

```
let longEnough2 = names.allSatisfy { $0.count > 4 }
```

That won't even check the length of "Adele", because "Paul" fails its test so it can immediately return false.

The second counter-example I want to give is finding the longest name in the array by comparing each name against the longest so far:

```
let names = ["Taylor", "Paul", "Adele"]
let longest = names.reduce("") { $1.count > $0.count ? $1 : $0 }
```

That code certainly works, but you could use `max()` instead, like this:

```
let longest2 = names.max { $1.count > $0.count }
```

In this example, `reduce()` will do one extra comparison because of the initial value of "", however because you're starting with *something* you're always guaranteed a value to work

with. In comparison, **max()** returns an optional value that you then need to unwrap, so which you use will depend on your context.

The key here is to think about your intent, and **max()** is a much clearer function choice than **reduce()** as well as being less code.

The final counter-example is converting an array of arrays into a single array.

To make this work, we need to provide **reduce()** with an initial value of an empty array. Then each time it's called we'll add the new item (another array) to the existing one by using `+`. So, the code is this:

```
let numbers = [
    [1, 1, 2],
    [3, 5, 8],
    [13, 21, 34]
]

let flattened = numbers.reduce([], { $0 + $1 })
print(flattened)
```

Again, that code certainly works, but not only does it have poor performance, it could be more succinctly expressed by using a different approach instead.

For example, you could use **flatMap()** like this:

```
let flattened2 = numbers.flatMap { $0 }
```

You could even use **joined()**, like this:

```
let flattened3 = Array(numbers.joined())
```

If for some reason you simply must use **reduce()** here, perhaps because you have some other work happening in the transformation closure, you should at least know there's a second form

Functional Programming

of **reduce()** called **reduce(into:)**, where the initial value is passed into the closure as **inout**. Rather than returning a new value, your closure should instead update that existing value directly.

In this case, it would mean appending items to the flattened array using **+=** rather than returning new arrays every time.

```
let flattened4 = numbers.reduce(into: []) { $0 += $1 }
```

The moral here is that as your functional arsenal grows, you'll discover several ways of solving the same problem – and that's perfectly fine!

sort()

Sorting is so common that you often don't need to write any special code to use it: if you have an array that holds a primitive data type such as strings or integers, Swift can sort it for you just by calling **sorted()** with no parameters. Things only get more complicated if you want a custom sort order or have a complex data type such as a struct.

Let's tackle custom sort orders first: a student has entered their grades into your app for you to process. An initial implementation might look like this:

```
let scoresString = ["100", "95", "85", "90", "100"]
let sortedString1 = scoresString.sorted()
print(sortedString1)
```

However, the results are unlikely to be what you want: that program will output **["100", "100", "85", "90", "95"]**. This is because the elements are strings, so they are sorted character by character where 1 comes before 9 even though 100 is greater than 90.

A second implementation might be to convert the strings to integers and sort them, like this:

```
let scoresInt = scoresString.compactMap(Int.init)
let sortedInt = scoresInt.sorted()
print(sortedInt)
```

Tip: **compactMap()** is used here rather than regular **map()** because creating an integer from a string returns **Int?**, which means **scoresInt** would end up being **[Int?]** rather than **[Int]**.

A third implementation is to write a custom sort function for our string array, which would convert each value to an integer and compare one with another. This is less efficient than using **compactMap()** then parameter-less **sorted()** because each element would need to be converted to an integer multiple times: Swift would compare 95 to 100, then 85 to 100, then 85 to 95, and so on, with each comparison involving re-creating the integers.

Even though it's less efficient, it's at least worth looking at the code:

```
let sortedString2 = scoresString.sorted {
    if let first = Int($0), let second = Int($1) {
        first < second
    } else {
        false
    }
}

print(sortedString2)
```

Note: This solution has the advantage of leaving its return type the same – we had an array of strings, and this returns an array of strings.

Sorting complex data

When you have an array that contains a custom data type, e.g. a struct with various properties, you need to tell Swift what you want to use for sorting. You can do this in two ways, and I want to explain both so you can use whichever works best for you. Here is some code to provide something for us to work with:

```
struct Person {
    var name: String
    var age: Int
}

let taylor = Person(name: "Taylor", age: 26)
let paul = Person(name: "Paul", age: 44)
let justin = Person(name: "Justin", age: 22)
let adele = Person(name: "Adele", age: 27)

let people = [taylor, paul, justin, adele]
```

sort()

Because **people** doesn't store primitive data types, there's no parameter-less **sorted()** method. So, the first way to sort the **people** array is simply by writing a custom closure for the **sorted()** method, like this:

```
let sortedPeople = people.sorted { $0.name < $1.name }
```

Easy, right? And that solution is perfectly fine when you don't need to repeat the operation more than once. However, a smarter solution is to make your struct conform to the **Comparable** protocol so that you can use Swift's built-in **sorted()** method. It's smarter because it keeps your sort logic centralized, which means changing it one place automatically changes it everywhere.

Note: **Comparable** builds on **Equatable**, but Swift is able to synthesize **Equatable** conformance for using a memberwise comparison.

To make **Person** conform to **Comparable** you first need to add it to the list of protocols, like this:

```
struct Person: Comparable {  
    var name: String  
    var age: Int  
}
```

You then need to add at least one function inside the struct: <. This allow us to compare and sort items, so this is where you'll add your sorting logic.

For this example, I want Swift to sort our structs using the **name** property, so I'll add this function inside the struct:

```
static func <(lhs: Person, rhs: Person) -> Bool {  
    lhs.name < rhs.name  
}
```

If you were dealing with a class rather than a struct, you *wouldn't* get the synthesized

Equatable conformance and would need to add an `==` function like this:

```
static func ==(lhs: Person, rhs: Person) -> Bool {
    lhs.name == rhs.name && lhs.age == rhs.age
}
```

Because Swift lets us compare tuples up to arity six, we can rewrite that as a tuple comparison:

```
static func ==(lhs: Person, rhs: Person) -> Bool {
    (lhs.name, lhs.age) == (rhs.name, lhs.age)
}
```

Now that Swift knows how to sort an array of **Person** instances, you can call **sorted()** without any parameters again:

```
let sortedPeople = people.sorted()
```

Reverse sorting

Sorting items in reverse is more interesting than you might think. As you might imagine, reverse sorting is just a matter of changing `<` for `>` when writing your own code, like this:

```
let sortedString = scoresString.sorted {
    if let first = Int($0), let second = Int($1) {
        first > second
    } else {
        false
    }
}

print(sortedString)
```

So far, so easy. Where the interest comes is when you use the built-in **sorted()** method,

regardless of whether you're storing primitive data or structs that conform to **Comparable**.

Number four in my list of Five Functional Principles was immutable data types, which is something that Swift does in abundance: strings, integers, arrays, structs and more are all immutable, and so represent a fixed value.

Using value types allows Swift to do all sorts of optimization, and reverse sorting is a great example. Take a look at the code below – what do you think it will print?

```
let names = [ "Taylor", "Paul", "Adele", "Justin" ]
print(names)

let sorted = names.sorted().reversed()
print(sorted)
```

The first **print()** will output **["Taylor", "Paul", "Adele", "Justin"]**, which is exactly what you would expect: it's an array containing four strings. But the *second* call to **print()** is what's clever, because it outputs this:

```
ReversedCollection<Array<String>>(_base: [ "Adele", "Justin",
"Paul", "Taylor" ])
```

That is quite clearly something else entirely: that's a *lazy* array that stores the original array as its base value and a function (reversing) alongside. This is because the array is a value type – it can't be changed – so Swift doesn't need to do the actual work of reversing items, and will instead just return values as needed. This means the **reversed()** method runs in constant time – $O(1)$, if you like Big O notation – which is extremely efficient.

If you want a “real” array rather than a **ReversedCollection** – for example if you want to access items individually – you can create an array like this:

```
let sortedArray = Array(names.sorted().reversed())
print(sortedArray)
```

Functional Programming

Alternatively, you can pass a custom sorting function that reverses the regular sort, like this:

```
let sortedArray = names.sorted(by: >)
```

Nice!

Key path expressions as functions

Now that you've seen a handful of useful examples of functional programming, I want to add a really important syntax variation: it's possible to send in a key path as a function for any place where you would write a closure that accepts an item and returns one value from that item.

For example, we could use **map()** to convert an array of strings into an array of the lengths of those strings:

```
let names = ["John", "Paul", "George", "Ringo"]
let nameLengths = names.map { $0.count }
print(nameLengths)
```

The **map()** method expects a closure that accepts one element from the **names** array, and returns some kind of transformed value.

We might be uppercasing it, formatting it somehow, etc, or in this case counting its values. So, we send in a name, and get back some value from that name.

This is a perfect example of where we can use a key path rather than a function. Key paths are uninvoked references to properties: they refer to properties rather than the value of those properties. For example, the key path **\String.count** refers to the **count** property of any string, rather than the number of characters in any one specific string.

So, rather than creating a closure to do the work of reading each string's count, we can instead write this:

```
let names = ["John", "Paul", "George", "Ringo"]
let nameLengths = names.map(\.count)
print(nameLengths)
```

That means "send back the **count** property of each name" – it does exactly the same, just with less typing.

Functional Programming

However, it also does something more important: just like using **map()** helps clarify our intent because we know we can't skip certain items or exit early, using a key path here means it's clear we're just accessing a property rather than doing any custom transformation.

This approach might seem simple, but once you start using it you'll realize how common it is. For example, we could start with some code to store exam results:

```
struct ExamResult {
    var student: String
    var score: Int
}

let results = [
    ExamResult(student: "Eric Effiong", score: 95),
    ExamResult(student: "Maeve Wiley", score: 70),
    ExamResult(student: "Otis Milburn", score: 100)
]
```

We can then use key paths to pull out just the student names or just the student scores:

```
let names = results.map(\.student)
let scores = results.map(\.score)
```

And of course it combines beautifully with other functions, for example using **reduce()** to get the total score:

```
let totalScore = results.map(\.score).reduce(0, +)
```

You'll also commonly see the special key path **\.self**, which refers to the whole object. This is helpful in many places, not least when using **compactMap()** to unwrap the items in an array and remove all nil values:

```
let albums: [String?] = ["Fearless", nil, "Speak Now", nil,
"Red"]
```

```
let result = albums.compactMap(\.self)  
print(result)
```

Function composition

If we use higher order functions and make our functions pure where possible, the combination of these two allows us to build new functions by composing – i.e., putting together – existing functions.

This comes in two forms: simple chaining – i.e., running one pure function and piping its output into another – and true functional composition, which is creating new functions by assembling others like Lego bricks.

Each approach is commonly used, so I'll cover them both here. Chaining really is very simple, so we'll start with that: if you can use **map()**, **compactMap()**, **filter()**, and so on, then chaining them is just a matter of making one call then immediately making another.

Here's some example data for us to work with, which builds an array of tuples containing basic data about some cities:

```
let london = (name: "London", continent: "Europe", population:
8_539_000)
let paris = (name: "Paris", continent: "Europe", population:
2_244_000)
let lisbon = (name: "Lisbon", continent: "Europe", population:
530_000)
let rome = (name: "Rome", continent: "Europe", population:
2_627_000)
let tokyo = (name: "Tokyo", continent: "Asia", population:
13_350_000)

let cities = [london, paris, lisbon, rome, tokyo]
```

To sum the population of all cities, you'd use **reduce()** like this:

```
let totalPopulation = cities.reduce(0) { $0 + $1.population }
```

To sum the population of cities located in Europe, you need to run **filter()** first then chain it to **reduce()** like this:

```
let europePopulation = cities.filter { $0.continent ==
"Europe" }.reduce(0) { $0 + $1.population }
```

To print formatted information about the biggest cities we can use **filter()** to select only cities with high population, call **sorted()** on the result, use **prefix()** to select the first three items, use **map()** to convert the city tuples into a formatted array, then use **joined()** to create a single string:

```
let biggestCities = cities.filter { $0.population >
2_000_000 }.sorted { $0.population >
$1.population }.prefix(3).map { "\( $0.name) is a big city, with
a population of \( $0.population)" }.joined(separator: "\n")
```

Clearly there soon comes a point where chaining functions together becomes hard to read, so you might want to consider adding line breaks like this:

```
let biggestCities = cities.filter { $0.population > 2_000_000 }
    .sorted { $0.population > $1.population }
    .prefix(3)
    .map { "\( $0.name) is a big city, with a population of \
( $0.population)" }
    .joined(separator: "\n")
```

probably anything more than three will cause more confusion than it clears up.

Functional building blocks

Once you have a collection of small, re-usable functions in your code, you can create new functions by assembling your existing ones in specific combinations. This is called function composition, and I hope it will encourage you to ensure your functions can be composed

easily!

Swift does not have a compose operator – i.e., a special operator that makes function composition easy – so most people combine functions like this:

```
let foo = functionC(functionB(functionA()))
```

Some functional programmers create a compose operator, `>>>`, that allows you to rewrite that code a little more clearly:

```
let foo = functionC >>> functionB >>> functionA
```

I don't find that particularly pleasant to read, because you still need to read right to left. Instead, I prefer to create a custom operator that lets you write code like this:

```
let foo = functionA >>> functionB >>> functionC
```

This isn't hard to do using operator overloading. I'll show you the code then explain how it works:

```
precedencegroup CompositionPrecedence {
    associativity: left
}

infix operator >>>: CompositionPrecedence

func >>><T, U, V>(lhs: @escaping (T) -> U, rhs: @escaping (U)
-> V) -> (T) -> V {
    { rhs(lhs($0)) }
}
```

That declares a new operator, `>>>`, which uses three generic data types: `T`, `U`, and `V`. It works with two operands, **lhs** and **rhs**, which are two functions: **lhs** accepts a parameter of type `T` and

returns a value of type **U**, and **rhs** accepts a parameter of type **U** and returns a value of type **V**.

The whole **>>>** function returns a function that accepts a **T** and returns a **V** – it just merges the two steps by calling **rhs(lhs(\$0))**. Both functions must be declared **@escaping** because we're creating and returning a new closure that will call them later.

To make things a bit clearer, imagine our first function accepts an integer and returns a string, and the second function accepts a string and returns an array of strings. That would make **>>>** look like this:

```
func >>>(lhs: @escaping (Int) -> String, rhs: @escaping
(String) -> [String]) -> (Int) -> [String] {
    { rhs(lhs($0)) }
}
```

Hopefully that makes its behavior a bit clearer: by combining the two functions together, the resulting combined function accepts an integer (the input for the first function) and returns an array of strings (the output of the second function).

This operator is just syntactic sugar, but let me give you an example that demonstrates how useful it is. Consider the following code:

```
func generateRandomNumber(max: Int) -> Int {
    let number = Int.random(in: 0...max)
    print("Using number: \(number)")
    return number
}

func calculateFactors(number: Int) -> [Int] {
    (1...number).filter { number.isMultiple(of: $0) }
}

func reduceToString(numbers: [Int]) -> String {
    numbers.reduce("Factors: ") { $0 + String($1) + " " }
}
```

```
}
```

When called in a sequence, that will generate a random number, calculate its factors, then convert that factor array into a single string. To call that in code you would normally write this:

```
let result = reduceToString(numbers: calculateFactors(number:
generateRandomNumber(max: 100)))
print(result)
```

That needs to be read from right to left: **generateRandomNumber()** is called first, then its return value is passed to **calculateFactors()**, then *its* return value is passed to **reduceToString()**. And if you need to write this code more than once, you need to make sure you keep the order correct even when making changes in the future.

Using our new compose operator, there's a better solution: we can create a new function that combines all three of those, then re-use that function however we need. For example:

```
let combined = generateRandomNumber >>> calculateFactors >>>
reduceToString
print(combined(100))
```

There are several reasons I love this approach. First, it's read naturally: generate a random number, calculate its factors, then reduce to a string. Second, it lets you save the combined function for use as many times as you need. Third, you can compose your functions even further: **combined** could be used with **>>>** to make an even bigger function. And fourth, the combined function automatically accepts the same parameter as the first function that was composed, so we use **combined(100)** rather than **combined()**.

I hope this technique gives you another good reason to write small, modular functions: you can write as many as you want, each performing one small task, then create larger functions that combine those together in useful ways.

Lazy functions

Lazy sequences are wonderful things, but I want to briefly mention one particular complexity you need to be aware of: they *can* save you time, but you need to be careful because they can generate huge amounts of work.

To demonstrate this, try the code below:

```
let numbers = Array(1...10)

let lazyFilter = numbers.lazy.filter {
    print("Calculating multiple")
    return $0.isMultiple(of: 2)
}

let lazyMap = numbers.lazy.map {
    print("Doubling number")
    return $0 * 2
}

print(lazyFilter.count)
print(lazyFilter.count)
print(lazyMap[5])
print(lazyMap[5])
```

When you run that, you'll see Swift prints "Calculating multiple" 10 times to calculate the first **print()** statement, then another 10 times to calculate the second **print()** statement.

This is because the result of **filter()** can't be known until all items have been processed – Swift has no idea how many items are in the resulting array until everything has been processed.

On the other hand, "Doubling number" is printed only twice, because Swift can run the **map()** closure on a single element and ignore the rest.

Functional Programming

This is particularly problematic when you chain functions together, because if **filter()** comes first in a lazy chain you will end up having to repeat vast amounts of work.

Functors and monads

Now that you understand some functional programming, I have some good news: you can start dropping words like “functor” and “monad” into your casual office conversations – something I’m sure you’re desperate to do.

First up: a functor is a container that implements **map()**. It doesn’t need to be *called* **map()**, but it does need to do the same thing: convert A into B by applying a transformation. We’ve been using lots of arrays and optionals, both of which are functors. **map()** is *not* the functor – array is the functor, because it implements **map()**.

A monad is a functor that also implements **flatMap()**. Again, arrays and optionals are monads. The formal definition for monads adds a few other rules that I’m going to include below for your reference, but when you’re working with Swift just think about **flatMap()** and you’re most of the way there.

So: a functor is a data type that can be mapped over, and a monad is a functor that also can be flatmapped over. That’s it – go forth and dazzle your friends with your new-found vocabulary! (Actually, please don’t: although the terms give us a shared vocabulary to use when discussing data types, they can also put off people who are new to functional programming.)

Formal definitions

A functor is any data type that can be mapped over using **map()**, as long it abides by two laws:

1. If your map function is the identity function (i.e. it just returns each element without transformation) then your input and output must be the same.
2. If you compose function A and function B to make function C then use the result with **map()**, the result should be the same as mapping first A then B individually.

Both of those laws sound obvious, but that doesn’t mean they aren’t important. As for monads, they include any data type that can be mapped over using **map()** and flatmapped over using **flatMap()**, as long it abides by three laws.

Functional Programming

You don't need to understand (or even be aware of) the three monad laws in order to use them, but if you're curious I'll try to explain. The three monad laws are best demonstrated using code, because honestly it's a bit heavy when you're just learning.

The first law is left identity, and means that if you have:

1. A value, e.g. the number 5.
2. A monad that contains that value, e.g. an array containing the number 5.
3. A function that accepts the same type of value (5) and sends back the same type of monad (an array).

...then calling **flatMap()** on the array should be equal to applying the function directly to the value.

In code:

```
// if you have a value, in this case 5
let myNumber = 5

// and you have a monad containing that value, in this case an
// array containing 5
let myMonad = [myNumber]

// and you have a function that accepts a number and returns
// the same type of monad as we had before (an array)
let doubleNumbers = { [$0 * 2] }

// then calling flatMap on the array...
let result1 = myMonad.flatMap(doubleNumbers)

// should be equal to applying the function directly to the
// value
let result2 = doubleNumbers(myNumber)
```

```
// so, this should print "true"
print(result1 == result2)
```

The second law is right identity, and means that if you have:

1. A value, e.g. the number 5.
2. A monad that contains that value, e.g. an array containing the number 5.
3. A function that accepts the same type of value (5) and sends back the same kind of monad (an array) without transforming the value.

...then calling **flatMap()** with that function on your monad should leave it unchanged.

In code:

```
// if you have a value, in this case 5
let value = 5

// and you have a monad containing that value, in this case an
// array containing 5
let array = [5]

// and you have a function that accepts a number and returns
// the same type of monad as we had before (an array) without
// transforming the value
let wrapInArray = { (value: Int) in [value] }

// then calling flatMap() with that function on your monad
// should leave it unchanged
let flatMapped = array.flatMap(wrapInArray)

// this should print "true"
print(array == flatMapped)
```

The third law is associativity, and means that if you have:

1. A value, e.g. the number 5.
2. A monad that contains that value, e.g. an array containing the number 5.
3. Two functions that can be run on that monad as a chain.

...then it shouldn't matter how those functions are nested.

```
// if you have a value, in this case 5
let anotherNumber = 5

// and you have a monad containing that value, in this case an
// array containing 5
let anotherArray = [anotherNumber]

// and you have two functions that can be run on that monad as
// a chain, in this case one that multiplies by 5 and one by 10
let multiplyBy5 = { [$0 * 5] }
let multiplyBy10 = { [$0 * 10] }

// then it shouldn't matter how those functions are nested
let chained =
anotherArray.flatMap(multiplyBy5).flatMap(multiplyBy10)
let nested = anotherArray.flatMap
{ multiplyBy5($0).flatMap(multiplyBy10) }

// this should print "true"
print(chained == nested)
```

Again, you don't need to understand these laws in order to use monads, so don't be too worried if you understood only part of the code above.

Chapter 8

Object-oriented Programming

Object-oriented programming

Although some people wish it were not the case, object-oriented programming (OOP) is the de facto standard way of building software on many platforms.

It's not *everywhere*: a lot of software is still built in C (which pre-dates OOP), and as you'll see Swift can be written using a similar-but-different approach called protocol-oriented programming (POP).

Even when using POP, Swift developers still commonly rely on the OOP underbelly of Cocoa Touch, which was written for Objective-C – a language so aggressively object-oriented that they baked “object” right into its name. If you need to write apps for iOS or macOS, you need to understand a little OOP in order to interact with the system.

If you intend to interact with other developers, more often than not they will use some amount OOP. If you intend to find examples online, more often than not they will use OOP too – it's unavoidable no matter how much you might love POP.

I'm going to start by giving you a lightning fast introduction to OOP, including examples in Swift. If you already understand terms like “polymorphism”, “encapsulation”, and “static” then you're welcome to move on.

Nouns first

OOP attempts to model programs in terms of entities that have real-world equivalents. So, if you were making an app to track hospital bookings, you'd have an entity for patients, an entity for wards, an entity for the building, an entity for staff, an entity for bookings, an entity for operations, and so on.

These could be classes, like this:

```
class Patient {  
  
}
```

```
class Ward {  
  
}  
  
// etc
```

Or structs, like this:

```
struct Patient {  
  
}  
  
struct Ward {  
  
}  
  
// etc
```

You can be as fine-grained or otherwise as you want, but the goal is to make your entities – usually classes in OOP – reflect actual things that exist in the real world.

This makes OOP easy to pick up: you can just write down a list of the nouns you care about – “patient”, “doctor”, “ward”, and so on – then make a class or struct for each of them.

In theory this breaks up your program into small, re-usable parts that each have specific pieces of responsibility for part of your program, but it also means you make your big architectural decisions very early in your development, and sometimes these can hinder your flexibility later.

Once you have decided your nouns – i.e., created all the classes for things you care about – you add data to them in the form of properties, and add verbs to them in the form of methods.

Methods are effectively identical to functions, but they carry an implicit **self** parameter. For

example, look at the code below:

```
class Ward {  
    func add(patient: Patient) { }  
}
```

The **add(patient)** function belongs to the **Ward** class, so we call it a *method* instead. And behind the scenes, it actually looks like this:

```
class Ward {  
    func add(self: Ward, patient: Patient) { }  
}
```

Instances of classes are *reference types* in Swift, which means they can have multiple simultaneous owners, any of which can manipulate the object. This creates implicit dependencies across your program, which can add complexity and fragility.

Encapsulation

Because classes and structs have their own properties and methods, they can be said to encapsulate functionality: a **Doctor** class can have all the functionality required to work as a doctor in a hospital, and the rest of your program can just call your doctor's methods to make things happen.

Encapsulation is a fundamental concept in OOP, and so access modifiers exist to enforce a set of rules. Continuing with the doctor example, consider this class:

```
class Doctor {
    var name = "Gregory House"
    var currentPatient = "Taylor Swift"
}
```

Note: I'm using default values for the properties to avoid having to write pointless initializers, and this could easily be a struct instead.

That **Doctor** class has a **currentPatient** property that holds the name of the patient they are currently attending to. Now something has gone wrong elsewhere, and Dr House is required to dash off to see another patient as a matter of priority – how should that change happen?

One option is to create a **Hospital** class that can force Dr House to move, something like this:

```
class Hospital {
    func assign(doctor: Doctor, to patient: String) {
        doctor.currentPatient = patient
    }
}
```

However, this is the antithesis of encapsulation: the **Doctor** class should contain all its own functionality, and having the **Hospital** class poke around in a doctor's properties will create spaghetti code. What if the doctor is in the middle of a check up? Or is on vacation?

By placing functionality inside the **Doctor** class you centralize your logic in one place: you can refuse to move if the current task is important, you can refuse to move if you're on vacation, and so on. This means treating objects like black boxes: you don't touch their properties directly.

So, the smarter way to assign a doctor to a new patient is like this:

```
class Doctor {  
    var name = "Gregory House"  
    var currentPatient = "Taylor Swift"  
  
    func assignToPatient(_ name: String) {  
        currentPatient = name  
    }  
}
```

You can then go ahead and add any validation directly into **assignToPatient()**.

When I was much younger, I wrote a small strategy game that was similar in intention – if not completion! – to the game *Civilization*. I was fairly new to programming back then, so I didn't use encapsulation at all.

When a turn ended and I wanted to figure out how much gold each city generated, I added a loop to my **Game** class that went through every city, pulled out its population, then multiplied that by the tax rate. I had to copy and paste that code into the city view so that the player could click on a city and see exactly how much money it generated, but that was okay.

When I let the player create buildings in cities to increase tax revenue, I had to modify my turn end loop: get population, multiply by tax rate, then modify by building improvements. Of course I had to copy that code into my city view calculation too, but that was only a few extra lines.

Then I let players improve the squares around their cities, and some squares brought in more gold than others, so I added that to my turn end loop... but completely forgot to add it to the

city view.

You should be able to see why my approach was bad, and why encapsulation would have made it go away. Duplicating code is often a bad idea, particularly because you need to remember all the places you've made duplication so you can update them all when you make a change – something you will inevitably get wrong.

Once I realized my mistake, I switched tax calculation into a single **calculateTax()** method on my **City** class, and all the logic to calculate a city's gold was contained in just one place.

Going back to the **Doctor** class, just creating an **assignToPatient()** method is insufficient to guarantee encapsulation, because other parts of your code are free to ignore the method and adjust the properties directly.

Remember, that **assignToPatient()** method might contain all sorts of logic to validate whether the doctor could move, as well as other things such as updating their diary – we really need people to use it, rather than adjust **currentPatient** directly.

This is where access modifiers come in: you can tell Swift that only certain parts of your code can read or write properties or call methods. There are four access modifiers you can use:

- **private** means “this can only be used by code in the same class or struct.”
- **fileprivate** means “this can only be used by code in the same Swift file.”
- **internal** means “this can only be used by code in my module”, which is usually your app.
- **public** means “anyone can use this.”

internal is mainly important when you're writing a framework, because it allows any of your code to use a property or method but any external code will be denied access.

When working with private properties, you can make a property fully private just by using the **private** keyword, like this:

```
private var currentPatient = "Taylor Swift"
```


If you want everyone to be able to read the property but make writing it private, you should use **private(set)** instead:

```
private(set) var currentPatient = "Taylor Swift"
```

Once your properties are private, you need to introduce methods to read and write them, commonly called getters and setters. The rest of your app then manipulates your objects using these getters and setters, thus ensuring everyone uses the logic you set down in your methods.

Inheritance

The classes you define in your OOP app will frequently have common properties and methods, but naturally you don't want to duplicate code so instead you can use a technique called inheritance.

This lets you create a new class based on an existing one, and it imports all the properties and methods as it goes.

When you create a new class this way – and it *must* be a class now, because structs don't support inheritance – we call it a subclass, and the class you inherited from is called the super class. You'll also hear subclasses referred to as “child” classes, which makes the class you inherited from the “parent” class.

You can inherit as many times as you want, so you can have grandparent classes and even great-grandparent classes, although those names are rarely used!

Inheritance is another fundamental technique inside OOP, and it's endemic in Apple's older UI frameworks such as UIKit. If you create a basic UIKit project in Xcode, you'll get a class called **ViewController** that inherits from **UIViewController**, but **UIViewController** itself inherits from **UIResponder**, which inherits from **NSObject**, which is the root class in Objective-C – the thing that all other classes must start with.

Inheritance is useful because it lets you import huge amounts of functionality then add to it. For example, nowhere in the **ViewController** class does it say how to behave when the screen is rotated, or how to load a layout from a storyboard, but all that functionality is automatically brought in by inheriting from **UIViewController**.

Let's try inheritance now:

```
class Doctor {  
    var name = "Gregory House"  
    var currentPatient = "Taylor Swift"  
}
```

```
class Surgeon: Doctor { }
```

When we create class **Surgeon** by inheriting from class **Doctor**, we bring with it all of **Doctor**'s properties and methods, which might in turn bring in properties and methods from an **Employee** parent class, and so on.

You can then add your own specific properties and methods to **Surgeon** to do things that doctors don't do, but you can also *override* existing behavior that came with the **Doctor** class – you can say “don't do *that*, but do *this* instead.”

The **Doctor** class defines a couple of basic properties, and the **Surgeon** class inherits from **Doctor** so it gets those properties. This makes sense: everyone has a name regardless of whether they are doctors or surgeons.

If we wanted to allow our surgeons to perform surgery, we could add a **performSurgery()** method to the surgeon class like this:

```
class Surgeon: Doctor {
    func performSurgery(isEmergency emergency: Bool) {
        if emergency {
            print("Okay, let's go!")
        } else {
            print("I'll do it next")
        }
    }
}
```

So far, so easy. But if we need to allow doctors to perform surgery too, things get more complicated. Just adding **performSurgery()** method to the **Doctor** class is easy enough:

```
class Doctor {
    var name = "Gregory House"
    var currentPatient = "Taylor Swift"
```

```
func performSurgery(isEmergency emergency: Bool) {  
    if emergency {  
        print("Okay, let's go!")  
    } else {  
        print("Please find a surgeon")  
    }  
}
```

However, now **Doctor** has a **performSurgery()** method which **Surgeon** inherits, then **Surgeon** declares its own **performSurgery()** method – Swift doesn't know which one to use, so it will refuse to compile.

To fix the problem, you need to make your intention clear. We either:

1. Delete the surgeon's **performSurgery()** method (so that Swift will use the doctor's **performSurgery()** method)
2. Change the surgeon's method so that it has a different signature (e.g. **performSurgery()** rather than **performSurgery(isEmergency emergency: Bool)**)
3. Tell Swift that the surgeon's method overrides the existing method from **Doctor**.

That last option is explicit in Swift, which is not the case in many other languages. To make that happen, we'd need to write **override** before the surgeon's **performSurgery()** method, like this:

```
class Surgeon: Doctor {  
    override func performSurgery(isEmergency emergency: Bool) {  
        if emergency {  
            print("Okay, let's go!")  
        } else {  
            print("I'll do it next")  
        }  
    }  
}
```

```
}  
}
```

Anything you don't override will be inherited from the parent class.

Super methods

As you've seen, when you inherit one class from another, you either use the parent class's methods or you override the ones you want to change in the child. There is a third way, which lets you use the existing functionality of a parent's method and add some custom tweaks in the child.

Here's some simple code we can work with as a sample:

```
class Doctor {
    var name = "Gregory House"
    var currentPatient = "Taylor Swift"

    func assignToPatient(_ name: String) {
        currentPatient = name
    }
}

class Surgeon: Doctor { }
```

By default, that **assignToPatient()** method will be inherited by the **Surgeon** subclass, but we want to make surgeons do something special: when we call **assignToPatient()** on **Surgeon** we first want them to ask some junior doctors for a diagnosis so they have a chance to learn, *then* we want to call the existing **assignToPatient()** method on **Doctor**.

Swift makes this easy using the **super** keyword, which works like **self** but refers to our direct parent class. If the **Surgeon** class overrides **assignToPatient()** to add its own functionality, it can at any time call **super.assignToPatient()** to have the code in the **Doctor** class execute. This means you don't need to duplicate any code: **Surgeon** handles the parts it cares about, then lets **Doctor** do the rest.

In code, it looks like this:

```
class Surgeon: Doctor {
```

```

override func assignToPatient(_ name: String) {
    print(getJuniorDoctorDiagnosis())
    super.assignToPatient(name)
}

func getJuniorDoctorDiagnosis() -> String {
    "He's dead, Jim"
}
}

```

The only complication is when working with **init()**, which has some strict rules. Specifically:

- If you create a class using inheritance, you must call **super.init()** in the child class to allow the parent to construct itself.
- Your subclass must have initialized its own properties fully before calling **super.init()**.
- You can't touch properties from your parent class until you have called **super.init()**.
- You can't call any other methods until you have initialized all your properties, including calling **super.init()**.

To illustrate the correct layout for initializing a subclass, here's a modified **Doctor** class with a simple initializer:

```

class Doctor {
    var name: String
    var currentPatient: String

    init(name: String, currentPatient: String) {
        self.name = name
        self.currentPatient = currentPatient
    }
}

```

If **Surgeon** doesn't introduce any new properties of its own, it will use the existing **Doctor**

initializer.

To make things more interesting, let's allow surgeons to have their own title. A curiosity in the UK is that most doctors are referred to using the title "Doctor", but some senior consultants such as surgeons prefer to be referred to as Mr, Mrs, Miss, or Ms even though they are also fully qualified doctors. So if I were a regular doctor I'd be "Dr Hudson," but if I were a surgeon I would be "Mr Hudson."

Let's modify our **Surgeon** class to allow a custom title to be used, as they are senior doctors. Here's the code:

```
class Surgeon: Doctor {
    var title: String

    init(title: String, name: String, currentPatient: String) {
        self.title = title
        super.init(name: name, currentPatient: currentPatient)
    }
}
```

Notice that **self.title** gets initialized *before* **super.init()** gets called.

If you try to put these two lines the other way around, Swift will refuse to build your code.

If you try to adjust **self.name** before calling **super.init()**, Swift will refuse to build your code.

If you remove **super.init()** or try to call any other methods before using **super.init()**, Swift will refuse to build your code.

Yes, Swift is really strict about these!

Polymorphism

Polymorphism is a word pulled from two Greek words that mean “many shapes”, and in OOP it means an object can be used like its own class or any of the classes it inherits from.

It’s best explained using an example, so below is an array of **Doctor** objects that lists which doctors are available for work right now:

```
class Doctor {
    func performSurgery(isEmergency emergency: Bool) {
        if emergency {
            print("Okay, let's go!")
        } else {
            print("Please find a surgeon")
        }
    }
}

class Surgeon: Doctor {
    override func performSurgery(isEmergency emergency: Bool) {
        if emergency {
            print("Okay, let's go!")
        } else {
            print("I'll do it next")
        }
    }
}

var availableDoctors = [Doctor]()
availableDoctors.append(Doctor())
availableDoctors.append(Doctor())
availableDoctors.append(Doctor())
```

In that code, the **availableDoctors** variable is an array containing **Doctor** objects. But **Surgeon** inherits from **Doctor**, so as far as Swift is concerned it is a **Doctor** too. This means we can add surgeons to the array directly, like this:

```
var availableDoctors = [Doctor]()
availableDoctors.append(Doctor())
availableDoctors.append(Doctor())
availableDoctors.append(Doctor())
availableDoctors.append(Surgeon())
availableDoctors.append(Doctor())
```

Swift knows this is safe, because **Surgeon** inherits from **Doctor** and so either inherits its properties and methods or overrides them.

For example, we can loop over the array and call **performSurgery()** on each item, and Swift will call the correct version of **performSurgery()** depending on each object it finds:

```
for doctor in availableDoctors {
    doctor.performSurgery(isEmergency: false)
}
```

That will output the following:

```
Please find a surgeon
Please find a surgeon
Please find a surgeon
I'll do it next
Please find a surgeon
```

Polymorphism allows the **Surgeon** object to work simultaneously like a **Doctor** and like a **Surgeon**: it can be added to an array of doctors, but use the **Surgeon** version of **performSurgery()** thanks to a technique called dynamic dispatch.

This would *not* work if we declared **availableDoctors** to have the data type **[Surgeon]** because

you can't add a parent class (**Doctor**) to an array of the child class (**Surgeon**).

Final classes

It can be really helpful to inherit from a class and modify the way it works, particularly because it lets you build complex objects quickly. However, some classes need to work a specific way, and it might be confusing and/or risky to modify that behavior.

For example, if you created a banking framework that others could integrate into their apps, would you really want to let them subclass your types and modify the way they worked? Probably not!

Swift lets you mark classes using the **final** keyword, which means “don’t let anyone subclass this.” So, this code will not compile:

```
final class Doctor { }  
class Surgeon: Doctor { }
```

You can also mark individual properties and methods if you want, which allows someone to inherit from your class but not modify specific parts of it.

Because of the way Swift is built, there is a potential performance benefit to declaring things as **final**. Unless you enable whole module optimization when building your project, Swift needs to perform extra checks before making a method call because it needs to check whether it has been overridden somewhere else. Marking properties, methods, or even whole classes as **final** eliminates this check, so your code will run a little faster. (It's worth adding that whole module optimization is enabled by default in Xcode.)

Unless you have specifically designed your class to be inheritable, you'll generally want to mark it **final**.

Static properties and methods

Sometimes you will want to create properties and methods that belong to a whole class or struct rather than a single instance. The difference might seem subtle, but it's important: a regular property or method belongs to an instance of the **Doctor** class, e.g. Gregory House, whereas a class property or method is shared across all doctors.

For example, you might create a method that recites the Hippocratic oath – a medical promise that many physicians take when they graduate medical school. This oath is the same for all doctors, so instead we can create a single method that can be called directly on the **Doctor** class. To do this, just write **static** before your method, like this:

```
class Doctor {  
    static func quoteHippocraticOath() {  
        print("I will prevent disease whenever I can, for  
prevention is preferable to cure.")  
    }  
}
```

```
Doctor.quoteHippocraticOath()
```

When you use **static func**, you're making a class method that also happens to be **final**. If you want to allow subclasses to override the method, you can use **class func** instead of **static func** depending on whether you're working with a class or a struct.

Static properties are also possible, and again just require the **static** keyword:

```
static let latinTitle = "Medicinae Doctor"
```

is-a vs has-a

Protocol-oriented programmers sometimes like to think they have a monopoly on composition, which isn't true: many OOP developers have been encouraging composition rather than inheritance for some time, although it's fair to say the uptake has been patchy.

In OOP, the contrast between inheritance and composition is often called “is-a vs has-a”. A surgeon *is a* doctor, so it makes sense to inherit **Surgeon** from **Doctor**. A surgeon *has a* scalpel, which means the **Surgeon** class could have a property called **scalpel**, but it would be strange if **Surgeon** inherited from **Knife**.

In the unlikely event you find yourself unsure whether to create a new class using inheritance or by adding a property, try using the “is-a / has-a” comparison and see if it helps.

Chapter 9

Protocol-oriented Programming

Protocol-oriented programming

Although many other languages lean heavily on object-oriented programming, Swift developers are more inclined to protocol-oriented programming (POP). This approach is not entirely new, and indeed builds upon Swift techniques you should already be familiar with: protocols, extensions, value types, and more.

This means Swift blends OOP, POP, and functional approaches all in a single language, so it has something for everyone.

POP uses nearly all the same tools you would use with OOP, so you should already have a sound understanding of OOP to continue. It follows the same principles of encapsulation and polymorphism from OOP, but it does away with inheritance.

Instead, POP encourages you to build your app architecture horizontally rather than vertically: you add functionality by adopting protocols rather than inheriting from parent classes. A large part of the approach comes through protocol extensions, which lets you add functionality in smart, focused ways.

Protocol extensions combine two previous technologies, which are – surprise! – protocols and extensions. A protocol is a promise of functionality: the compiler will ensure that anyone who wants to conform to the protocol must implement all the methods you specify.

For example, we could create a **Payable** protocol that requires a **calculateWages()** method to be implemented in any conforming data type:

```
protocol Payable {  
    func calculateWages() -> Int  
}
```

When an object adopts a protocol like **Payable**, it means you can call **calculateWages()** on it and expect to get an integer back. What the actual method does internally is an implementation detail, but that's not a bad thing: it means you can use any data type that conforms to **Payable** without wondering about what their code does, as long as they stick to the API contract – as

long as they implement **calculateWages()** with the specified type signature.

Extensions allow you to add functionality to specific types. For example, you can add a **squared()** method to integers like this:

```
extension Int {
    func squared() -> Int {
        self * self
    }
}
```

That will then apply to all **Int** values, so you can call it like this:

```
let number: Int = 5
let result = number.squared()
```

Protocols have the disadvantage that they contain only declarations of methods; if you try to include any functionality in a protocol, the compiler will refuse to build your code.

Extensions *can* add functionality, but they only apply to specific data types – we just extended **Int**, but not any of the other integer types. So code like this won't work because it uses a **UInt** rather than an **Int**:

```
let number: UInt = 5
let result = number.squared()
```

Protocol extensions combine the two: they let you provide complete default implementations for functions in the same way as extensions, but apply at the protocol level rather than the individual data type level.

So, we could extend *all* integers to add a **squared()** method like this:

```
extension BinaryInteger {
    func squared() -> Self {
```

Protocol-oriented Programming

```
        self * self
    }
}
```

These implementations are *default* implementations, which means they exist as standard but can be overridden if you need. Note that **squared()** returns **Self** with a capital S: this means “return whatever data type I’m being used with.”

Protocol extensions in detail

Protocol extensions let us add functionality to whole classes of data types, either by creating new protocols and extensions, or just by adding a protocol to an existing type. But it does have a few complexities that you must be aware of before starting, so let's get them out of the way now.

First, if you're creating a new protocol you need to separate creating the protocol and adding functionality. For example, if we wanted to create an **Payable** protocol that had a default implementation of the **calculateWages()** method, we would need to create the protocol then separately extend it, like this:

```
protocol Payable {
    func calculateWages() -> Int
}

extension Payable {
    func calculateWages() -> Int {
        10_000
    }
}
```

Second, Objective-C is not aware of any protocol extensions you build, which means you can't create extensions for UIKit protocols like **UICollectionViewDataSource** and expect them to work. To be more specific, you're welcome to *create* the extensions, but UIKit won't be able to see them – they'll have no practical effect.

It also means that if you create any **@objc** protocols your extensions will be ignored by Swift too. For example, in the code below the extension is ignored:

```
@objc protocol Payable {
    func calculateWages() -> Int
}
```

```
extension Payable {  
    func calculateWages() -> Int {  
        10_000  
    }  
}
```

Note: If you're following along in Xcode, please remove the `@objc` – we don't want that!

Third, this separation of declaration (listing the type signature in the protocol) and the definition (writing the actual code in the extension) has meaning: if you do things slightly differently, your code will behave differently and perhaps even unexpectedly.

To test that out, we can create an empty **Surgeon** struct, then extend it so that it uses our **Payable** protocol, like this:

```
struct Surgeon { }  
  
extension Surgeon: Payable { }
```

Just doing that is enough for surgeons to have the **calculateWages()** method – we retroactively declare that the **Surgeon** struct builds on the **Payable** protocol, and don't need to add any custom code to get all the **Payable** functionality for free. So this will print out “10000”:

```
let gregory = Surgeon()  
print(gregory.calculateWages())
```

Remember, the **calculateWages()** method we added to **Payable** is just a default implementation, so any conforming data type can write its own version that does something different, like this:

```
extension Surgeon: Payable {  
    func calculateWages() -> Int {  
        20_000  
    }  
}
```

```

    }
}

```

Here is where things get complicated: when you list a method in your protocol, you're making it available as a customization point for people who want to write their own implementations.

If you *don't* list a method in your protocol, but *do* provide a default implementation, that implementation might be used even if it has been overridden later on. What matters is the data type you're using. Yes, this is confusing, but I hope the code below will explain:

```

protocol Payable {
    // func calculateWages() -> Int
}

```

```

extension Payable {
    func calculateWages() -> Int {
        10_000
    }
}

```

```

struct Surgeon { }

```

```

extension Surgeon: Payable {
    func calculateWages() -> Int {
        20_000
    }
}

```

```

let gregory = Surgeon()
print(gregory.calculateWages())

```

```

let doogie: Payable = Surgeon()
print(doogie.calculateWages())

```

The protocol is empty because I commented out its declaration of **calculateWages()**. There's a default implementation provided for this in the **Payable** extension, and another implementation inside the **Surgeon** extension. They both return different values: regular employees get paid 10000, but surgeons get paid 20000.

Take a look at the last four lines, because they print different things: even though both **gregory** and **doogie** are instances of the **Surgeon** struct, **doogie** is being stored as an **Payable**.

This is polymorphism, but it *matters*: when there is no protocol declaration of a method, Swift will decide which method to call based on the data type it sees. In this case, **gregory** will use the **calculateWages()** method of **Surgeon**, whereas **doogie** will use the method of **Payable**. If you uncomment the **calculateWages()** declaration in the **Payable** protocol, both **gregory** and **doogie** will get paid the same.

So, including a method in a protocol declaration is the signal to Swift that we want to allow any default implementation to be overridden in the situation when we refer to an instance by one of its protocols rather than its specific type.

My advice: Until you're comfortable with this distinction, always declare your methods in your protocols.

Thinking horizontally

One of the most important characteristics of protocol-oriented programming is that it encourages you to build your app architecture horizontally rather than vertically,” but what does that really mean?

“Vertical architecture” is inheritance: creating a base class, then adding functionality through subclassing until you get to your most specific data types. This can easily lead to a place where your types grow really bloated with functionality and storage.

In comparison, “Horizontal architecture” is using whichever protocols make sense to add specific pieces of functionality. You can still use inheritance if you want, but it’s likely you won’t need it. As a result, POP developers prefer to rely on structs and enums rather than classes, which brings its own set of benefits.

If you’re an experienced developer, POP might sound like multiple inheritance – a technique that let you create classes by inheriting from more than one thing at a time, thus creating monsters like **Pegasus** by inheriting from the classes **Horse** and **Bird** at the same time.

There is a subtle difference: protocols let you add only methods and computed properties, not stored properties. This means they can never add more *state* to your data types, so they keep complexity low.

When working with inheritance, you can inherit from exactly one superclass, and doing so brings with it all the properties and methods of that superclass whether or not you need them. When you switch to protocols, you can adopt as many protocols as you need, and each one can add one or more methods that provide particular functionality.

So, rather than your superclasses dictating what methods their child classes will receive – a process that could easily result in complex, fragile hierarchies – the *child* classes dictate what functionality they want to bring in by selectively adding the protocols they want.

Does your **Surgeon** class need to inherit from **Doctor**, which itself inherits from **Employee**, just to get paid? With POP, all you need to do is add a **Payable** protocol that **Surgeon**

Protocol-oriented Programming

conforms to, and you get all the functionality immediately. In Swift, the advantage of POP is extended even further, because protocols can be adopted by enums as well as classes and structs, so you can share functionality wherever it's needed.

Splitting up your functionality into distinct logical components conveys two further benefits.

First, your code becomes significantly more testable: when each protocol handles one specific behavior (e.g. validating user input), you can write tests for that behavior and be confident that you have 100% coverage.

Second, you can easily change your mind about your architecture long after you designed it just by adjusting the protocols you're using – it's significantly easier than trying to change class **C** when you know that **D**, **E**, **F**, and **G** all inherit from it.

POP in practice

At this point you should understand the core tenets of OOP – most of which apply in POP – as well as how protocol definitions work and why horizontal architecture is important.

Now it's time to look at a specific example of POP in action; I'm going to define six protocols with accompanying extensions. To keep things simple, I'm only going to give each protocol a single method.

Here's the code:

```
protocol Payable {
    func calculateWages() -> Int
}

extension Payable {
    func calculateWages() -> Int {
        10_000
    }
}

protocol ProvidesTreatment {
    func treat(name: String)
}

extension ProvidesTreatment {
    func treat(name: String) {
        print("I have treated \(name)")
    }
}

protocol ProvidesDiagnosis {
    func diagnose() -> String
}
```

Protocol-oriented Programming

```
}

extension ProvidesDiagnosis {
    func diagnose() -> String {
        "He's dead, Jim"
    }
}

protocol ConductsSurgery {
    func performSurgery()
}

extension ConductsSurgery {
    func performSurgery() {
        print("Success!")
    }
}

protocol HasRestTime {
    func takeBreak()
}

extension HasRestTime {
    func takeBreak() {
        print("Time to watch TV!")
    }
}

protocol NeedsTraining {
    func study()
}
```

```
extension NeedsTraining {
    func study() {
        print("I'm reading a book")
    }
}
```

Those six protocols give us enough data to start modeling some staff members in a hospital. Specifically, we're going to define four roles: receptionist, nurse, doctor, and surgeon.

In a real hospital the lines between these roles aren't black and white – roles like nurse practitioner exist that can diagnose and prescribe treatment, for example – but I'm trying to keep it simple.

So, with the protocols in place, we can create four structs:

```
struct Receptionist { }
struct Nurse { }
struct Doctor { }
struct Surgeon { }
```

And now for the best bit: to give those roles the functionality we defined in our protocols, we just need to decide who should get what. We don't need to create an inheritance tree – we just need to pick which protocols each role needs.

For example, the **Receptionist** role should probably adopt the protocols **Payable**, **HasRestTime**, and **NeedsTraining**, like this:

```
extension Receptionist: Payable, HasRestTime, NeedsTraining { }
```

The **Nurse** role needs the same roles as **Receptionist** as well as **ProvidesTreatment**:

```
extension Nurse: Payable, HasRestTime, NeedsTraining,
ProvidesTreatment { }
```

Protocol-oriented Programming

The **Doctor** role needs the same roles as **Nurse** with the addition of **ProvidesDiagnosis** because they can diagnose as well as treat:

```
extension Doctor: Payable, HasRestTime, NeedsTraining,  
ProvidesTreatment, ProvidesDiagnosis { }
```

The **Surgeon** role is a little different: the surgeons in our example hospital won't provide treatment (they have junior doctors to do that), but they can provide diagnosis as well as conduct surgery.

So, they look like this:

```
extension Surgeon: Payable, HasRestTime, NeedsTraining,  
ProvidesDiagnosis, ConductsSurgery { }
```

That completes our line up. Notice how we can be really specific about what each role needs, rather than try to craft a hierarchy?

If we had made **Surgeon** inherit from **Doctor** it would have been given the ability to provide treatment as part of that inheritance, and we would need to override that functionality to try to opt out.

However, our current solution is a bit messy: all staff members adopt the protocols **Payable**, **HasRestTime**, and **NeedsTraining**, so there's a lot of duplication right now.

To make things easier to read, we can create a new **Employee** protocol that groups together **Payable**, **HasRestTime**, and **NeedsTraining**, then make our four roles adopt **Employee** rather than the individual protocols. In code, it looks like this:

```
protocol Employee: Payable, HasRestTime, NeedsTraining { }  
  
extension Receptionist: Employee { }  
extension Nurse: Employee, ProvidesTreatment { }  
extension Doctor: Employee, ProvidesDiagnosis,
```

```
ProvidesTreatment { }  
extension Surgeon: Employee, ProvidesDiagnosis, ConductsSurgery  
{ }
```

Apple's advice for POP is this: "don't start with a class, start with a protocol." And that's exactly what we've done here: we started by defining the various behaviors we wanted to represent in our app, then created structs that adopted those behaviors to bring it all to life.

Constrained extensions

Swift makes it easy to extend a protocol to add new functionality. For example, we could add a `checkInsurance()` method to all hospital roles that adopt the **Employee** protocol, like this:

```
protocol Employee {  
}  
  
extension Employee {  
    func checkInsurance() {  
        print("Yup, I'm totally insured!")  
    }  
}
```

But that's a pretty big change to make, and it brings back the same problem that inheritance gave us: everyone gets this method whether they want it or not.

Sometimes this just adds extra unnecessary complexity, but other times it will stop your code from building because the method you want to add doesn't work on every **Employee**.

Swift's solution to this problem is called constrained extensions, and allows us to specify that an extension applies only to certain kinds of protocols. In our current instance, we only need staff to have insurance if they provide treatment to patients, so we can rewrite the above code like this:

```
protocol ProvidesTreatment {  
}  
  
extension Employee where Self: ProvidesTreatment {  
    func checkInsurance() {  
        print("Yup, I'm totally insured")  
    }  
}
```

Using this approach, employees that don't provide treatment, or people who provide treatment that aren't employees (e.g. a doctor who is on loan from another hospital to cover vacation time), won't be given the **checkInsurance()** method.

Things get more complicated when you want to handle sub-types of data, for example if you want to target any kind of collection that stores strings. To make this work, you need to specify a **where** clause to filter by **Element** where the element type matches something specific. You can use **Element** inside the method as a placeholder for the matching data type, or you can use specific data types if you prefer.

For example, the extension below targets collections that store integers, and adds the ability to count how many odd and even numbers are in the collection:

```
extension Collection where Element: BinaryInteger {
    func countOddEven() -> (odd: Int, even: Int) {
        var even = 0
        var odd = 0

        for val in self {
            if val.isMultiple(of: 2) {
                even += 1
            } else {
                odd += 1
            }
        }

        return (odd, even)
    }
}
```

Example extensions

Swift’s extension system is remarkably powerful: you can extend a specific type or whole groups of types all at once, plus you can build constrained extensions that work on specific subsets.

In the event that you create two methods of the same signature and try to give them both to a single data type, Swift will automatically choose the one that is the most constrained – i.e., the most specific. So, an extension for **Int** would be considered more specific than an extension for all **BinaryInteger** data types.

I want to demonstrate Swift extensions by building some examples all using common protocols. I’ve written ten examples in all, built around the following four protocols:

- The **Equatable** protocol covers things that can be compared using `==` and `!=`.
- The **Comparable** protocol covers things that can be compared using `<` and `>`.
- The **Collection** protocol covers things that store multiple values, like arrays.
- The **BinaryInteger** protocol covers things that hold round numbers like 1, 2, and 128.

I’ll walk you through all ten examples, but if you want to get the maximum benefit I’d like you to attempt the even-numbered examples yourself before reading my answers.

I’ll be using the odd-numbered examples to introduce new concepts, and the even-numbered examples to help you try things out for yourself.

Example one: Squaring integers

Let’s start by creating an extension for the **Int** type that will calculate the value squared. This is a regular extension that we’ve looked at previously, so this should just be a refresher:

```
extension Int {  
    func squared() -> Int {  
        self * self  
    }  
}
```



```
}
```

```
let i: Int = 8
print(i.squared())
```

Because we extended the **Int** type, that extension won't be visible in other similar data types. For example, this won't compile:

```
let j: UInt = 8
print(j.squared())
```

To make that work, we need to extend the **BinaryInteger** protocol, which will affect all types of integer – **Int**, **UInt**, **Int64**, etc:

```
extension BinaryInteger {
    func squared() -> Self {
        self * self
    }
}
```

Notice the **Self** with a capital S – it means “whatever data type we’re being used with,” which is different to “self” with a lowercase s.

So, **Self** means **Int** when working with that data type, or **Int64** when working with that instead, whereas **self** (lowercase S!) means the actual number being stored.

Example two: Clamping integers

Now it's time for you to try creating an extension for yourself. I'll provide the answer below, but I do want you to try solving it yourself, to make sure you understand how extensions work.

Here's your goal: create an extension for all integer types that returns the value clamped between two values. So, if you feed it the number 15 and ask it to clamp between 1 and 10,

Protocol-oriented Programming

you'll get back 10. Here's some example input and output so you can test your code:

- Minimum 5, maximum 10, input 8 should output 8.
- Minimum 5, maximum 10, input 3 should output 5.
- Minimum 5, maximum 100, input 800 should output 100.

Remember, you're extending **BinaryInteger**, so your input number will be available inside **self**. You just need to write the method that accepts low and high values for clamping.

Please try now, then check my answer below to see how you got on.

This space left intentionally blank, so you don't see my answer by accident.

Still here?

Okay, here's my answer:

```
extension BinaryInteger {  
    func clamp(low: Self, high: Self) -> Self {  
        min(max(self, low), high)  
    }  
}
```

Don't worry if your answer was different to mine – there are lots of ways of solving this problem, and as long as yours is easy to read and efficient to run it's a good solution.

Example three: Matching value types

Swift uses value types almost everywhere, which means you can compare most things of the same type using a straight `==`.

This isn't magic; instead, it's the **Equatable** protocol providing that `==` functionality: everything that conforms to this protocol can be checked for equality with another item of the same type.

We're going to add an extension to **Equatable** so that one value can be checked against an array of values of the same type in one method. So, **2.matches(array: [2, 2, 2, 2])** will return true, but **2.matches(array: [2, 2, 2, 3])** will return false.

Here's the code:

```
extension Equatable {
    func matches(array items: [Self]) -> Bool {
        for item in items {
            if item != self {
                return false
            }
        }

        // if we made it this far, all items match!
        return true
    }
}
```

We can also implement that much more concisely using **allSatisfy()**, like this:

```
extension Equatable {
    func matches(array items: [Self]) -> Bool {
        items.allSatisfy {
            $0 == self
        }
    }
}
```

Example four: Comparing arrays

You've seen how to create an extension for **Equatable** that works on data types that can be checked for equality using **==**. Now I'd like you to try your hand at writing an extension for

Comparable, which works on data types that can be ordered using < and >.

You're probably thinking of numbers right now, and it's true that they conform to **Comparable**. But so do strings: "alpha" is ordered before "omega" because they can be sorted lexicographically.

Your job is to create a method that returns true if a value is less than all other values in an array. Here are some example calls and their result:

- Running `5.lessThan(array: [6, 7, 8])` should return true.
- Running `5.lessThan(array: [5, 6, 7])` should return false.
- Running `"cat".lessThan(array: ["dog", "fish", "gorilla"])` should return true.

This is an even-numbered example, so I would like you to try it yourself before reading my example. You need to use the **Comparable** protocol, but other than that my code from example three should be enough to get you through.

Here's a little gap so you don't accidentally see my answer.

All done? Here's my solution:

```
extension Comparable {
    func lessThan(array items: [Self]) -> Bool {
        for item in items {
            if item <= self {
                return false
            }
        }

        return true
    }
}
```

Again, `allSatisfy()` does that really concisely:

```
extension Comparable {
    func lessThan(array items: [Self]) -> Bool {
        items.allSatisfy {
            $0 > self
        }
    }
}
```

Example five: Rewriting contains()

You've been extending specific protocols so far by using **BinaryInteger**, **Equatable**, and **Comparable**, but I want to take things a step further now: I want to create an extension for *collections* that hold specific data types.

More specifically, we're going to recreate the **contains()** method that exists in Swift's standard collections, and I think you'll find it's straightforward.

The method needs to loop over every item in an array until it finds one that matches – i.e., returns true for `==` - and input object. This means using the **Equatable** protocol again, which is what guarantees us support for `==`.

This time we'll be extending the **Collection** protocol, but we need to constrain it so that it applies only to collections that contain equatable elements. In Swift this is done by adding a constraint to **Element** – here's the code:

```
extension Collection where Element: Equatable {
    func myContains(element: Element) -> Bool {
        for item in self {
            if item == element {
                return true
            }
        }
    }
}
```

```
        return false
    }
}
```

Example six: Fuzzy array matching

Now that you've seen how to constrain protocols based on the element type that a collection holds, your next challenge is to create a method that returns true if two arrays contain the same elements, regardless of their order.

So:

- Using `[1, 2, 3].fuzzyMatches([1, 2, 3])` will return true
- Using `[1, 2, 3].fuzzyMatches([3, 2, 1])` will return true
- Using `[1, 2, 3].fuzzyMatches([1, 2])` will return false
- Using `[1, 2, 3].fuzzyMatches([1, 2, 3, 1])` will return false

Although this can be solved without adding support for **Comparable**, I wouldn't recommend it.

Again, please give it a try yourself.

Done? Here's my solution:

```
extension Collection where Element: Comparable {
    func fuzzyMatches(_ other: Self) -> Bool {
        let usSorted = self.sorted()
        let otherSorted = other.sorted()
        return usSorted == otherSorted
    }
}
```

As you can see, I'm taking advantage of the fact that Swift's own `==` operator can directly

compare two arrays, so all we need to do is ensure they are in the same order before doing the comparison!

Note: I wrote **self.sorted()** rather than just **sorted()** for the sake of clarity. The **self.** isn't needed, but it makes the intent a bit clearer.

Example seven: average string length

The previous two examples extended protocols where their items *also* conformed to a protocol. We can also extend protocols where items are a specific data type by using **==** for the constraint rather than **:.**

To demonstrate this, we're going to extend the **Sequence** type for sequences that hold strings (a specific data type) by writing a method that returns the average length of all the strings. This highlights a curiosity in the way Swift's sequences work, so pay attention!

Here's the code:

```
extension Sequence<String> {
    func averageLength() -> Double {
        var sum = 0
        var count = 0

        for item in self {
            sum += item.count
            count += 1
        }

        return Double(sum) / Double(count)
    }
}
```

The algorithm itself is just a mean average, which is straightforward enough. But notice how I keep track of a **count** variable inside the loop – how come?

Well, Swift's sequences don't have a built-in **count** property. It's easy to think of sequences as being things like **[1, 2, 3, 4, 5]** or dictionaries of strings and so on, but in practice they can be a far wider range of things.

In fact, as long as the iterator keeps producing values, a sequence can effectively be infinite. For example, you could write an iterator that outputs the Fibonacci sequence without having to store the entire sequence – it just needs to store a couple of previous values and have an iterator that puts them together.

Example eight: counting integers

Now you've seen how to constrain collections to a specific element type, it's time for another problem you need to solve: create an extension that accepts an array of integers and returns how many times the digit 5 appears in any number.

For example:

- The array **[5, 3, 5, 1, 5]** should return 3.
- The array **[5, 15]** should return 2.
- The array **[5, 15, 55, 555]** should return 7.
- The array **[555, 555, 555]** should return 9.

So, you're counting how many times the digit 5 appears in any number, i.e. 5, 55, 515, and so on.

This is an even-numbered example, so I would like you to try it yourself. Remember, you can use **==** to constrain to a specific data type, in this case **Int**, or use shorthand syntax such as **Collection<Int>**.

This space should stop you from seeing my solution by accident.

All done? Here's my solution:

```
extension Collection<Int> {
```



```

func numberOf5s() -> Int {
    var count = 0

    for item in self {
        let str = String(item)

        for letter in str {
            if letter == "5" {
                count += 1
            }
        }
    }

    return count
}

```

If you were feeling fancy, we could implement the same thing using **reduce()** and **count(where:)**:

```

extension Collection<Int> {
    func numberOf5s() -> Int {
        reduce(0) {
            $0 + String($1).count { $0 == "5" }
        }
    }
}

```

Example nine: De-duping an array

You've seen how to extend the **Collection** protocol where its elements are a specific type, but you can also do the opposite: extend a *fixed* collection type where its elements conform to a

particular protocol.

For example, we could create a method that de-dupes an array. We already implemented our own **myContains()** method and in doing so learned that **Equatable** is what provides the **==** operator for a given data type.

So, in this example we can require that our array holds elements that conform to **Equatable**, and in doing so we'll automatically get the built-in **contains()** method.

Time for some code, and again the algorithm isn't too hard: create an empty array that will hold our return value, then loop through every item in **self** – i.e., the current collection. If the result array doesn't already contain the item we should add it, otherwise just move on. Finally, return the result array.

Here's the code:

```
extension Array where Element: Equatable {
    func uniqueValues() -> [Element] {
        var result = [Element]()

        for item in self {
            if result.contains(item) == false {
                result.append(item)
            }
        }

        return result
    }
}
```

There are two things to note there. First, we're extending the **Array** type specifically, so it's just a group of values. Second, I declared the result array to be of type **[Element]**, which means it will match whatever the input array held.

Example ten: Array is sorted

At this point you should be fairly comfortable with extensions, so here's a simple challenge to ensure you're able to extend a specific data type where elements conform to a protocol. Your final mission: write a method that returns true if an array is sorted.

So:

- The array `[1, 3, 5]` should return true.
- The array `[5, 3, 1]` should return false.

This shouldn't trouble you – go ahead and try now before reading my solution below.

Finished? Here you go:

```
extension Array where Element: Comparable {  
    func isSorted() -> Bool {  
        self == self.sorted()  
    }  
}
```

Again, that takes advantage of Swift's built in `==` operator for **Comparable** arrays – this stuff really is powerful, I think you'll agree.

That's the last of my examples, and I hope you've managed to complete the even-numbered ones yourself without too much pain.

Don't be worried if your solution was more complex than mine, or even if yours was much simpler – there are lots of ways of solving each of these, and as long as your code is clear and efficient then it's a good solution.

Afterword

Thank you!

You've made it to the end of this book: I hope you feel much more confident in your skills than when you started it, and I hope you also had fun too.

But while you've finished the book, you *haven't* finished learning. Programming is an art, so you shouldn't spend all your time sharpening your pencil when you should be drawing. That means you need to put what you've learned into practice, both soon and often. Remember, just joining a gym isn't enough to make you fit!

And there's more to learn from me too, whether that's more of my books, or joining Hacking with Swift+ to learn how to build the Ultimate Portfolio App, score discounts on my books, and get access to my monthly app building livestreams.

The most important thing to say, though, is thank you, even if this book is the only thing of mine you ever buy. Your support has directly contributed to me being able to share more knowledge for free on **hackingwithswift.com**, where the world learns Swift.

And so, one last time: thank you.

What next?

Level up your skills with the the most popular books from Hacking with Swift:

- Pro Swift
- Pro SwiftUI
- Swift Design Patterns
- Testing Swift
- Swift Coding Challenges

Or better yet, try my bundles to get a great selection of knowledge and also save money: the Swift Power Pack, the Swift Platform Pack, and the Swift Plus Pack can really help take your skills to the next level.

You can buy all these and more at **<https://www.hackingwithswift.com/store>**.

Hacking with Swift+

Make the ultimate investment in your iOS career, with HWS+:

- Build the Ultimate Portfolio App
- Downloadable projects and learning challenges
- 20% discount on all my books year-round
- Access to monthly app building livestreams
- Support Hacking with Swift so I can share more learning
- Free gifts and more!

Find out more and subscribe at [**https://www.hackingwithswift.com/plus**](https://www.hackingwithswift.com/plus).